MET's Institute of Engineering

Bhujbal Knowledge City, Adgaon, Nashik.

Department of Computer Engineering

"Linear Data Structures, searching and sorting"

Prepared By

Prof. Anand N. Gharu

(Assistant Professor)

Computer Dept.

CLASS : SE COMPUTER 2019

20 July 2025

SUBJECT : DS (SEM-I)

UNIT : II

Note: The material to prepare this presentation has been taken from internet and are generated only for students reference and not for commercial use.

Study material provided by: Vishwajeet Londhe

Join Community by clicking below links

Telegram Channel

https://t.me/SPPU_TE_BE_COMP

(for all engineering Resources)

WhatsApp Channel

(for all Engg & tech updates)

https://whatsapp.com/channel/0029ValjFriICVfpcV9HFc3b

Insta Page

(for all Engg & tech updates)

https://www.instagram.com/sppu_engineering_update

SYLLABUS

Overview of Array, Array as an Abstract Data Type, Operations on Array, Storage Representation, Multidimensional Arrays[2D, nD], Sparse matrix representation using 2D

Searching: Sequential Search/Linear Search, Binary Search, Fibonacci Search, and Indexed Sequential Search.

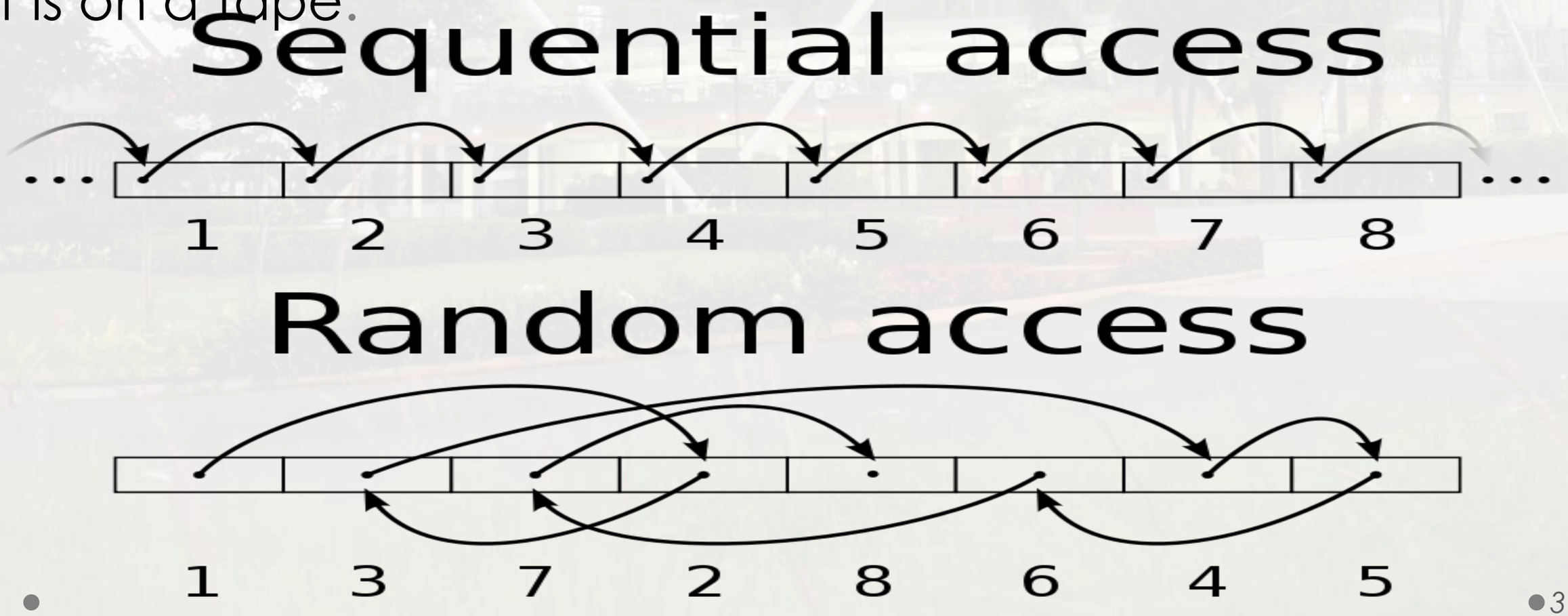
Sorting: Concepts- Stability, Efficiency, and Number of Passes, Internal and External Sorting, Bubble sort, Insertion Sort, Selection Sort , Quick Sort, Merge sort

Case Study : Social Network Adjacency Matrix Representing friendship connections among millions of users.

SEQUENTIAL ORGANIZATION

- **Definition :**

"In computer science, **sequential** access means that a group of elements (such as **data** in a memory array or a disk file or on magnetic tape **data** storage) is accessed in a predetermined, ordered sequence.**Sequential** access is sometimes the only way of accessing the **data**, for example if it is on a tape."



Linear Data Structure Using Sequential Organization

- **Definition :**

"The data structure where data items are organized sequentially or linearly one after another is called as **Linear Data Structure**"

A **linear data structure**traverses the **data** elements sequentially, in which only one **data** element can directly be reached. Ex: Arrays, Linked Lists.

Sequential Organization

In a sequential organization, the elements or components of the system are arranged in a specific order, and each element depends on the previous one.

Sequential organization has the advantage of fast access to elements, but has the disadvantage of slow insertion and deletion of elements, especially when the data is stored in a contiguous block of memory.

Advantages of Sequential Organization

- Easy to understand
- Simple to implement
- Allow for fast traversal and access
- Random access is possible
- Suitable for static data structure
- Can be used for other data structure like Array, Stack etc

Disadvantages of Sequential Organization

- Fixed size structure
- Not suitable for dynamic data structure.
- Insertion and deletion is difficult
- Only linear sequential list is possible
- Fixed length data in each element.
- Not suitable for large list

Array

- **Definition :**

"An array is a finite ordered **collection of homogeneous data elements** which provides **direct access** (or random access) to any of its elements.

An array as a **data structure** is defined as a set of pairs (index,value) such that with each index a value is associated.

- **index** — indicates the location of an element in an array.
- **value** - indicates the actual value of that data element.

Declaration of an array in 'C++':

- **int Array_A[20];**

"

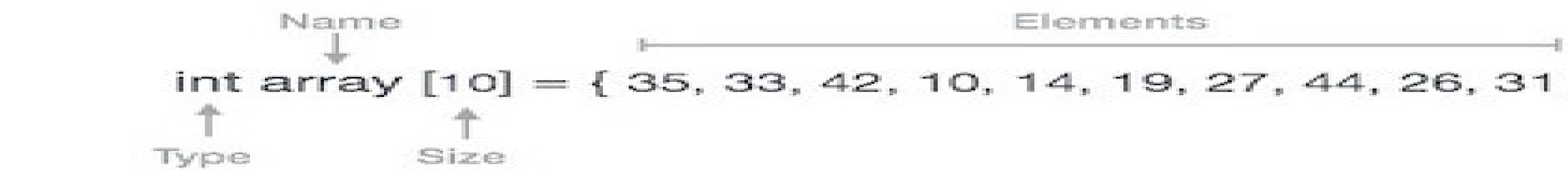
Array

- **Array Representation**
- Arrays can be declared in various ways in different languages. For illustration, let's take C array declaration.

- **Index** — Each location of an element in an array has a numerical index, which is used to identify the element.

Array Representation

Arrays can be declared in various ways in different languages. For illustration, let's take C array declaration.



Arrays can be declared in various ways in different languages. For illustration, let's take C array declaration.

- Arrays can be declared in various ways in different languages. For illustration, let's take C array declaration.
- As per the above illustration, following are the important points to be considered.

Array

- **Array Representation**
- Index starts with 0.
- Array length is 10 which means it can store 10 elements.
- Each element can be accessed via its index. For example, we can fetch an element at index 6 as 9.
- Basic Operations
- Following are the **basic operations** supported by an array.
- **Traverse** − print all the array elements one by one.
- **Insertion** − Adds an element at the given index.
- **Deletion** − Deletes an element at the given index.
- **Search** − Searches an element using the given index or by the value.
- **Update** − Updates an element at the given index.

Advantage of Array Data Structure

1. Array size is fixed. which means you cannot add/delete elements after creation.
2. Array is homogenous. cannot store values of different data types in a single array.
3. Array is Contiguous blocks of memory.
4. Another disadvantage of arrays is that they don't support random access.
5. Insertion and deletion are not easy in Array.
6. Wastage of memory space if all array elements are not used.
7. Arrays have limited functionality compared to other data structures.

Applications of Arrays

Below are some applications of arrays.

1. **Storing and accessing data:** Arrays are used to store and retrieve data in a specific order. For example, an array can be used to store the scores of a group of students, or the temperatures recorded by a weather station.

2. **Sorting:** Arrays can be used to sort data in ascending or descending order. Sorting algorithms such as bubble sort, merge sort, and quicksort rely heavily on arrays.

3. **Searching:** Arrays can be searched for specific elements using algorithms such as linear search and binary search.

Applications of Arrays

Below are some applications of arrays.

4. **Matrices:** Arrays are used to represent matrices in mathematical computations such as matrix multiplication, linear algebra, and image processing.

5. **Stacks and queues:** Arrays are used as the underlying data structure for implementing stacks and queues, which are commonly used in algorithms and data structures.

6. **Graphs:** Arrays can be used to represent graphs in computer science. Each element in the array represents a node in the graph, and the relationships between the nodes are represented by the values stored in the array.

Abstract Data Type as an Array

- C/C++ provides Array as a basic data structure.
- Representation of data is defined by the language compiler itself. But the operations on the data are not defined by the compiler.
- We have to implement or provide the operations on Array data structure. So, data structure array and the set of operations together we can call it as Abstract Data Type (ADT)

Abstract Data Type as an Array

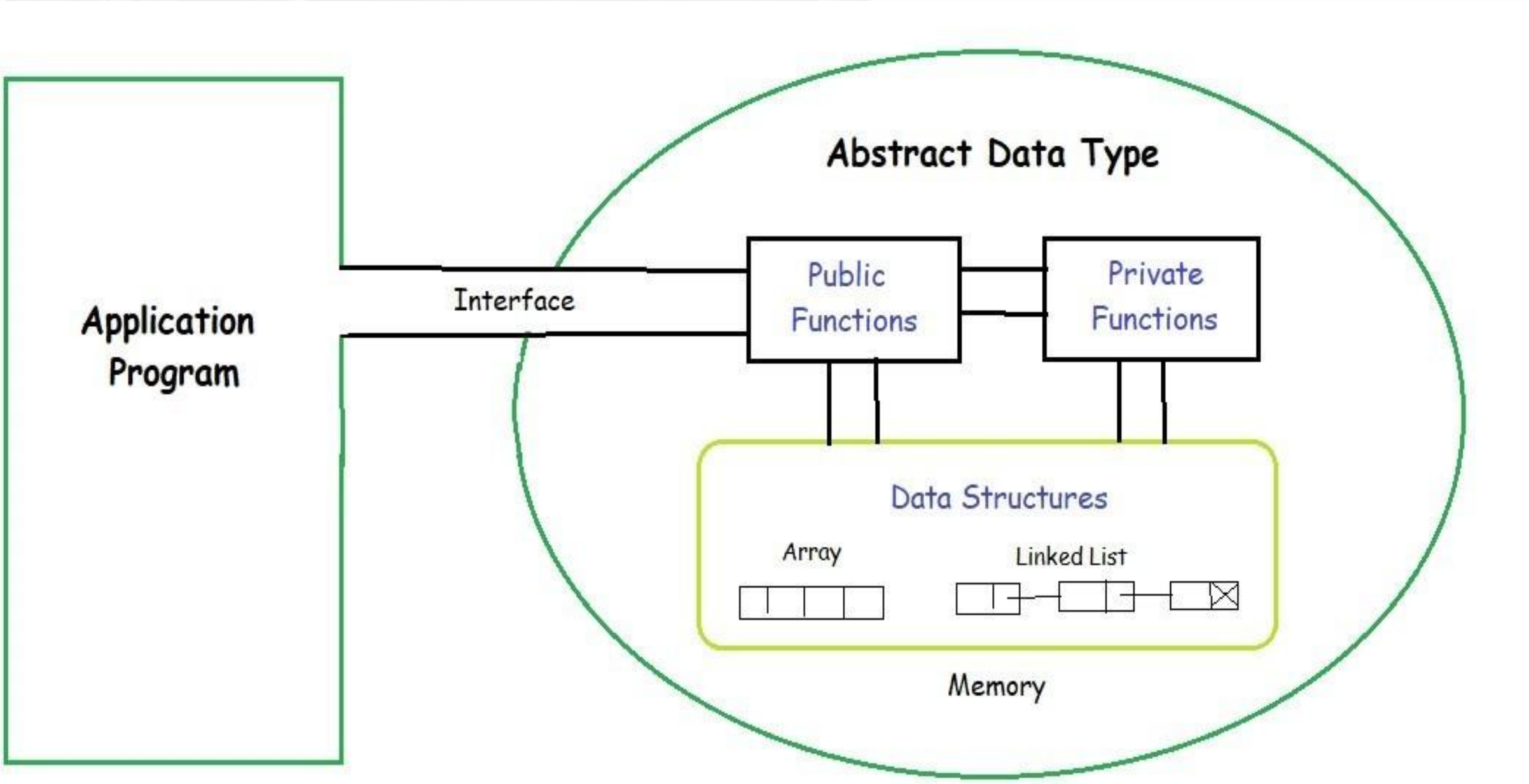
- ADT indicates for Abstract Data Type.
- Arrays are defined as ADT's because they are capable of holding contiguous elements in the same order. And they permit
- access for the specific element via index or position.
- They are abstract because they can be String, int or Person

Example :

```
int[] arrA = new int[1];

String[] arrB = new String[1];

Person[] arrC = new Person[3]; // where Person is treated as a defined class
```

Abstract Data Type as an Array

The process of providing only the essentials and hiding the details is known as abstraction.



OPERATION OF ARRAY

1. **Display ()** – To Display the entire array on the screen.
2. **Add(n) / Append(n)** – To add a particular element on the end of the array.
3. **Insert (index, n)** – To add an element to a particular index.
4. **Delete (index)** – To delete an element with the help of an index in the given array.
5. **Search (n)** – To check whether the given element is present or not in an array.
6. **Get (index)** – It will return the element which presents on the given index.
7. **Set (index, x)** – It will change the element with the new element at a particular index.
8. **Max () / Min ()** – These will return the max and min element in the given array.
9. **Reverse ()** – It will reverse the order of elements in the given array.
10. **Shift ()** – It will shift the whole elements either on the left or right side by the given number.

MERGING OF TWO ARRAY

The primary goal is to merge two sorted arrays in ascending order and display the results as output.

Combining two arrays into a single array is known as merging two arrays. For example, if the first array has 5 elements and the second array has 4, the resultant array will have 9 elements. A merged array is the result of this process.

If you are unfamiliar with the concept of an array, let's see the example :

MERGING OF TWO ARRAY

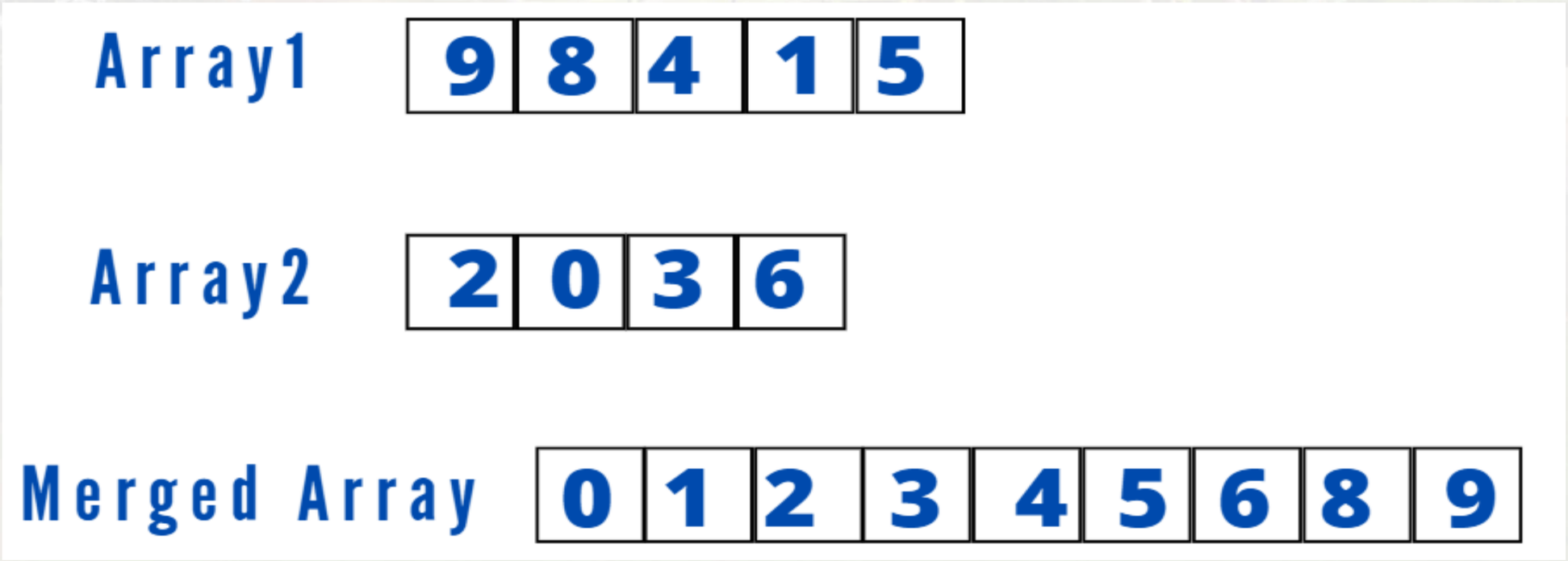
If you are unfamiliar with the concept of an array, let's see the example :

Input:

1st Array 1: 9 8 4 1 5

2nd Array 2: 2 0 3 6

Output : The merged sorted Array : 0 1 2 3 4 5 6 8 9



MERGING OF TWO ARRAY

Program 1: Using the + operator

With the help of the + operator, We can merge two arrays. The + operator concatenates the elements of two arrays into a single array. Below is the program.

arr1 = [1, 2, 3]

arr2 = [4, 5, 6]

merged_arr = arr1 + arr2

print("Array After Merging =",merged_arr)

Python

Output:

Array After Merging = [1, 2, 3, 4, 5, 6]

Reverse Array in Python

```
#Initialize array
arr = [1, 2, 3, 4, 5];
print("Original array: ");
for i in range(0, len(arr)):
    print(arr[i]),
print("Array in reverse order: ");
#Loop through the array in reverse order
for i in range(len(arr)-1, -1, -1):
    print(arr[i])
```

Output:

```
Original array:
1  2  3  4  5
Array in reverse order:
5  4  3  2  1
```

Count key using Array in Python

```
# Python code to count the number of occurrences of key element
def countX(lst, key):
    count = 0
    for ele in lst:
        if (ele == key):
            count = count + 1
    return count

# Driver Code
lst = [8, 6, 8, 10, 8, 20, 10, 8, 8]
key = 8
print('{} has occurred {} times'.format(key, countX(lst, key)))
```

Output : 8 has occurred 5 times

Ex-Arrays

```
#include <stdio.h>
int main ()
{
    int a[10],i,size;

        printf(“\nhow many no of elements u want to  scan”);

         scanf(“%d”,&size);

         printf(“\nEnter the elements in  the array”);

        for(i=0;i<size;i++)
        {
                        scanf(“%d”,&a[i]);

          }    //end for

      for(i=0;i<size;i++)
      {
                printf(“The array is %d”,a[i]); //Displaying Array

          }     //end for

return 0;
```

Output will be

1

2

3

4

5

Copy operation

```
#include <stdio.h>

int main()
{
  int a[100],b[100] position, c n;

  printf("Enter number of elements in array\n");
  scanf("%d", &n);

  printf("Enter %d elements\n", n);

  for ( c = 0 ; c < n ; c++ )
    scanf("%d", &a[c]);

  printf("Enter %d elements\n", n);


    for( c = 0 ; c < n - 1 ; c++ )
      printf("%d\n", a[c]);

//Coping the element of array a to b

  for( c = 0 ; c < n - 1 ; c++ )
    {
      b[c]=a[c];
    }
  }

  return 0;
}
```

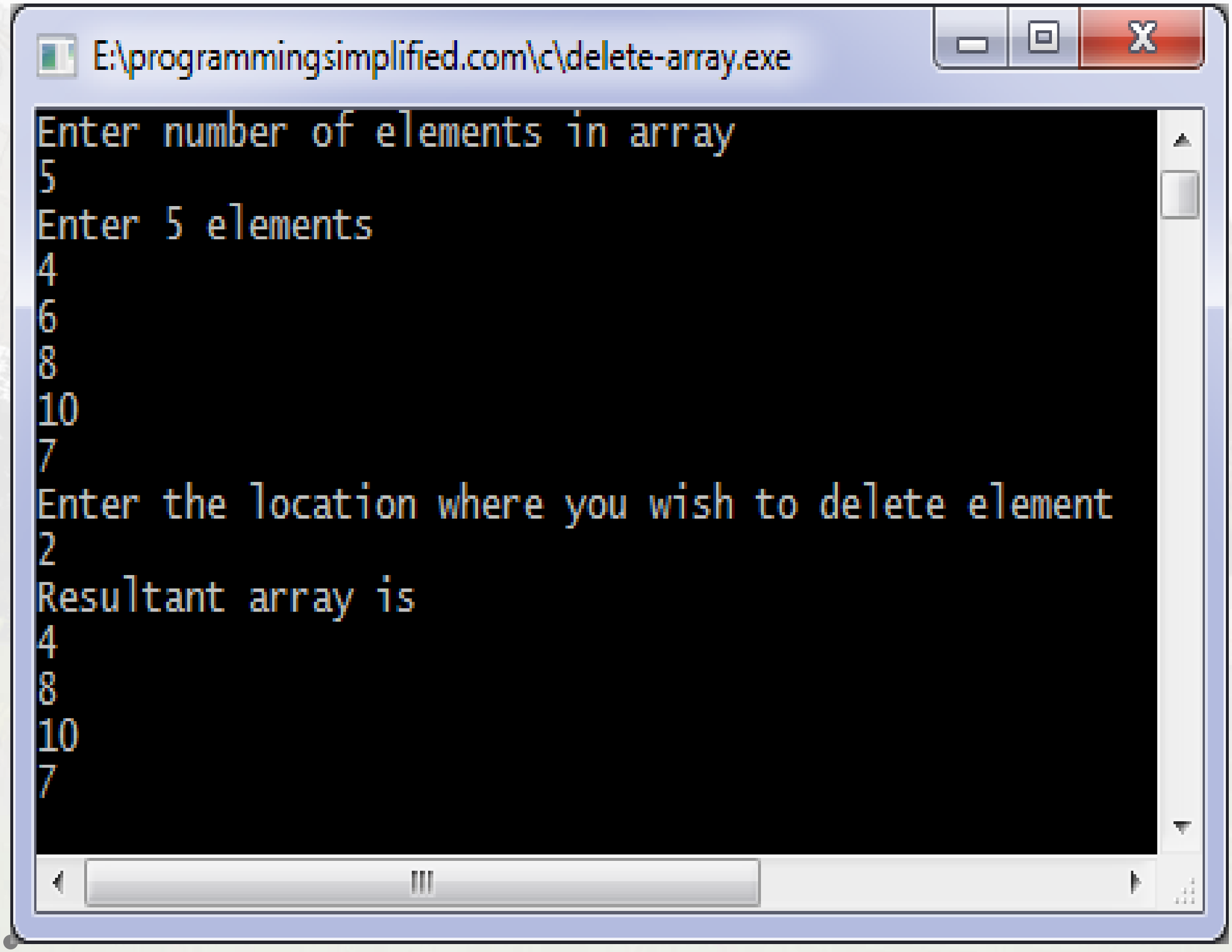
Output

```
Enter number of elements in array  -4

    Enter  4 elements

1
 2
 3
 4
  displaying array a
1
2
3
4
displaying array b
1
2
3
4
```

Delete operation

```
#include <stdio.h>

int main()
{
  int array[100], position, i, n;

  printf("Enter number of elements in array\n");
  scanf("%d", &n);

  printf("Enter %d elements\n", n);

  for ( i = 0 ; i < n ; i++ )
    scanf("%d", &array[i]);

  printf("Enter the location where you wish to delete element\n");
  scanf("%d", &position);

  for ( i = position ; i < n; i++ )
    {

    array[i] = array[i+1];

  }
   printf("Resultant array is\n");

   for( i = 0 ; i < n-1  ; i++ )

   printf("%d\n", array[i]);
  return 0;
}
```

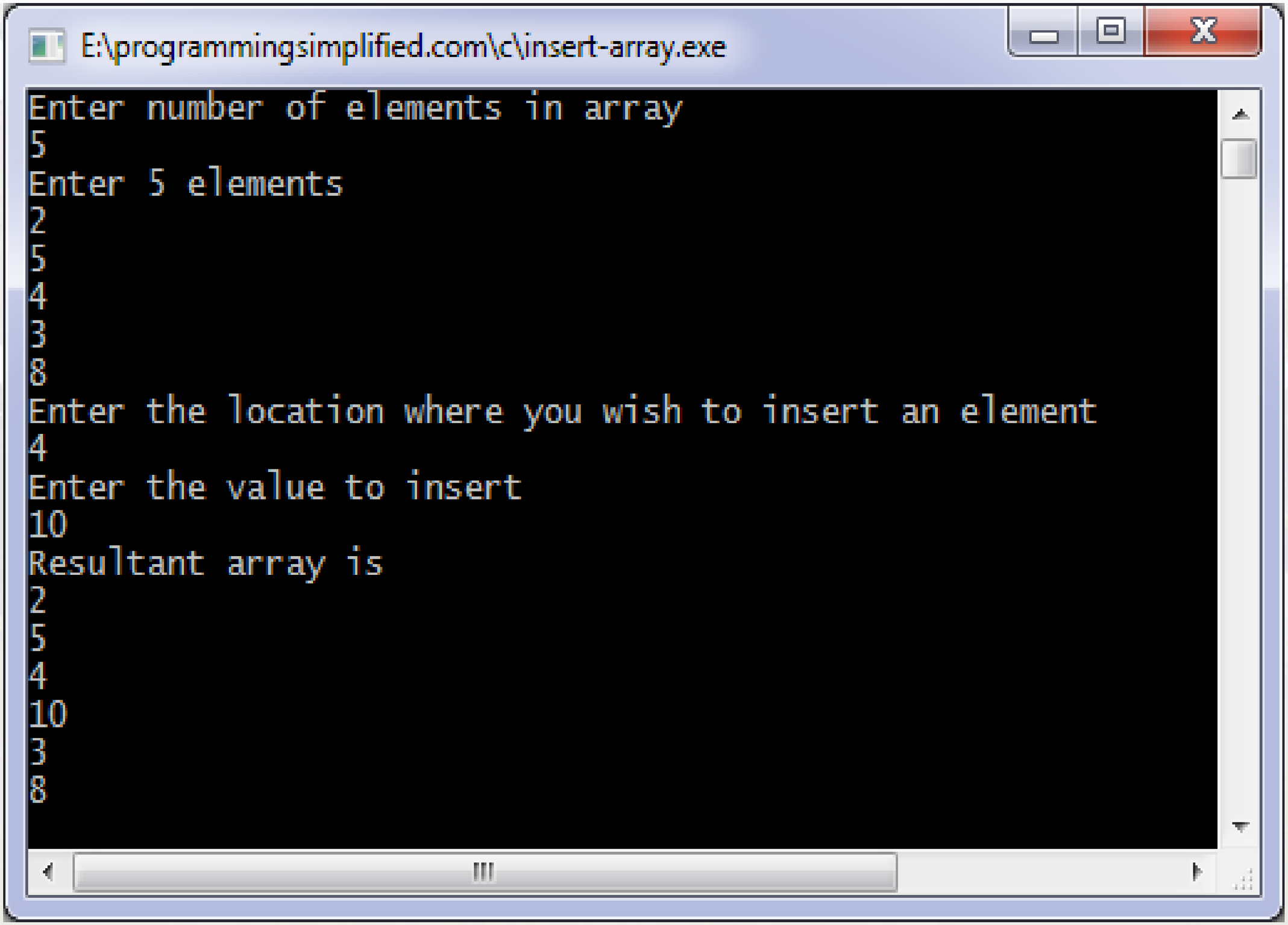
Delete operation



Inserting an element

```
#include <stdio.h>

int main()
{
   int array[100], position, i, n, value;

   printf("Enter number of elements in array\n");
   scanf("%d", &n);

   printf("Enter %d elements\n", n);

   for (i= 0;i< n; i++)
      scanf("%d", &array[i]);

printf("Enter the location where you wish to insert an element\n");
scanf("%d", &position);

printf("Enter the value to insert\n");
scanf("%d", &value);

for (i = n - 1; i >= position ; i--)
   array[i+1] = array[i];

array[position] = value;

printf("Resultant array is\n");

for (i= 0; i <= n; i++)
   printf("%d\n", array[i]);
return 0;
```

Inserting an element



Sort an array

```
Int a[10]={5,4,3,2,1}
for(i=0;i<n-1;i++)
{
  for(j=0;j<=n-1;j++)
   {
      if(a[j]>a[j+1])
        {
          temp=a[i];
            a[i]=a[j];
            a[j]=temp;
          }
}
```

Reverse array in C

```
#include <stdio.h>

int main() {
  int array[100], n, i, temp, end;

  scanf("%d", &n);
  end = n - 1;

  for (i = 0; i < n; i++) {
    scanf("%d", &array[i]);
  }

  for (i= 0; < n/2; i++)
  {
     temp  = array[i];
    array[i]   = array[end];
    array[end] = temp;

    end--;
  }

  printf("Reversed array elements are:\n");

  for ( i= 0; i < n; i++) {
    printf("%d\n", array[i]);
  }

  return 0;
}
```

Sort element using array

```
int a[10]={5,4,3,2,1}
for(i=0;i<n;i++)
  for(j=i+1;j<n;j++)
  {
        if(a[i]>a[j])
        {
           temp=a[i];
            a[i]=a[j];
            a[j]=temp;
         }
}
```

Storage Representation of Array

Note : As we have already studied and discussed Basic concepts of array, representation, operation, 1-D array etc.

Here, we will see types of array and their address calculation.

Types of Arrays

The various types of arrays are as follows.

1. One dimensional array
2. Two dimensional array
3. Multi-dimensional array

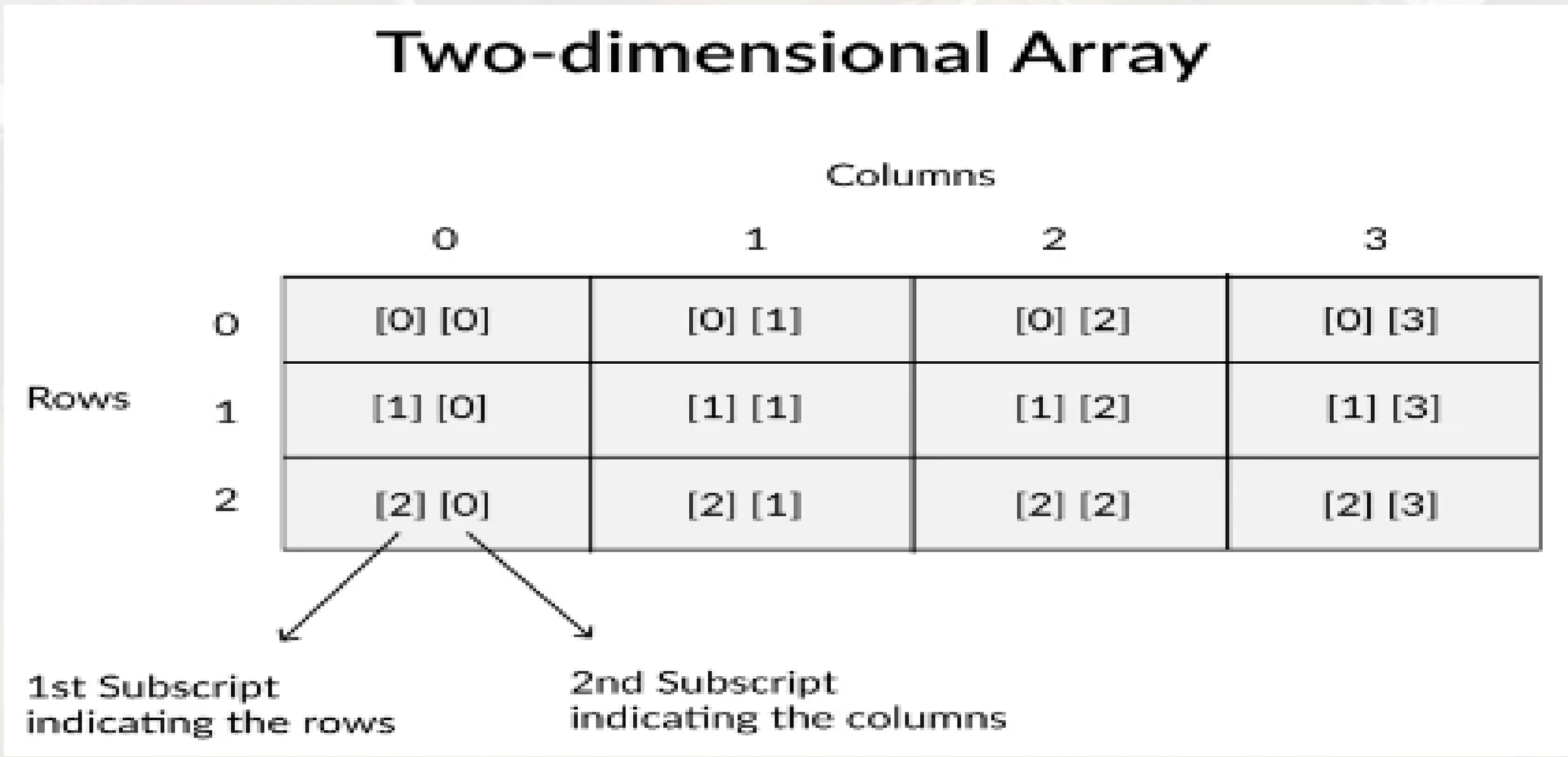
1. One-Dimensional Array

A one-dimensional array is also called a single-dimensional array where the elements will be accessed in sequential order. This type of array will be accessed by the subscript of either a column or row index

Storage Representation of Array

Multi-Dimensional Array :

When the number of dimensions specified is more than one, then it is called as a multi-dimensional array. Multidimensional arrays include 2D arrays and 3D arrays.



Storage Representation of Array

Multi-Dimensional Array :

A two-dimensional array will be accessed by using the subscript of row and column index. For traversing the two-dimensional array, the value of the rows and columns will be considered. In the two-dimensional array face [3] [4], the first index specifies the number of rows and the second index specifies the number of columns and the array can hold 12 elements (3 * 4).

Similarly, in a three-dimensional array, there will be three dimensions. The array face [5] [10] [15] can hold 750 elements (5 * 10 * 15).

1-D & 2-D dimensional Arrays Syntax

- **One dimensional array declaration**

  type arrayname[size];

  e.g. int array[10]

- **Multidimensional array is the two-dimensional array**

  type arrayName [size_row][size_col];

  e.g. int array[3] [3];

- **Multidimensional array is the three-dimensional array**

  type arrayName [size][size][size];;

Storage Representation of Array

Declaration/Initialization of Arrays

// A sample program for Array Declaration

```
#include <stdio.h>
int main()
{
int one_dim [10];    # declaration of 1D array
int two_dim [2][2];  #declaration of 2D array
int three_dim [2][3][4] = {
                { {3, 4, 2, 3}, {0, -3, 9, 11}, {23, 12, 23, 2} },
                { {13, 4, 56, 3}, {5, 9, 3, 5}, {3, 1, 4, 9} }
            }; #declaration of 3D array. Here the elements are also defined.return 0;
}
```

```
m-no of rows
n-no of columns
Printf(“\n Enter the rows and columns”);
Scanf(%d %d”,&m,&n);
for(i=0;i<m;i++)
{
for(j=0;j<n;j++)
    {
    Printf(“\n Enter the value of(%d)(%d)=“,i,j);

    Scanf(“%d”,&a[i][j]);
    }
    }
```

```
for(i=0;i<m;i++)
{
    Printf("\n");
     for(j=0;j<n;j++)
    {
        printf("%d",&a[i][j]);
    }
    }
```

1-D ADDRESS CALCULATION

Calculating the address of any element In the 1-D array:

A 1-dimensional array (or single-dimension array) is a type of linear array. Accessing its elements involves a single subscript that can either represent a row or column index.

1-D ADDRESS CALCULATION

To find the address of an element in an array the following formula is used-

Address of A[I] = B + W * (I – LB)

I = Subset of element whose address to be found,

B = Base address,

W = Storage size of one element store in any array(in byte),

LB = Lower Limit/Lower Bound of subscript(If not specified assume zero).

2

1-D ADDRESS CALCULATION

Example: Given the base address of an array A[1300 …… 1900] as 1020 and the size of each element is 2 bytes in the memory, find the address of A[1700].

Solution: Given:

Base address B = 1020

Lower Limit/Lower Bound of subscript LB = 1300

Storage size of one element store in any array W = 2 Byte

Subset of element whose address to be found I = 1700

Formula used: Address of A[I] = B + W * (I – LB)

Solution:

Address of A[1700] = 1020 + 2 * (1700 – 1300)

= 1020 + 2 * (400)

= 1020 + 800

Address of A[1700] = 1820

Memory Representation and Calculation

A. Memory Representation

— Memory arrangement after declaring one dimensional array shown in Fig. 2.3.1.

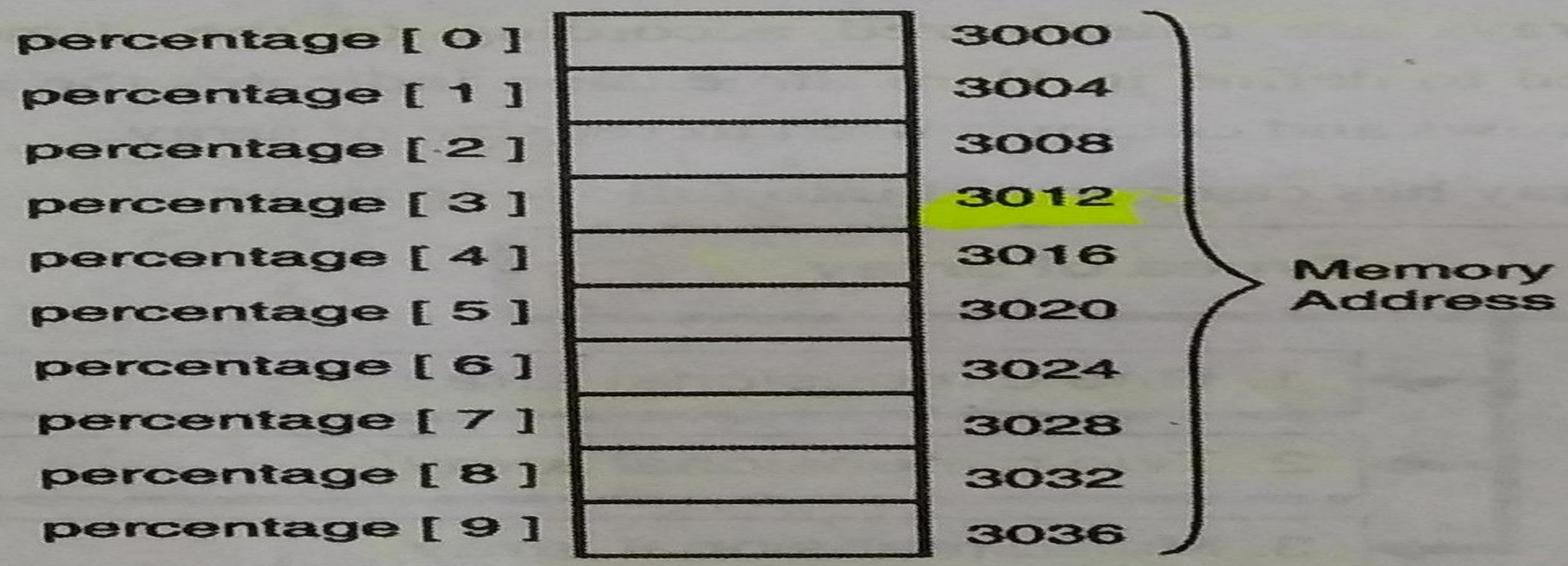


Fig. 2.3.1

— Initialize an array to store percentage of 10 students

```
percentage[0] = 99.07;
percentage[1] = 79.47;
percentage[2] = 60.60;
percentage[3] = 54.04;
percentage[4] = 80.60;
percentage[5] = 91.70;
percentage[6] = 90.30;
percentage[7] = 85.27;
percentage[8] = 99.70;
percentage[9] = 94.50;
```

ADDRESS CALCULATION

- The address of the i^{th} element is calculated by the following formula

(Base address) + (offset of the i^{th} element from base address)

Here, base address is the address of the first element where array storage starts.

☞ Address calculation of 1D array

Address of arr[i] = base address + i * element_size

Address of percentages[3] = 3000 + 3 * sizeif(float)

= 3000 + 3 * 4 = 3012

2-D ADDRESS CALCULATION

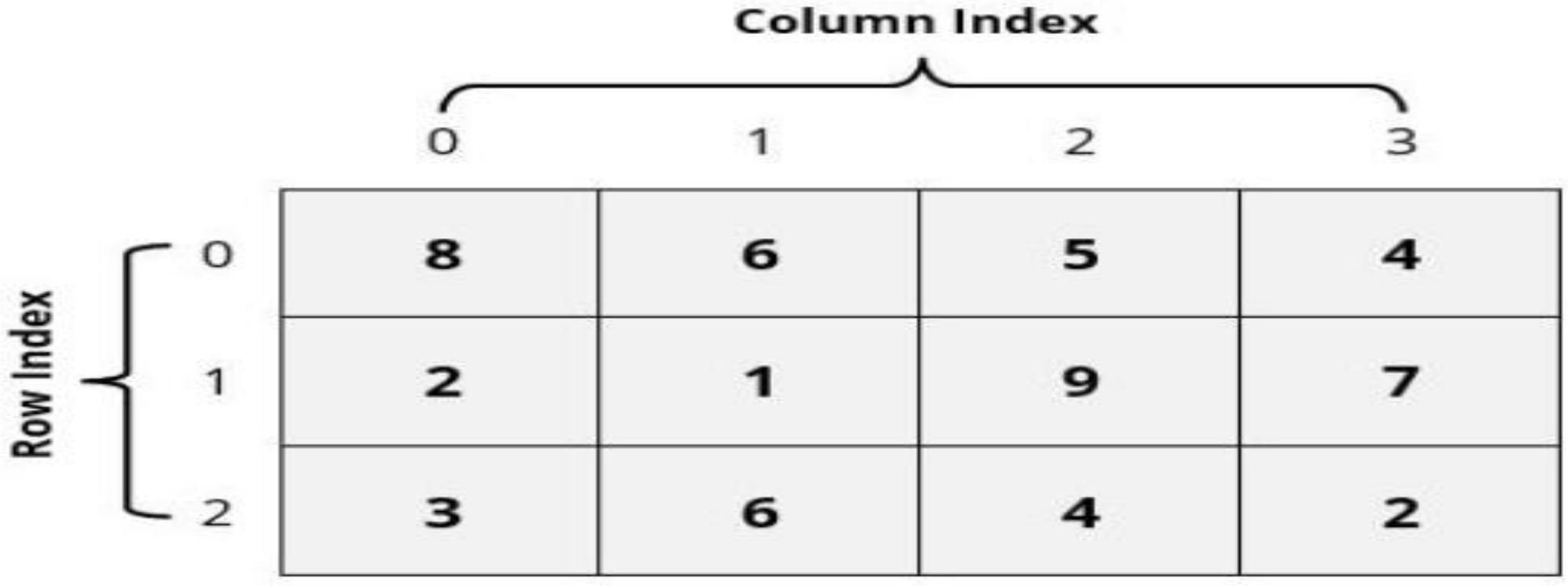
The 2-dimensional array can be defined as an array of arrays. The 2-Dimensional arrays are organized as matrices which can be represented as the collection of rows and columns as array[M][N] where M is the number of rows and N is the number of columns.

Columns

2D Array	0	1	2
0	a[0][0]	a[0][1]	a[0][2]
1	a[1][0]	a[1][1]	a[1][2]
2	a[2][0]	a[2][1]	a[2][2]

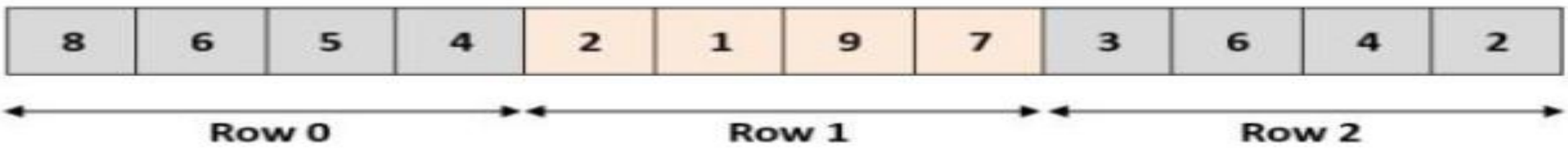
Rows

Row major & Col major Representation

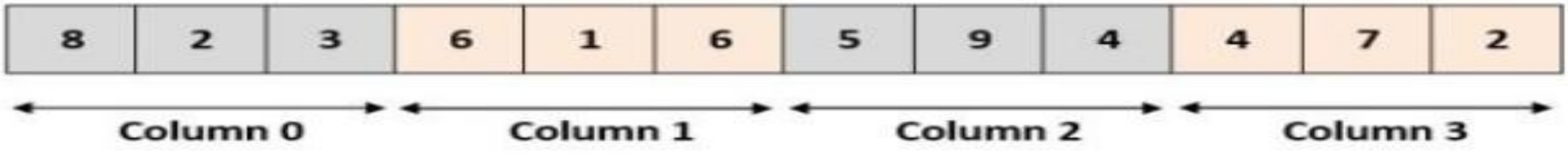


Two-Dimensional Array

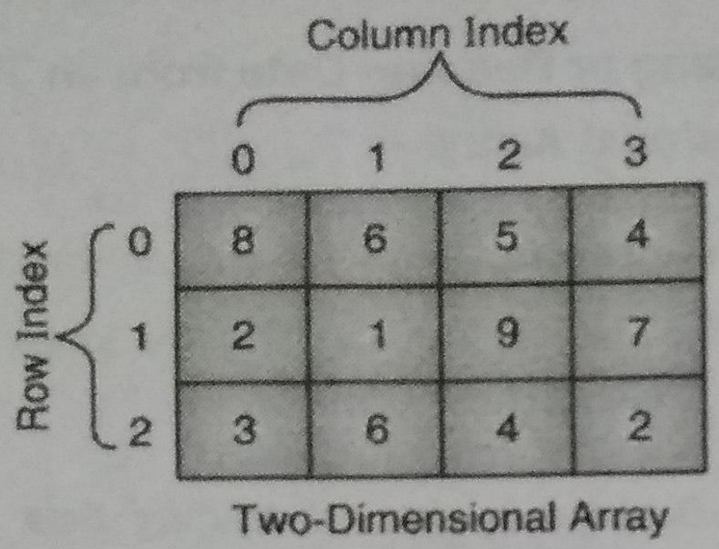
Row-Major (Row Wise Arrangement)



Column-Major (Column Wise Arrangement)

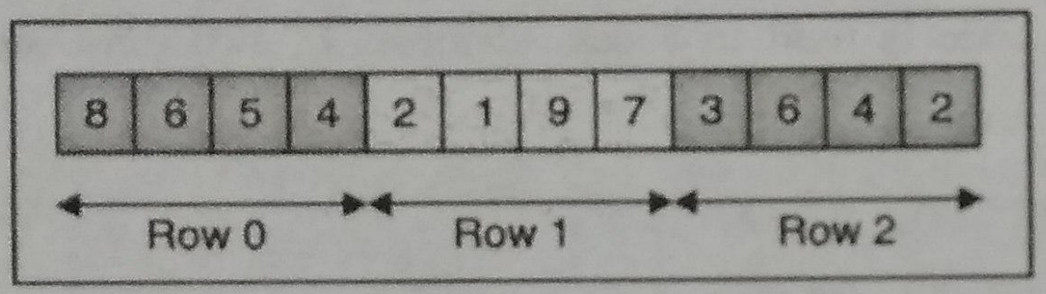


Row-major representation

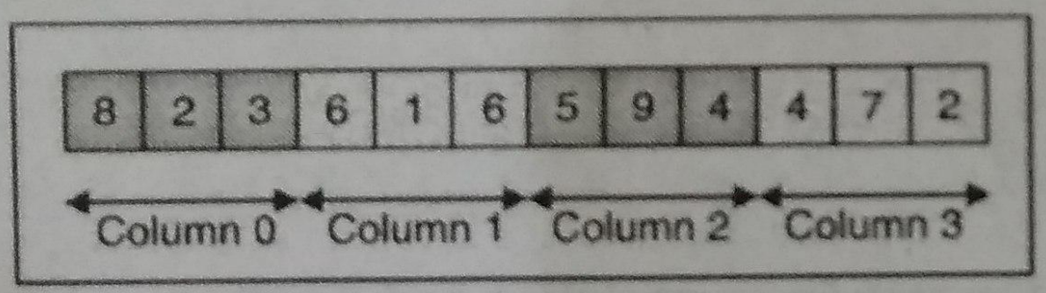


Two-Dimensional Array

- For this array the row-major representation will be :



- For column-major order, all elements of the first column come before all elements of the second column, etc.
- For the given array the column-major representation will be :



☞ **Address Calculation of 2D array**

A. Row major representation

Address of [i][j] = base address + i *c * element_size
+ j * element_size
= base address + (i*c*+j) * element_size

Example

- Adresss of arr[1][2] (Consider base address as 1000)

Arr[1][2 = 1000 + (1*3+2) * sizeof(int)
= 1000 + 10 = 1010

B. Column major representation

Address of [i][j] = base address + j *r * element_size
+ i * element_size
= base address + (j*r*+i) * element_size

Example

- Adresss of arr[1][2] (Consider base address as 1000)

Arr[1][2] = 1000 + (2*4+1) * sizeof(int)
= 1000 + 18 = 1018

SPARSE MATRIX

- A matrix is a two-dimensional data object made of m rows and n columns, therefore having total m x n values. If most of the elements of the matrix have 0 value, then it is called a sparse matrix.

Why to use Sparse Matrix instead of simple matrix ?

Storage: There are lesser non-zero elements than zeros and thus lesser memory can be used to store only those elements.

Computing time: Computing time can be saved by logically designing a data structure traversing only non-zero elements.

Example: Matrix :

|0 0 3 0 4|
|0 0 5 7 0|
|0 0 0 0 0|
|0 2 6 0 0|

SPARSE MATRIX

- Representing a sparse matrix by a 2D array leads to wastage of lots of memory as zeroes in the matrix are of no use in most of the cases. So, instead of storing zeroes with non-zero elements, we only store non-zero elements. This means storing non-zero elements with triples-(Row, Column, value).

Sparse Matrix Representations can be done in many ways following are two common representations:

Array representation

Method 1: Using Arrays:

2D array is used to represent a sparse matrix in which there are three rows named as

Row: Index of row, where non-zero element is located

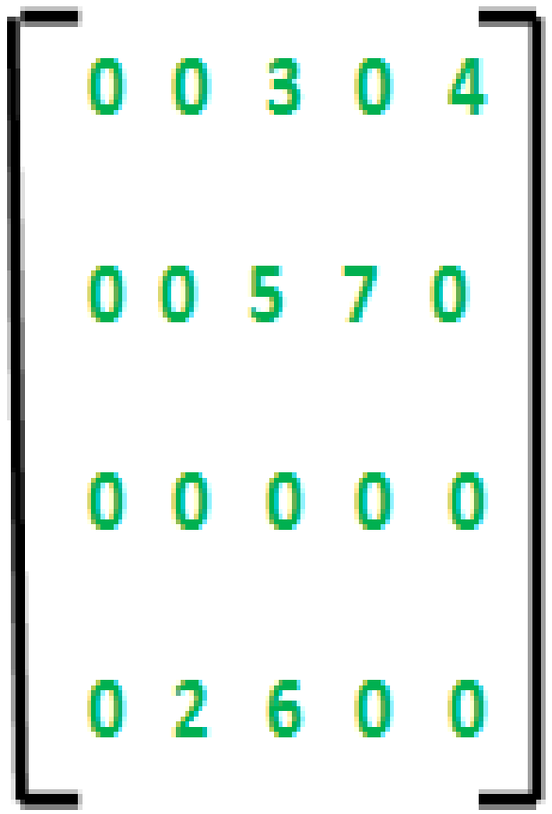
Column: Index of column, where non-zero element is located

Value: Value of the non zero element located at index – (row,column)

SPARSE MATRIX

Array representation

Method 1: Using Arrays:



Row	0	0	1	1	3	3
Column	2	4	2	3	1	2
Value	3	4	5	7	2	6

Time Complexity: O(NM), where N is the number of rows in the sparse matrix, and M is the number of columns in the sparse matrix.

Auxiliary Space: O(NM), where N is the number of rows in the sparse matrix, and M is the number of columns in the sparse matrix.

Let us take an example of a logical matrix LA and LB as follows:

$$LA = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}_{7 \times 5}$$

$$LB = \begin{pmatrix} 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 1 \end{pmatrix}_{7 \times 5}$$

Sparse Logical Matrix

Matrix A =

0	0	0	0	0
0	1	0	0	0
0	0	9	8	0
0	3	0	0	0
0	0	0	5	4
0	0	2	3	0
0	0	0	0	0

6 x 7

Alternate Representation of Sparse Matrix A=

6 Rows	7 Columns	8 NonZero Entries
1	1	1
2	2	9
2	3	8
3	1	3
4	3	5
4	4	4
5	2	2
5	3	3

9 x 3

Sparse matrix and its representation

Transpose Of Sparse Matrix

- *Simple Transpose*
- *Fast Transpose*

Transpose Of Sparse Matrix

The transpose of a matrix is found by interchanging its rows into columns or columns into rows. The transpose of the matrix is denoted by using the letter "T" in the superscript of the given matrix. For example, if "A" is the given matrix, then the transpose of the matrix is represented by A' or A^T.

The fo

$$If\ A = [a_{ij}]_{m \times n}, then\ A' = [a_{ij}]_{n \times m}.$$

Thus Transpose of a Matrix is defined as "A Matrix which is formed by turning all the rows of a given matrix into columns and vice-versa."

Transpose Of Sparse Matrix

How to Find the Transpose of a Matrix?

Consider an example, if a matrix is a 2×3 matrix. It means it has 2 rows and 3 columns. While finding the transpose of a matrix, the elements in the first row of the given matrix are written in the first column of the new matrix. Similarly, the elements in the second row of the given matrix are written in the second column of the new matrix. Hence, the order of the new matrix becomes 3×2, as it has 3 rows and 2 columns.

$$A = \begin{bmatrix} a & b & c \\ d & e & f \end{bmatrix}_{2 \times 3} \qquad A^T = \begin{bmatrix} a & d \\ b & e \\ c & f \end{bmatrix}_{3 \times 2}$$

Transpose Of Sparse Matrix

Example- Find the transpose of the given matrix

$$M = \begin{bmatrix} 2 & -9 & 3 \\ 13 & 11 & -17 \\ 3 & 6 & 15 \\ 4 & 13 & 1 \end{bmatrix}$$

Solution- Given a matrix of the order 4×3.

The transpose of a matrix is given by interchanging rows and columns.

$$M^T = \begin{bmatrix} 2 & 13 & 3 & 4 \\ -9 & 11 & 6 & 13 \\ 3 & -17 & 15 & 1 \end{bmatrix}$$

Time complexity of manual technique is O (mn).

$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{pmatrix}$$

$$\begin{pmatrix} 1 & 5 & 9 \\ 2 & 6 & 10 \\ 3 & 7 & 11 \\ 4 & 8 & 12 \end{pmatrix}$$

Sparse matrix transpose

3 x 4

Sparse Matrix and its Transpose

4 x 3

Let B=

6	7	5
1	2	7
2	4	2
3	6	5
5	0	4
5	3	9
6	1	8

B^T=

7	6	5
2	1	7
4	2	2
6	3	5
0	5	4
3	5	9
1	6	8

Simple Sparse matrix transpose

- *Time complexity will be O (n. **T**)*

$$= \boldsymbol{O\,(n\,.\,mn)}$$

$$= \boldsymbol{O\,(mn^2)}$$

***which is worst than** the conventional transpose with time complexity O (mn)*

Python code to transpose of matrix

```
# Program to transpose a matrix using a nested loop

X = [[12,7],
    [4 ,5],
    [3 ,8]]

result = [[0,0,0],
          [0,0,0]]

# iterate through rows
for i in range(len(X)):
   # iterate through columns
   for j in range(len(X[0])):
       result[j][i] = X[i][j]

for r in result:
   print(r)
```

Advantages of matrix

1. It helps in 2D Visualization.
2. It stores multiple elements of the same type using the same name.
3. Elements can be accessed using an index.
4. It enables access to items at random.
5. Any form of data with a fixed size can be stored.
6. It is easy to implement.
7. It is the simplest 2D data structure.
8. It is used by all programming languages.

Disadvantages of matrix

1. The matrix's size must be known beforehand.
2. It is of fixed size and its size can't be modified further.
3. Insertion and deletion operations are costly if shifting occurs.
4. Memory wastage may result if the specified matrix size is more than what is required.
5. Resizing a matrix can be time-consuming, especially if it needs to be done frequently or the size of the matrix is very large.
6. No built-in functionality..

Fast Transpose Of Sparse Matrix

- A fast-transpose is a computer algorithm that quickly transposes a sparse matrix using a relatively small amount of memory.
- Using arrays normally to record a sparse matrix uses up a lot of memory since many of the matrix's values are zero.
- In addition, using the normal transpose algorithm to transpose this matrix will take O(cols*elements) amount of time.
- The fast-transpose algorithm only uses a little memory to record the matrix and takes only O(cols+elements) amount of time, which is efficient considering the number of elements equals cols*rows.

Fast Transpose Of Sparse Matrix

Fast transpose

- Algorithm computes in advance the starting array index and size for each row in the transposed matrix.
- Time complexity

1. O(NumberOfTerms + NumberOfColumns), which is less than O(NumberOfRows * NumberOfColumns) for sparse matrices.
2. If matrix is not too sparse then NumberOfTerms is O(NumberOfRows * NumberOfColumns), so complexity is O(NumberOfRows * NumberOfColumns)

- Pay in space complexity due to the extra.”

Fast Sparse matrix transpose

Fast Transpose

- It is very fast but little complex method to find out transpose of sparse matrix
- Time complexity is O(no. of columns + no. on non-zero values)

Steps:

1. Obtained the triplet form of sparse matrix
2. Create 2 one dimensional arrays : total_array and index_array
3. Size of total array is the total number of columns in the original matrix
4. At every index of total, put the number of times respective index appear in second column of sparse matrix
5. Size of index array : size of total array + 1
6. Assign index[0] = 1 and index[i] = index[i-1] + total[i-1]
7. Now traverse the sparse matrix from second row and consider column elements
8. Location = index[Col_No]
9. $location^{th}$ index of transpose matrix = swapped row from original matrix.
10. Increase index[Col_No] by 1
11. Repeat steps from 7 to 10 for the remaining triplets of original matrix

Activate Windows
Go to Settings to activate Windows.

Fast Sparse matrix transpose

	row	col	value
smArray[0]	0	0	15
[1]	0	3	22
[2]	0	5	-15
[3]	1	1	11
[4]	1	2	3
[5]	2	3	-6
[6]	4	0	91
[7]	5	2	28

(a)

	row	col	value
smArray[0]	0	0	15
[1]	0	4	91
[2]	1	1	11
[3]	2	1	3
[4]	2	5	28
[5]	3	0	22
[6]	3	2	-6
[7]	5	0	-15

(b)

Figure 2.3: Sparse matrix and its transpose stored as triples

Searching and Sorting

Outline :

- **Searching-**
  - Search Techniques:
    - Sequential search,
    - Binary search,
    - Fibonacci search,
    - Index sequential search

- **Sorting-**
  - Types of sorting:
    - Internal and external sorting,
    - General sort concepts,
    - Comparison and Non-Comparison methods

Introduction

Searching :

“ Searching is a techniques of finding an element in a given list of elements.”

List of element could be represented using an

1. Array
2. Linked list
3. Binary tree
4. B-tree
5. Heap

Why do we need searching?

- ✓Searching is one of the core computer science algorithms.
- ✓We know that today's computers store a lot of information.
- ✓To retrieve this information proficiently we need very efficient searching algorithms.

Types of Searching

- *Linear search*
- *Binary search*
- *Sentinel search*

SEARCH TECHNIQUES

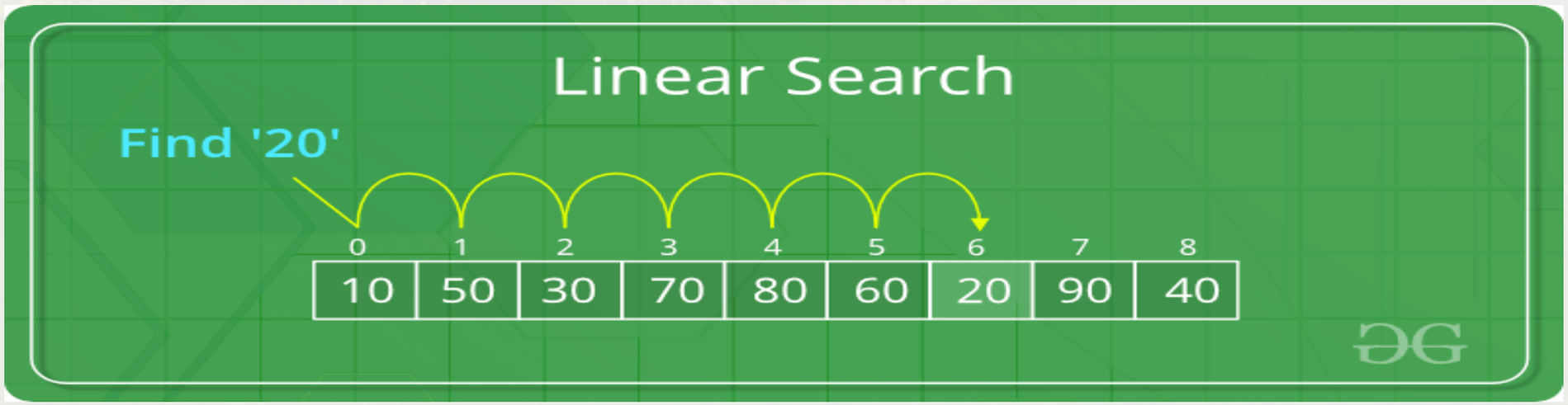
- ***Sequential search***
- ***Binary search***
- ***Fibonacci search***
- ***Hashed search***
- ***Index sequential search***

Linear Search

- The linear search is a sequential search, which uses a loop to step through an array, starting with the first element.
- It compares each element with the value being searched for, and stops when either the value is found or the end of the array is encountered.
- If the value being searched is not in the array, the algorithm will unsuccessfully search to the end of the array.

Linear Search

- Since the array elements are stored in linear order searching the element in the linear order make it easy and efficient.
- The search may be successful or unsuccessfully. That is, if the required element is found them the search is successful other wise it is unsuccessfully.



Advantages Linear Search

- easy to understand.
- It operates on both sorted and unsorted list
- It doest not require array to be in order
- Easy to implement
- Time complexity O(n)

Disadvantages Linear Search

- Linear search is not efficient when list is large
- Maximum no. of comparision are N(n Element).
- Not suitable for large problem.
- You need to search whole list.
- Linear search is slower.

Linear Search Algorithm

Consider an integer type array A with size n. So list of elements from that array are,

A[0], A[1], A[2], A[3],……., A[n-1]

1. *Declare and initialize one variable which contents the number to be search in an array A.*
   *(variable **key** is declared)*
2. *Start Comparing each element from array A with the key*
   LOOP: A[size] == key
   Repeat step no 2 while A[size] key
3. *if key is found, display the location of element(index+1)*
   or else display message KEY NOT FOUND
4. *Terminate the program successfully*

Linear Search Algorithm

```
printf("accept  number to search");
scanf( key );
 for( i=0 ;i<n ;i++)
   {
       if( A [ i ] == key )
          {
    printf( key is FOUND);
    break;
    }
    }
if( i==n )
  {
     printf(NOT FOUND);
  }
```

Analysis of Linear Search

Complexity of linear search :

1.Best Case = O(1)

2.Average Case = O(n)

3.Worst case = O(n)

Binary Search

"Binary search is an searching algorithm which is used to find element from the sorted list"

Concepts :

- Algorithm can be applied only on sorted data
- **Mid** = (low + high) / 2 - formula used to find mid
- Given element(Key) compares with middle element of the list.
- If **key=mid** then element is found
- Otherwise list divide into two part.(key <mid) (key >mid) **First to mid-1** or **mid+1 to last**.

Binary Search

Concepts :

- If given element is less than middle element . (key <mid) then continue searching in first part (first+mid-1) otherwise (key > mid)searching is in second part(mid+1 to last).
- Repeat above step still element is found.

Binary Search

Assume that two variables are declared, variable first and last, they denotes beginning and ending indices of the list under consideration respectively.

Step 1. Algorithm compares key with middle element from list (A[middle] == key), if true go to step 4 or else go to next.

Step 2. if key < A[middle], search in ***left half of the list*** *or else go to step 3*

Step 3. if key > A[middle], search in ***right half of the list*** *or go to step 1*

Step 4. display the position of key else display message "NOT FOUND"

Binary Search algorithm

```
int i, first=0, last=n-1 , middle;
 while( last>=first )
   {
     middle = (first + last)/2;

      if( key > A[middle] )
        { first = middle + 1; }
          else if ( key < A[middle] )
       { last= middle - 1; }
          else
           { printf( FOUND ) }
    }
if( last < first )
  {
     printf( NOT FOUND );
  }
```

Advatages Binary Search

1.Binary search is optimal searching algorithms

2.Excellent time efficiency

3.Suitable for large list.

4.Faster because no need to check all element.

5.Most suitable for sorted array

6.It can be search quickly

7.Time complexity O(log n)

Disadvatages Binary Search

1.Element must be sorted

2.Need to find mid element

3.Bit more complicated to implement and test

4.It does not support random access.

5.Key element require to compare with middle.

Linear Search Vs Binary Search

1. Element is searched by scanning the entire list from first element to the last
2. Many times entire list is search
3. Simple to implementation
4. Time complexity is O(n)
5. Less efficient sort

1. First list is divided into two sub-lists. Then middle element is compared with key element and then accordingly left or right sub-list is searched
2. Only sub-list is search
3. Complex to implement, since it involves computation for finding the middle element
4. Time complexity is $O(\log_2 n)$
5. More efficient sort

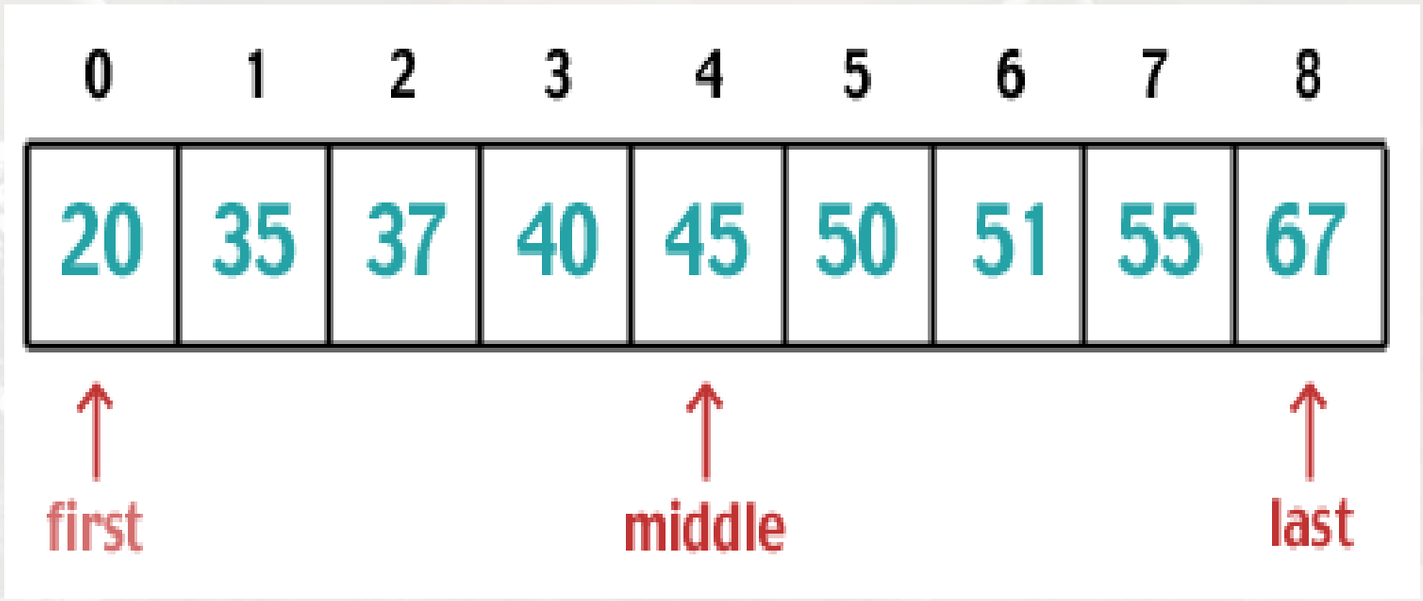
Binary Search

0	1	2	3	4	5	6	7	8
20	35	37	40	45	50	51	55	67

Binary Search

2. *Calculate middle = (low + high) / 2.*

= (0 + 8) / 2 = 4 .



*If **37** == array[middle] return middle*

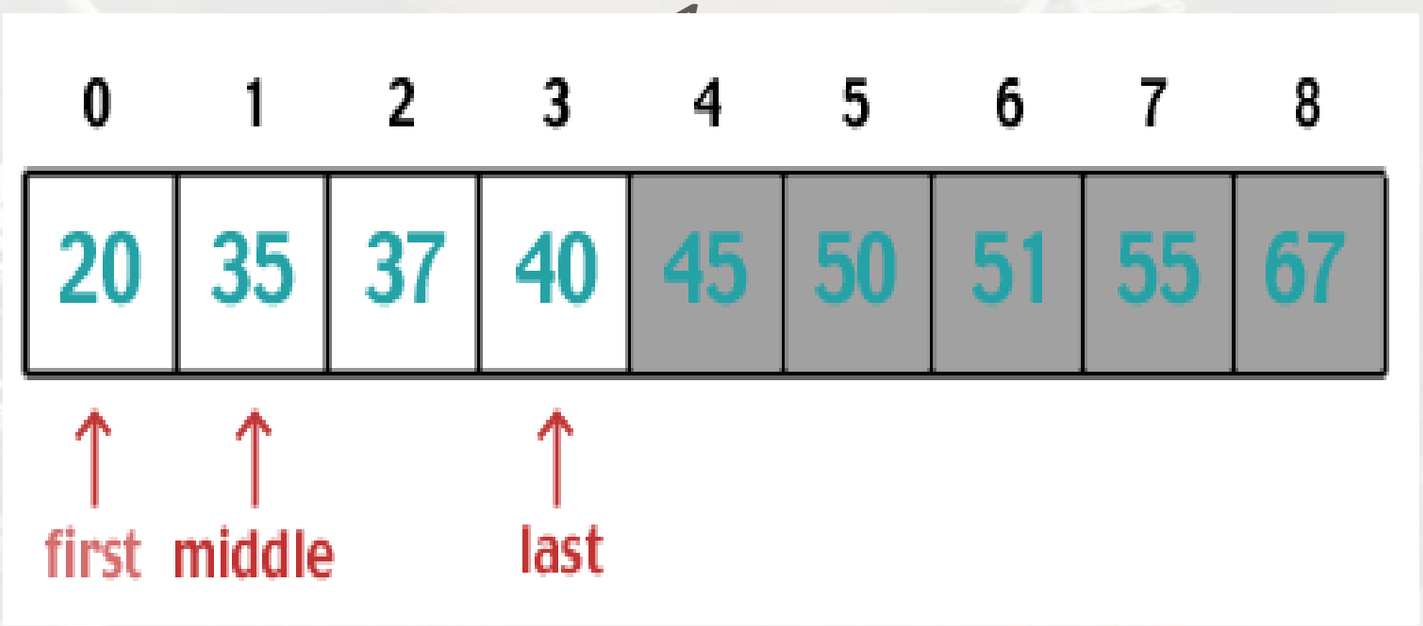
Else if 37 < array[middle] high = middle -1

Else if 37 > array[middle] low = middle +1

Binary Search

Repeat 2. Calculate middle = (low + high) / 2.

= (0 + 3) / 2 =



If 37 == array[middle] □ return middle

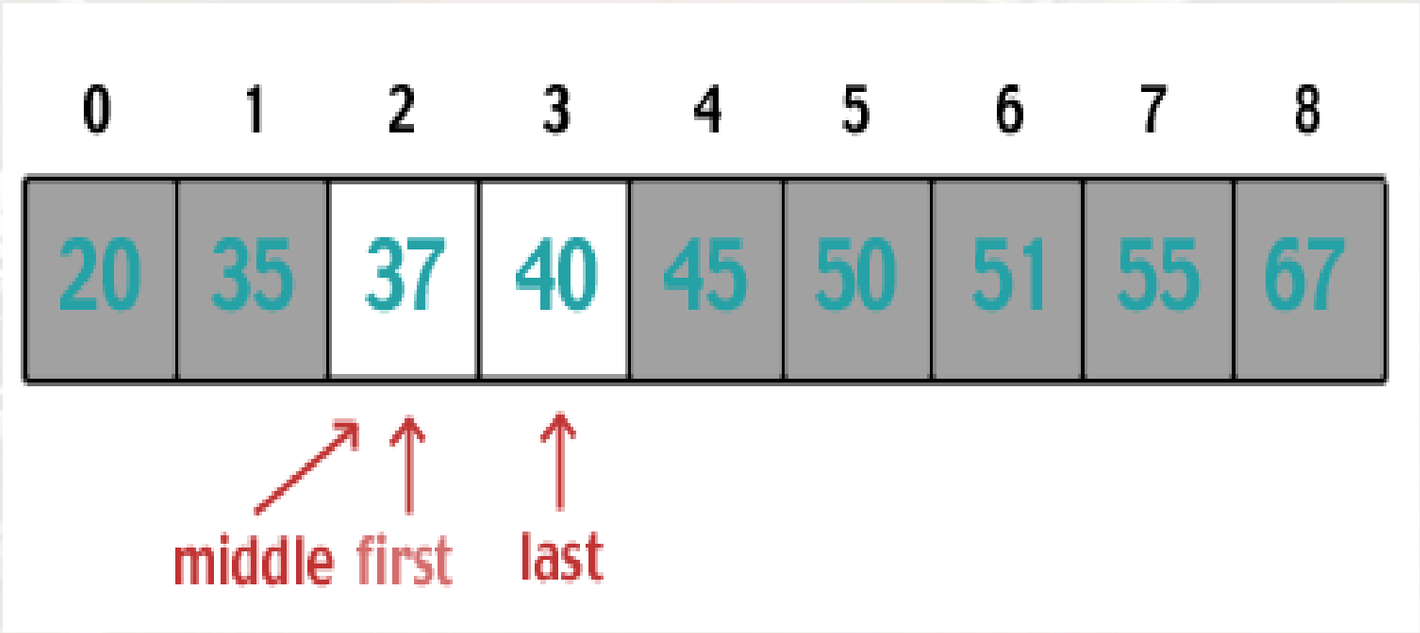
Else if 37 < array[middle] □ high = middle -1

Else if 37 > array[middle] □ low = middle +1

Binary Search

Repeat 2. Calculate middle = (low + high) / 2.

= (2 + 3) / 2 =



If 37 == array[middle] □ return middle

Else if 37 < array[middle] □ high = middle -1

Else if 37 > array[middle] □ low = middle +1

Binary Search

Search 23

0	1	2	3	4	5	6	7	8	9
2	5	8	12	16	23	38	56	72	91

23 > 16
take 2nd half

L=0	1	2	3	M=4	5	6	7	8	H=9
2	5	8	12	16	23	38	56	72	91

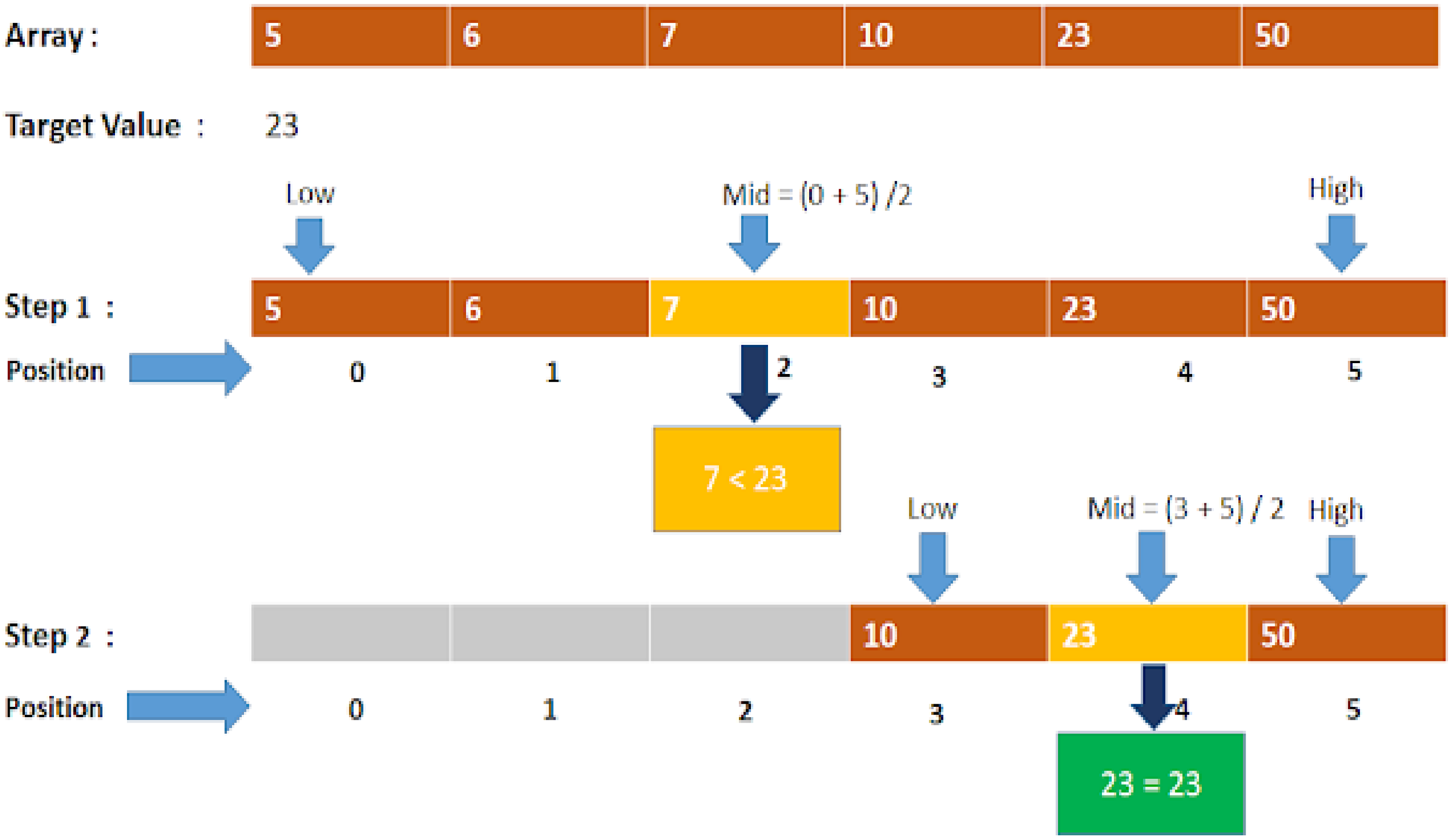
23 > 56
take 1st half

0	1	2	3	4	L=5	6	M=7	8	H=9
2	5	8	12	16	23	38	56	72	91

Found 23,
Return 5

0	1	2	3	4	L=5, M=5	H=6	7	8	9
2	5	8	12	16	23	38	56	72	91

Binary Search



Result : Target value – 23 found and Position in array is 4

Item to be searched = 23

Step 1 →

1	5	7	8	13	19	20	23	29
0	1	2	3	4	5	6	7	8

a [mid] = 13
13 < 23
beg = mid + 1 = 5
end = 8
mid = (beg + end)/2 = 13 / 2 = 6

Step 2 →

1	5	7	8	13	19	20	23	29
0	1	2	3	4	5	6	7	8

a [mid] = 20
20 < 23
beg = mid + 1 = 7
end = 8
mid = (beg + end)/2 = 15 / 2 = 7

Step 3 →

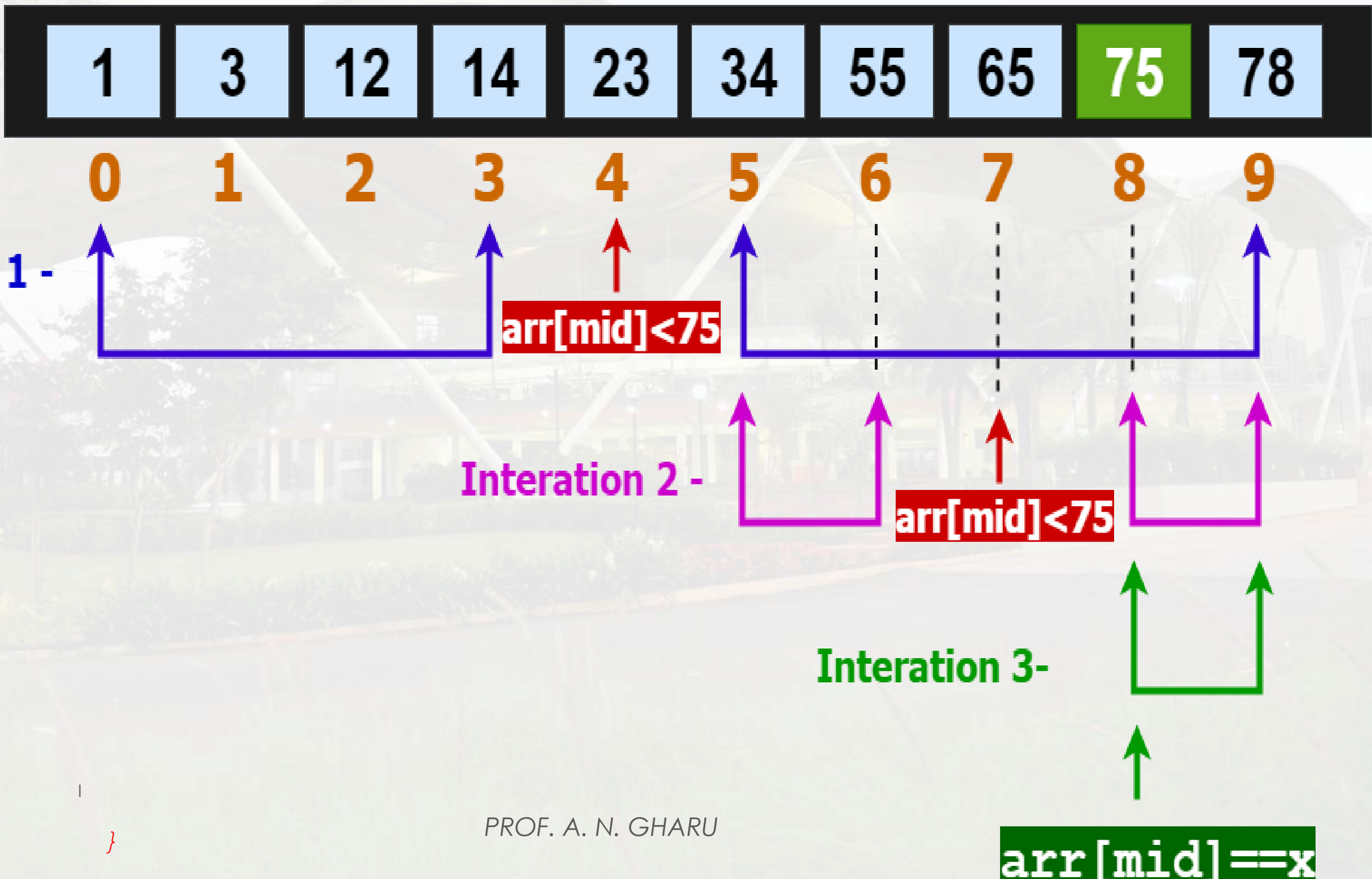
1	5	7	8	13	19	20	23	29
0	1	2	3	4	5	6	7	8

a [mid] = 23
23 = 23
loc = mid

Return location 7

Binary Search Example

Find - '75'



}

Example 7.3.1

[Searching the element 29 in the given array]

Index	0	1	2	3	4	5	6	7	8
Elements	5	9	11	15	25	29	30	35	40

$i = 0$ $\quad c = \frac{i + j}{2} = \frac{0 + 8}{2} = 4$ $\quad j = 8$

Solution :

Element to be searched, key = 29

Step 1 : Since key > a[c] (29>25) right half is selected.

index	5	6	7	8
elements	29	30	35	40

$i = 5$ $\quad c = 6$ $\quad j = 8$

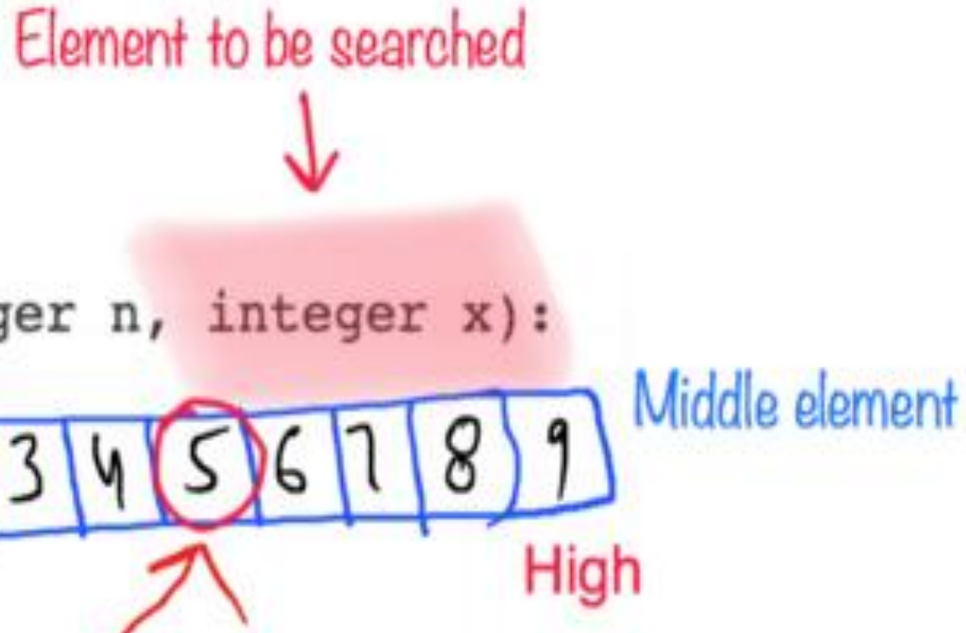
Step 2 : Since key < a[c] (29 < 30) left half is selected.

Index	5
element	29

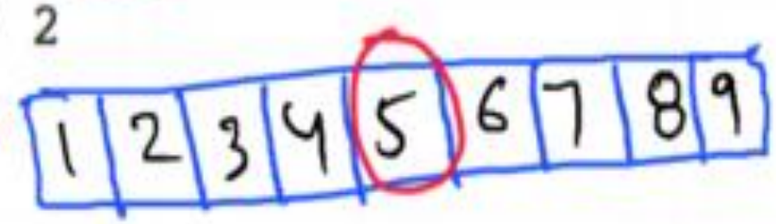
$i = 5$ $\quad c = 5$ $\quad j = 5$

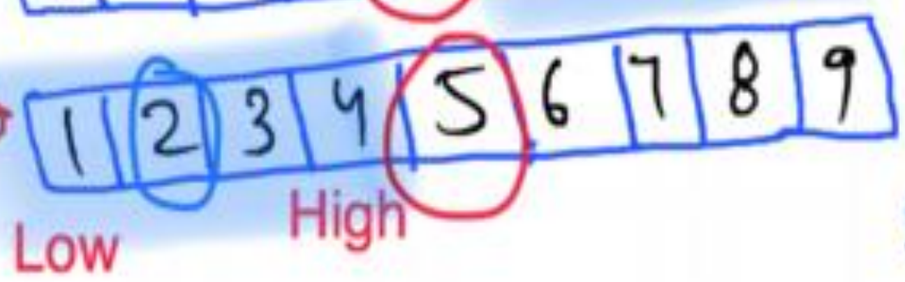
element is found and search is successful.

Binary Search Routine




```
integer binary_search(array a, integer n, integer x):
    integer low, high, mid
    low := 1
    high := n
    while low ≤ high:
        mid := (low + high) / 2
        if a[mid] == x:
            break
        else if a[mid] is less than x:
            low := mid+1
        else:
            high := mid-1
    return mid
```





Outline :

- **Searching-**
  - Search Techniques:
    - Sequential search,
    - Binary search,
    - Fibonacci search,
    - Index sequential search

- **Sorting-**
  - Types of sorting:
    - Internal and external sorting,
    - General sort concepts,
    - Comparison and Non-Comparison methods

Sentinel search

- **Sentinel Search**. To reduce overhead of checking the list's length, the value to be searched can be appended to the list at the end (or beginning in case of Reverse **Search**) as a "**sentinel** value". A **sentinel** value is one whose presence guarantees the termination of a loop that processes structured (or sequential) data.

- In computer programming, a sentinel node is a specifically designated node used with linked lists and trees as a traversal path terminator. This type of node does not hold or reference any data managed by the data structure.

Sentinel search

- This additional entry at the end of the list is called as Sentinel.
  - The speed of sequential search can be improved by storing the key being searched at end of the array.
  - This will eliminate extra comparision inside the loop for number of element in the array.

The example is given below.

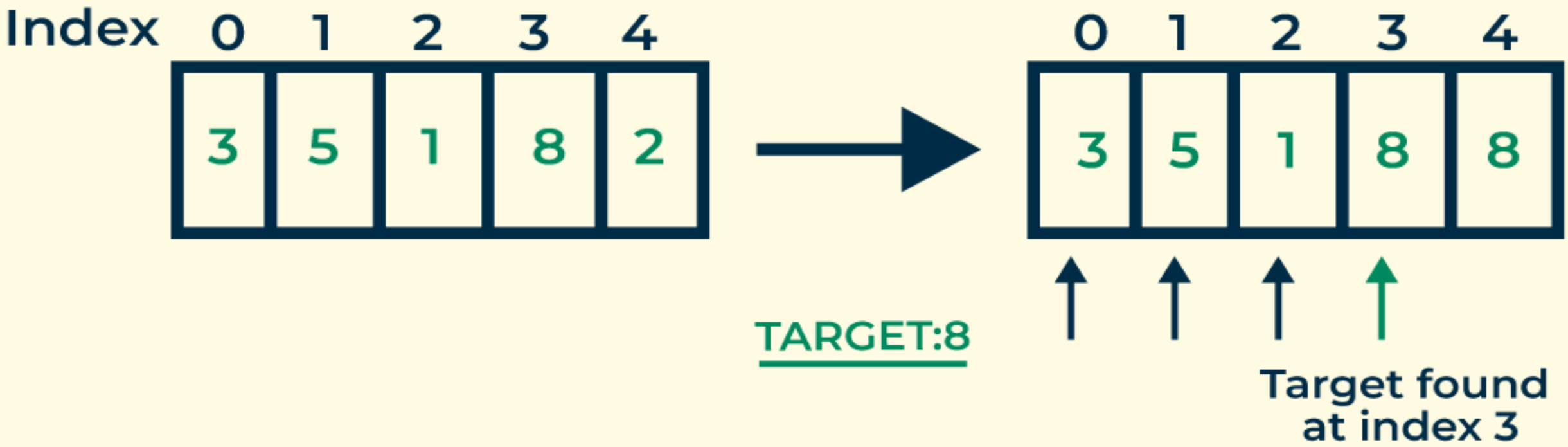
```
int sentinel search (int a[], int n , int key)
        {           int i;
                    a[n] = key;
                              i = 0;
                    while (a[i] ! = key)
                              i++;
                    return(i);
        }
```



```
# Return index of value.
def SEN_search(a_list, value) :
    a_list.append(value)
    index = 0
    while True:
        if a_list[index] == value: break
        index = index+1
    return index
```

Sentinel search

Sentinal Linear Search

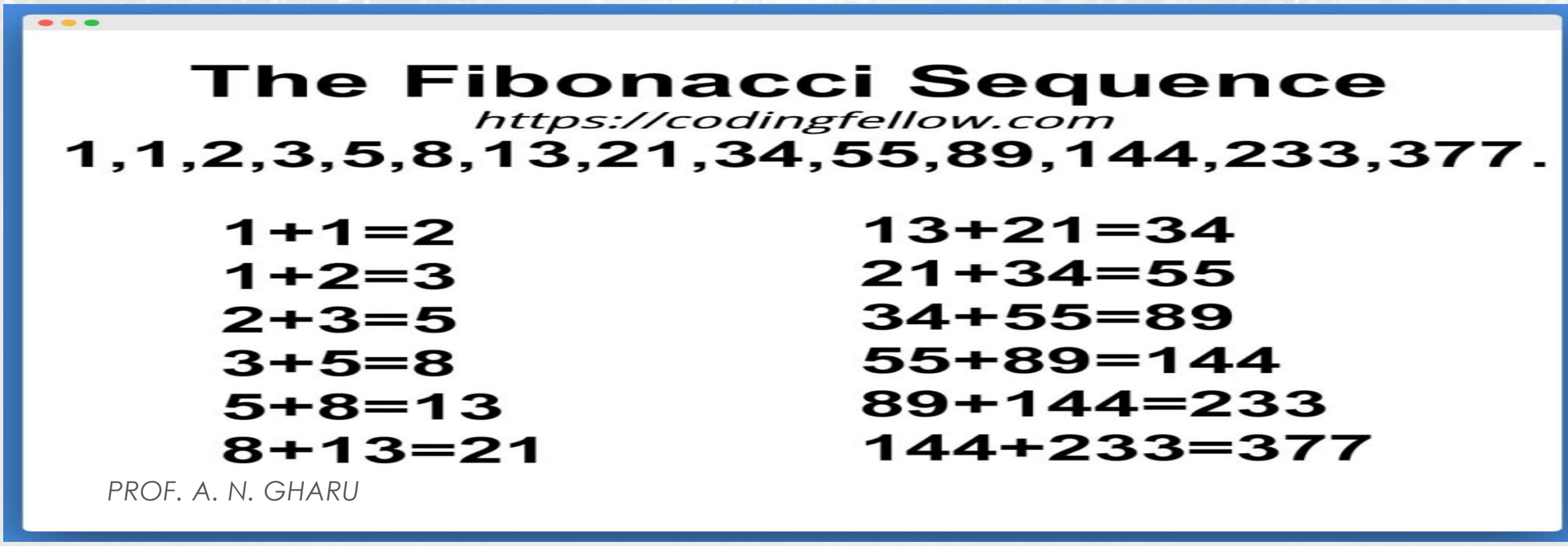


Input

```
Array[] = [3, 5, 1, 8, 2]

Target = 8
```

Fibonacci search

- Fibonacci search technique is a method of searching a sorted array using a divide and conquer algorithm that narrows down possible locations with the aid of Fibonacci numbers. Compared to binary search where the sorted array is divided into two equal-sized parts, one of which is examined further, Fibonacci search divides the array into two parts that have sizes that are consecutive Fibonacci numbers.



Fibonacci Search :

- The Fibonacci series has 0 and 1 as the first two terms, and each successive term is the sum of the previous two terms. Fibonacci numbers are 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

  With $Fn = F_{n-1} + F_{n-2}$ for $n \geq 2$ where, $F0 = 0$ and $F1 = 1$.
- FS modifies the binary search algorithm slightly. Instead of halving the index for a search, a Fibonacci number is subtracted from it.
- The Fibonacci number to be subtracted decreases as the size of the list decreases.
- Fibonacci search starts searching for the target by comparing it with the element at the Fkth location. Here, $Fk \geq n$ and $Fk-1 < n$.

Fibonacci Search :

Steps:

1. Find F(k)(kth Fibonacci number). Which is greater than or equal to n
2. If F(k) = 0 , stop and print message as element not found
3. Offset = -1
4. min(offset + F(k-2), n-1)
5. If S ==A[i]

return i and stop the search

If S > A[i]

k=k-1, offset = i and repeat steps 4,5

If S < A[i]

k = k-2 repeat steps 4,5

Fibonacci Search

	0	1	2	3	4	5	6	7	8	9 (N-1)
A	3	4	5	8	10	12	15	16	18	20

Input Array

k	0	1	2	3	4	5	6	7	8	9
F(k)	0	1	1	2	3	5	8	13	21	34

Fibonacci Array

1. F(k) >= n -> F(7) = 13 >= 10

2. If F(k)=0 STOP

3. Offset = -1

4. i = min (offset + F(k-2), n-1)
i = min (-1+5, 9) = 4

5. If X = 10 = = A [i] = 10

stop, number found

If X > A[i] # k = k-1, offset = i, find i and compare. step 4 & 5

If X < A[i] #k = k-2, find i and compare

step 4 & 5

Fibonacci search

- Fibonacci search changes the binary search algorithm slightly
- Instead of halving the index for a search, a Fibonacci number is subtracted from it
- The Fibonacci number to be subtracted decreases as the size of the list decreases
- Note that Fibonacci search sorts a list in a non decreasing order
- Fibonacci search starts searching the target by comparing it with the element at Fkth location

Cases in Fibonacci search

Case 1: if equal the search terminates;

Case 2: if the target is greater and F_1 is 1, then the search terminates with an unsuccessful search;

else the search continues at the right of list with new values of low, high, and mid as

$\texttt{mid} = \texttt{mid} + F_2, F_1 = F_{k-4}$ and $F_2 = F_{k-5}$

Case 3: if the target is smaller and F_2 is 0, then the search terminates with unsuccessful search;

else the search continues at the left of list with new values of low, high, and mid as

$\texttt{mid} = \texttt{mid} - F_2, F_1 = F_{k-3}$ and $F_2 = F_{k-4}$

The search continues by either searching at the left of mid or at the right of mid in the list.

ARCH. A. MADHAVAN

SORTING

PROF. A. N. GHARU

SORTING

- “Sorting is the process ordering a list of element in either ascending or descending order.”

- Sorting is the operation of arranging the records of a table according to the key value of each record, or it can be defined as the process of converting an unordered set of elements to an ordered set of elements

- Sorting is a process of organizing data in a certain order to help retrieve it more efficiently

INTERNAL SORTING(types)

- Any sort algorithm that uses main memory exclusively during the sorting is called as internal sort algorithm
- Internal sorting is faster than external sorting

Internal Sorting techniques :

1. Bubble sort
2. Selection sort
3. Insertion sort
4. Quick sort
5. Shell sort
6. Heap sort
7. Radix sort
8. Bucket sort

EXTERNAL SORTING

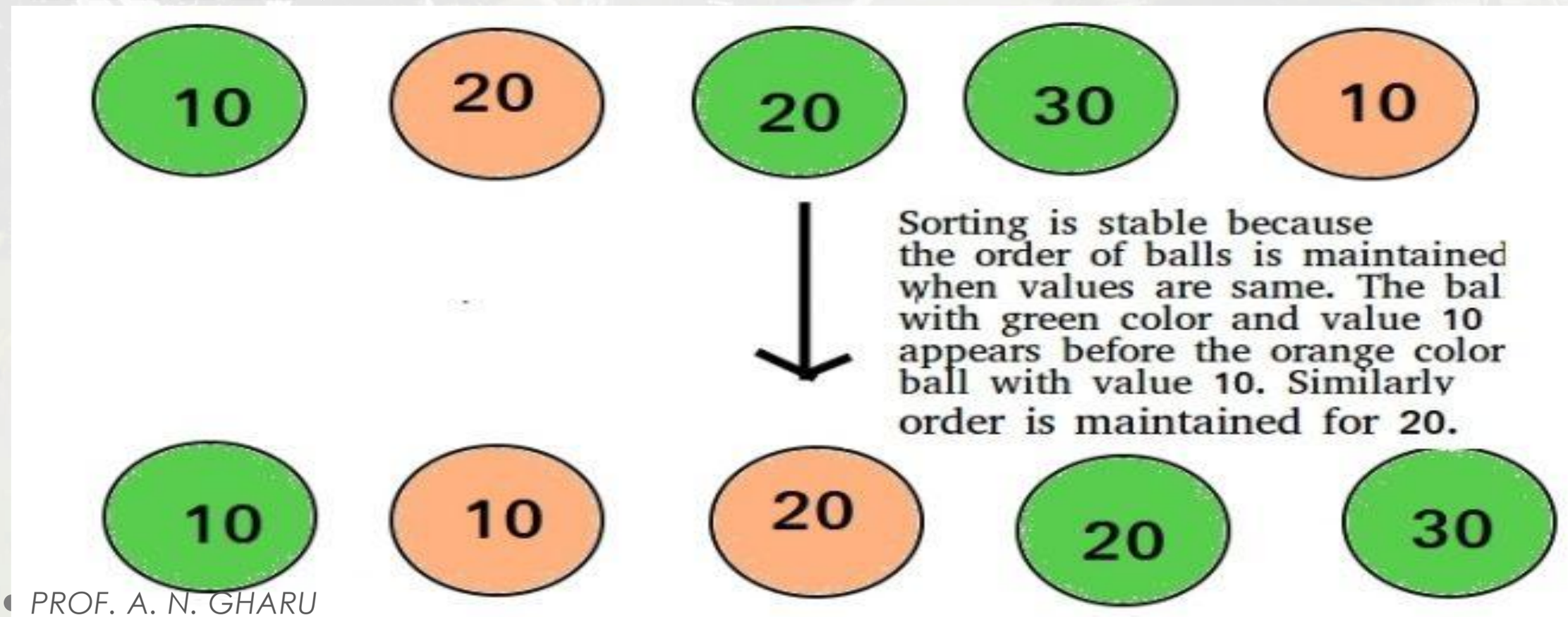
- Any sort algorithm that uses external memory, such as tape or disk, during the sorting is called as **external sort algorithm**

- **Merge sort** is used in external sorting

STABILITY OF SORTING

- A sorting method is said to be stable if at the end of the method, identical elements occur in the same relative order as in the original unsorted set

1. EXAMPLE :



SORT EFFICIENCY

- Sort efficiency is a measure of the relative efficiency of a sort
- It is usually an estimate of the number of comparisons and data movement required to sort the data

PASSES IN SORTING

- During the sorted process, the data is traversed many times
- Each traversal of the data is referred to as a sort pass
- In addition, the characteristic of a sort pass is the placement of one or more elements in a sorted list

BUBBLE SORTING

- Bubble sort is a simple sorting algorithm.
- This sorting algorithm is comparison-based algorithm in which each pair of adjacent elements is compared and the elements are swapped if they are not in order.
- This algorithm is not suitable for large data sets as its average and worst case complexity are of $O(n^2)$ where **n** is the number of items.

How Bubble Sort Works?

- We take an unsorted array for our example. Bubble sort takes

  $O(n^2)$ time so we're keeping it short and precise.

  Bubble sort start with first two element, compare them to check which one is greater. And swap it.

Algorithm Bubble sorting

1. In Bubble sort pairs of adjacent elements (start from 0^{th} and 1^{st} locations) is compared and then swapping is performed when first element is greater than another element in pair.
2. Repeat step 1 until (n - 2) position element is compared with (n - 1) position element.
3. Here first iteration is completed and largest value in list is stored at (n - 1) location.
4. Now start second iteration, Repeat step 1 until (n - 3) position element is compared with (n - 2) position element.
5. If there are n elements, then (n - 1) passes are required.

PROF. A. TIWARI

Bubble Sort

Advantages of Bubble Sort:

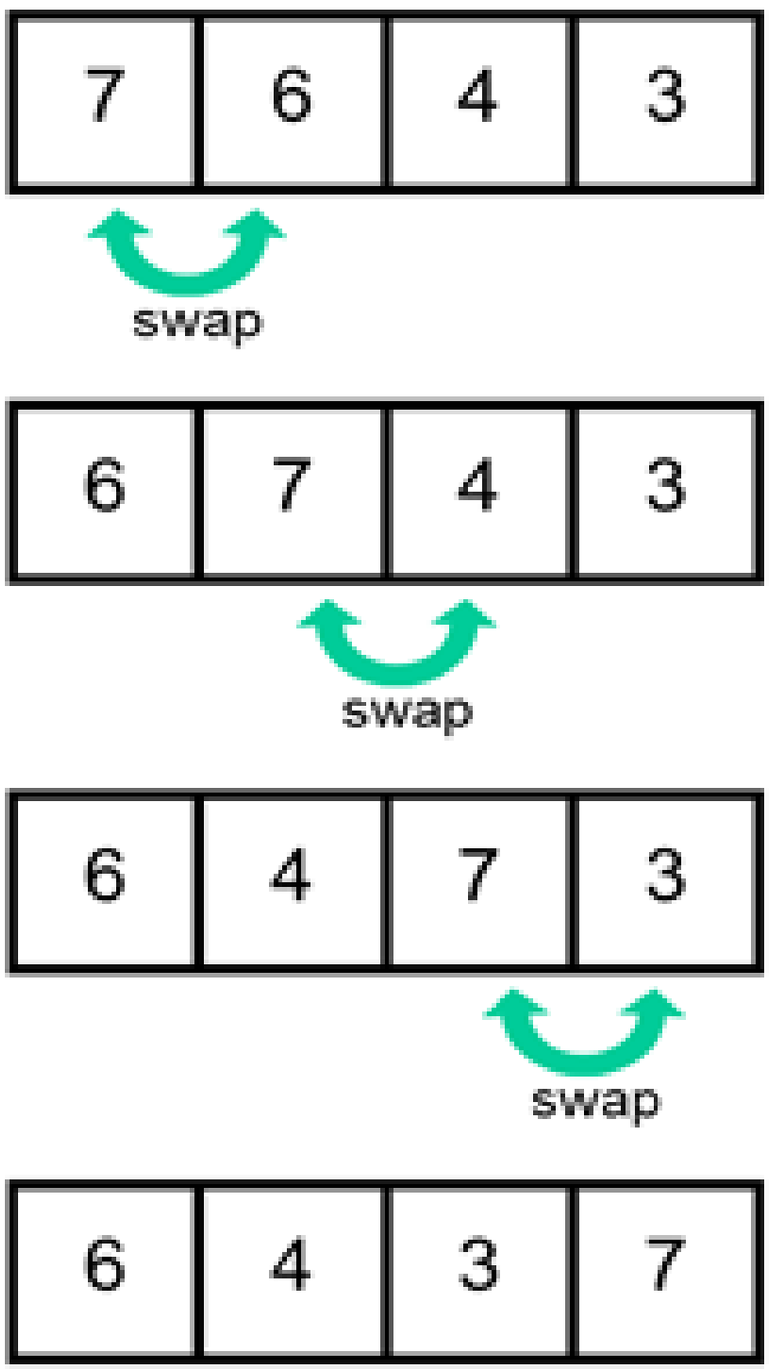
- Bubble sort is easy to understand and implement.
- It does not require any additional memory space.
- It is a stable sorting algorithm, meaning that elements with the same key value maintain their relative order in the sorted output.

Disadvantages of Bubble Sort:

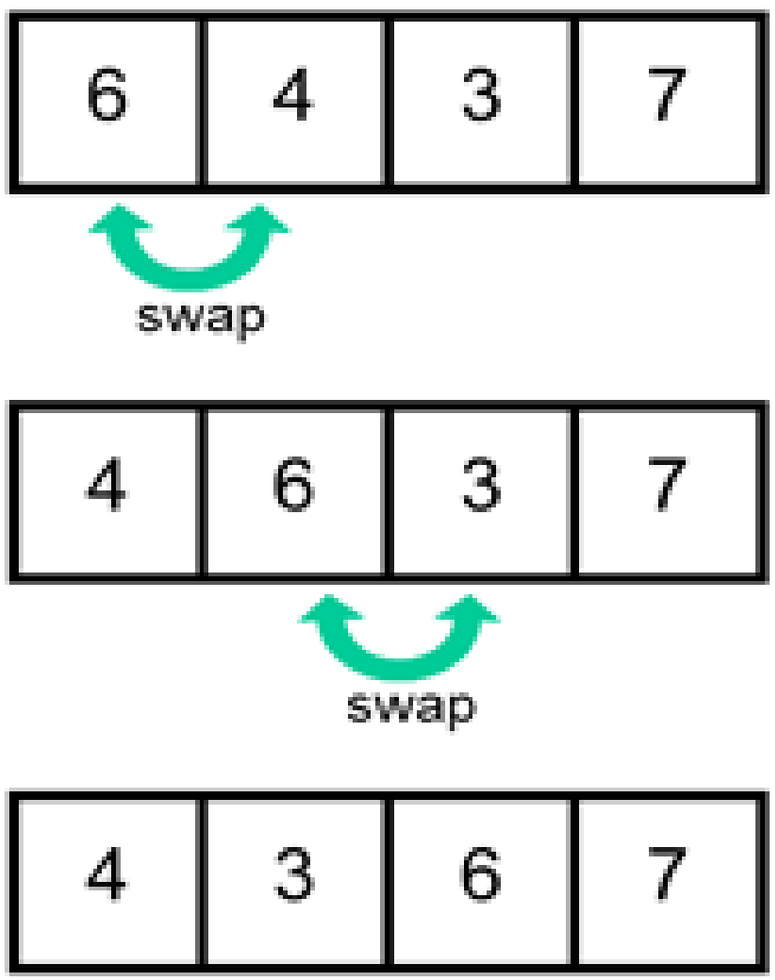
- Bubble sort has a time complexity of O(N2) which makes it very slow for large data sets.
- Bubble sort is a comparison-based sorting algorithm, which means that it requires a comparison operator to determine the relative order of elements in the input data set. It can limit the efficiency of the algorithm in certain cases.

Bubble Sort Example

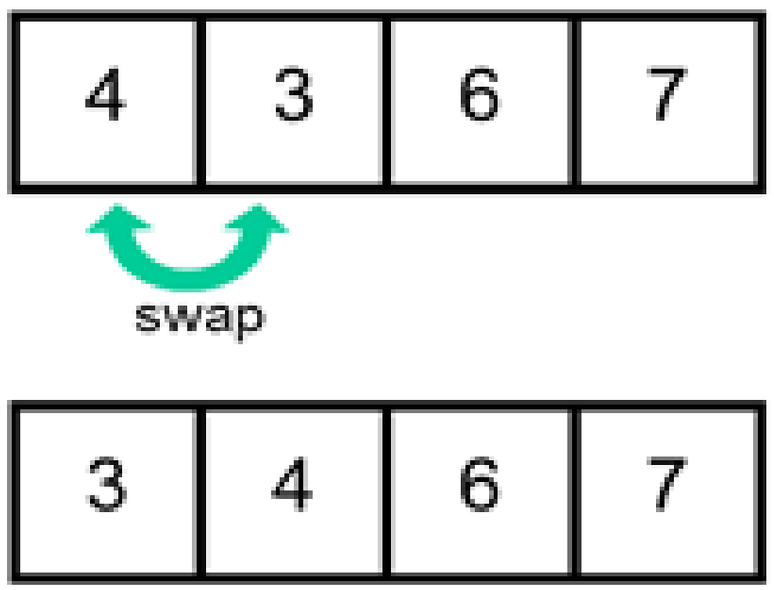
First pass



Second pass



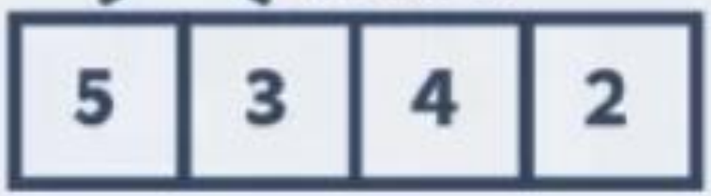
Third pass



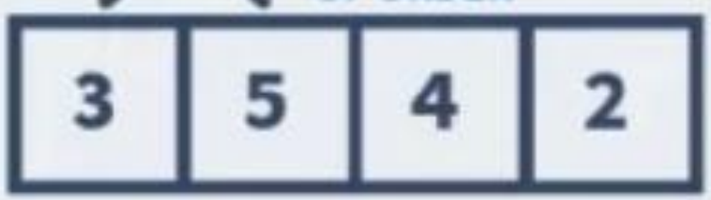
Bubble Sort Example

FIRST PASS

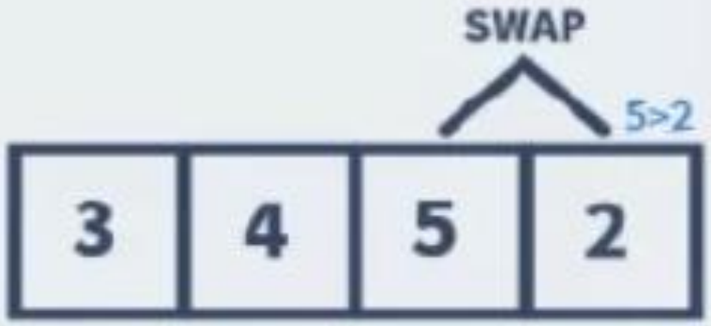
SWAP — 5>3, ELEMENTS ARE OUT OF ORDER



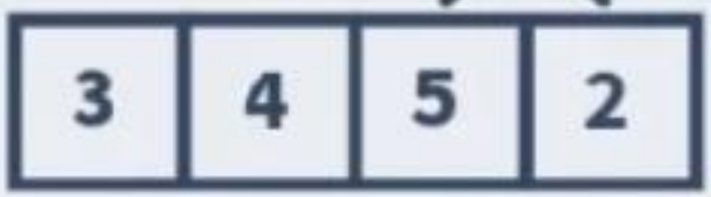
SWAP — 5>3, ELEMENTS ARE OUT OF ORDER



SWAP — 5>2

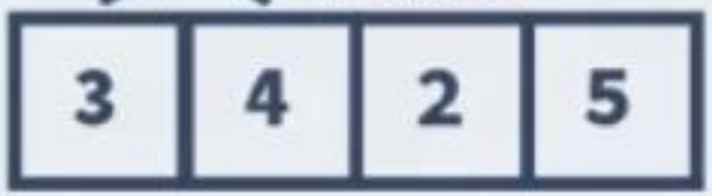


SWAP — 5>2

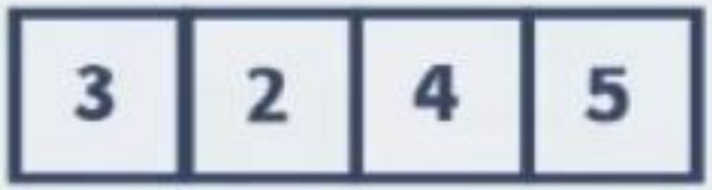


SECOND PASS

NO SWAP — 5>3, ELEMENTS ARE OUT OF ORDER



SWAP — 4>2

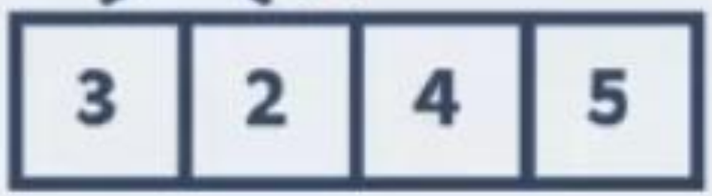


Second-Highest element obtained on the right-hand side.

We will not compare 4 and 5 because we know that they are already sorted.

THIRD PASS

SWAP — 3>2



We got the sortrd array.

In this case, value 33 is greater than 14, so it is already in sorted locations. Next, we compare 33 with 27.

We find that 27 is smaller than 33 and these two values must be swapped.

The new array should look like this –

Next we compare 33 and 35. We find that both are in already sorted positions.

https://www.youtube.com/watch?v=NiyEqLZmngY

Then we move to the next two values, 35 and 10.

We know then that 10 is smaller 35. Hence they are not sorted.

We swap these values. We find that we have reached the end of the array. After one iteration, the array should look like this –

To be precise, we are now showing how an array should look like after each iteration. After the second iteration, it should look like this –

Notice that after each iteration, at least one value moves at the end.

And when there's no swap required, bubble sorts learns that an array is completely sorted.

Now we should look into some practical aspects of bubble sort.

Algorithm

We assume **list** is an array of **n** elements. We further assume that **swap** function swaps the values of the given array elements.

```
begin BubbleSort(list)

   for all elements of list
      if list[i] > list[i+1]
         swap(list[i], list[i+1])
      end if
   end for

   return list

end BubbleSort
```

Implementation in C

Try it

```
#include <stdio.h>
#include <stdbool.h>

#define MAX 10

int list[MAX] = {1,8,4,6,0,3,5,2,7,9};

void display() {
   int i;
   printf("[");

   // navigate through all items
   for(i = 0; i < MAX; i++) {
      printf("%d ",list[i]);
   }

   printf("]\n");
}

void bubbleSort() {
   int temp;
   int i,j;

   bool swapped = false;

   // loop through all numbers
   for(i = 0; i < MAX-1; i++) {
      swapped = false;
```

```
      // loop through numbers falling ahead
      for(j = 0; j < MAX-1-i; j++) {
         printf("     Items compared: [ %d, %d ] ", list[j],list[j+1]);

         // check if next number is lesser than current no
         //   swap the numbers.
         //  (Bubble up the highest number)

         if(list[j] > list[j+1]) {
            temp = list[j];
            list[j] = list[j+1];
            list[j+1] = temp;

            swapped = true;
            printf(" => swapped [%d, %d]\n",list[j],list[j+1]);
         }else {
            printf(" => not swapped\n");
         }

      }

      // if no number was swapped that means
      //   array is sorted now, break the loop.
      if(!swapped) {
         break;
      }

      printf("Iteration %d#: ",(i+1));
      display();
   }

}
```

```
main() {
   printf("Input Array: ");
   display();
   printf("\n");

   bubbleSort();
   printf("\nOutput Array: ");
   display();
}
```

If we compile and run the above program, it will produce the following result –

Output

```
Input Array: [1 8 4 6 0 3 5 2 7 9 ]

     Items compared: [ 1, 8 ]  => not swapped
     Items compared: [ 8, 4 ]  => swapped [4, 8]
     Items compared: [ 8, 6 ]  => swapped [6, 8]
     Items compared: [ 8, 0 ]  => swapped [0, 8]
     Items compared: [ 8, 3 ]  => swapped [3, 8]
     Items compared: [ 8, 5 ]  => swapped [5, 8]
     Items compared: [ 8, 2 ]  => swapped [2, 8]
     Items compared: [ 8, 7 ]  => swapped [7, 8]
     Items compared: [ 8, 9 ]  => not swapped
```

```
Iteration 1#: [1 4 6 0 3 5 2 7 8 9 ]
     Items compared: [ 1, 4 ]  => not swapped
     Items compared: [ 4, 6 ]  => not swapped
     Items compared: [ 6, 0 ]  => swapped [0, 6]
     Items compared: [ 6, 3 ]  => swapped [3, 6]
     Items compared: [ 6, 5 ]  => swapped [5, 6]
     Items compared: [ 6, 2 ]  => swapped [2, 6]
     Items compared: [ 6, 7 ]  => not swapped
     Items compared: [ 7, 8 ]  => not swapped
Iteration 2#: [1 4 0 3 5 2 6 7 8 9 ]
     Items compared: [ 1, 4 ]  => not swapped
     Items compared: [ 4, 0 ]  => swapped [0, 4]
     Items compared: [ 4, 3 ]  => swapped [3, 4]
     Items compared: [ 4, 5 ]  => not swapped
     Items compared: [ 5, 2 ]  => swapped [2, 5]
     Items compared: [ 5, 6 ]  => not swapped
     Items compared: [ 6, 7 ]  => not swapped
Iteration 3#: [1 0 3 4 2 5 6 7 8 9 ]
     Items compared: [ 1, 0 ]  => swapped [0, 1]
     Items compared: [ 1, 3 ]  => not swapped
     Items compared: [ 3, 4 ]  => not swapped
     Items compared: [ 4, 2 ]  => swapped [2, 4]
     Items compared: [ 4, 5 ]  => not swapped
     Items compared: [ 5, 6 ]  => not swapped
```

```
Iteration 4#: [0 1 3 2 4 5 6 7 8 9 ]
     Items compared: [ 0, 1 ]  => not swapped
     Items compared: [ 1, 3 ]  => not swapped
     Items compared: [ 3, 2 ]  => swapped [2, 3]
     Items compared: [ 3, 4 ]  => not swapped
     Items compared: [ 4, 5 ]  => not swapped
Iteration 5#: [0 1 2 3 4 5 6 7 8 9 ]
     Items compared: [ 0, 1 ]  => not swapped
     Items compared: [ 1, 2 ]  => not swapped
     Items compared: [ 2, 3 ]  => not swapped
     Items compared: [ 3, 4 ]  => not swapped

Output Array: [0 1 2 3 4 5 6 7 8 9 ]
```

Original array with n = 6 5 9 6 2 8 1

First pass i = 1

Comparisons
- j = 0: 5 9 6 2 8 1
- j = 1: 5 9 6 2 8 1
- j = 2: 5 6 9 2 8 1
- j = 3: 5 6 2 9 8 1
- j = 4: 5 6 2 8 9 1

Second pass i = 2

Comparisons
- j = 0: 5 6 2 8 1 9
- j = 1: 5 6 2 8 1 9
- j = 2: 5 2 6 8 1 9
- j = 3: 5 2 6 8 1 9

Third pass i = 3

Comparisons
- j = 0: 5 2 6 1 8 9
- j = 1: 2 5 6 1 8 9
- j = 2: 2 5 6 1 8 9

Fourth pass i = 4

Comparisons
- j = 0: 2 5 6 1 8 9
- j = 1: 2 5 1 6 8 9

Fifth pass i = 5

Comparisons
- j = 0: 2 1 5 6 8 9

Sorted array

a[ ] → 1 2 5 6 8 9 9

Example 8.3.1

Show output of each pass using bubble sort to arrange the following nos. in ascending order. Write pseudo C code for bubble sort : 10, 9, 8, 7, 6, 5, 4, 3, 2, 1.

Solution :

Pass No.	Data at the end of the pass
1.	9, 8, 7, 6, 5, 4, 3, 2, 1, 10
2.	8, 7, 6, 5, 4, 3, 2, 1, 9, 10
3.	7, 6, 5, 4, 3, 2, 1, 8, 9, 10
4.	6, 5, 4, 3, 2, 1, 7, 8, 9, 10
5.	5, 4, 3, 2, 1, 6, 7, 8, 9, 10
6.	4, 3, 2, 1, 5, 6, 7, 8, 9, 10
7.	3, 2, 1, 4, 5, 6, 7, 8, 9, 10
8.	2, 1, 3, 4, 5, 6, 7, 8, 9, 10
9.	1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Write pseudo C code to sort a list of integers using bubble sort. Show output of each pass for the following list :

10, 5, 4, 18, 17, 1, 2.

Solution :

Pass – I, i = 1 :

Pass – I , i = 1

j = 0	10	5	4	18	17	1	2
j = 1	5	10	4	18	17	1	2
j = 2	5	4	10	18	17	1	2
j = 3	5	4	10	18	17	1	2
j = 4	5	4	10	17	18	1	2
j = 5	5	4	10	17	1	18	2
	5	4	10	17	1	2	18

Pass – II, i = 2

Pass – II , i = 2

j = 0	5	4	10	17	1	2	18
j = 1	4	5	10	17	1	2	18
j = 2	4	5	10	17	1	2	18
j = 3	4	5	10	17	1	2	18
j = 4	4	5	10	1	17	2	18
	4	5	10	1	2	17	18

Pass – III, i = 3

Pass – III , i = 3

j = 0	4	5	10	1	2	17	18
j = 1	4	5	10	1	2	17	18
j = 2	4	5	10	1	2	17	18
j = 3	4	5	1	10	2	17	18
	4	5	1	2	10	17	18

Pass – IV, i = 4

Pass – IV , i = 4

j = 0	4	5	1	2	10	17	18
j = 1	4	5	1	2	10	17	18
j = 2	4	1	5	2	10	17	18
	4	1	2	5	10	17	18

Pass – V, i =5

Pass – V , i = 5

j = 0	4	1	2	5	10	17	18
j = 1	1	4	2	5	10	17	18
	1	2	4	5	10	17	18

Pass – VI, i = 4

Pass – VI , i = 6

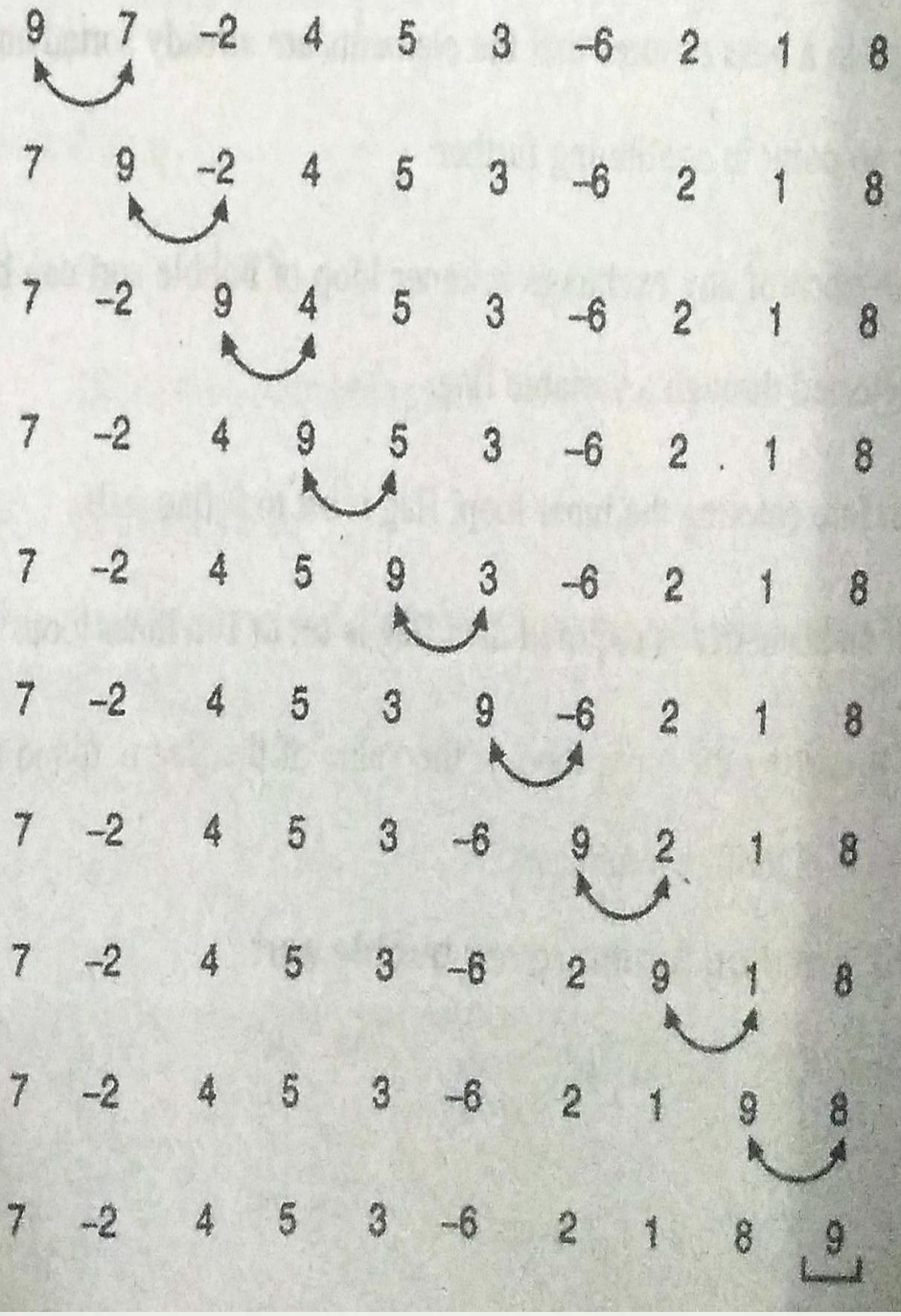
j = 0	1	2	4	5	10	17	18
	1	2	4	5	10	17	18

Sort the following list in ascending order using bubble sort Show all passes. Analyze time complexity.

9, 7, – 2, 4, 5, 3, – 6, 2, 1, 8

Solution :

Pass 1 :

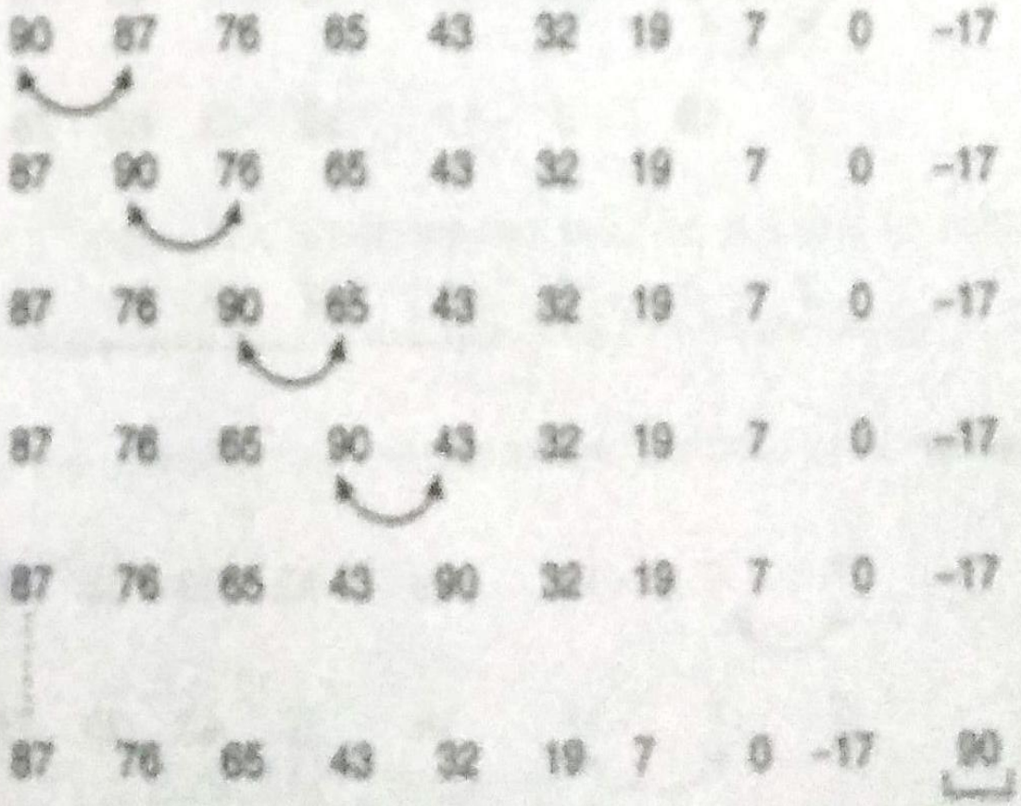


Show the output of each pass using bubble sort to arrange the following numbers in ascending order.

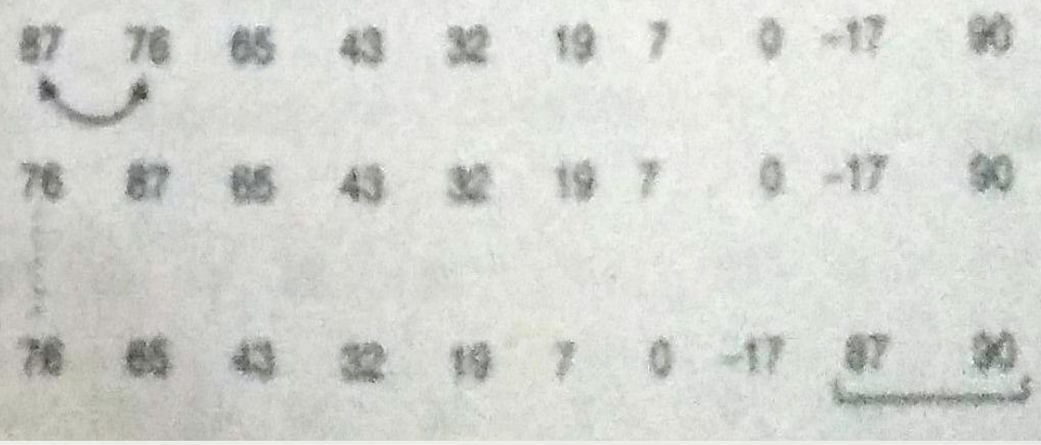
90, 87, 76, 65, 43, 32, 19, 7, 0, – 17.

Solution :

Pass-I



Pass-II



Insertion Sort

This is an in-place comparison-based sorting algorithm. Here, a sub-list is maintained which is always sorted. For example, the lower part of an array is maintained to be sorted. An element which is to be 'insert'ed in this sorted sub-list, has to find its appropriate place and then it has to be inserted there. Hence the name, **insertion sort**.

The array is searched sequentially and unsorted items are moved and inserted into the sorted sub-list (in the same array). This algorithm is not suitable for large data sets as its average and worst case complexity are of $O(n^2)$, where **n** is the number of items.

How Insertion Sort Works?

We take an unsorted array for our example.

Insertion sort compares the first two elements.

It finds that both 14 and 33 are already in ascending order. For now, 14 is in sorted sub-list.

Insertion sort moves ahead and compares 33 with 27.

And finds that 33 is not in the correct position.

It swaps 33 with 27. It also checks with all the elements of sorted sub-list. Here we see that the sorted sub-list has only one element 14, and 27 is greater than 14. Hence, the sorted sub-list remains sorted after swapping.

By now we have 14 and 27 in the sorted sub-list. Next, it compares 33 with 10.

These values are not in a sorted order.

So we swap them.

However, swapping makes 27 and 10 unsorted.

Hence, we swap them too.

Again we find 14 and 10 in an unsorted order.

We swap them again. By the end of third iteration, we have a sorted sub-list of 4 items.

This process goes on until all the unsorted values are covered in a sorted sub-list. Now we shall see some programming aspects of insertion sort.

Algorithm

Now we have a bigger picture of how this sorting technique works, so we can derive simple steps by which we can achieve insertion sort.

```
Step 1 - If it is the first element, it is already sorted. return 1;
Step 2 - Pick next element
Step 3 - Compare with all elements in the sorted sub-list
Step 4 - Shift all the elements in the sorted sub-list that is greater than the
         value to be sorted
Step 5 - Insert the value
Step 6 - Repeat until list is sorted
```

Insertion Sort

Advantages of Insertion Sort:

- Simple and easy to understand and implement.
- Efficient for small data sets or nearly sorted data.
- In-place sorting algorithm, meaning it doesn’t require extra memory.
- Stable sorting algorithm, meaning it maintains the relative order of equal elements in the input array.

Disadvantages of Insertion Sort:

- Inefficient for large data sets or reverse-ordered data, with a worst-case time complexity of O(n^2).
- Insertion sort has a lot of swaps, which can make it slow on modern computers.

Insertion Sort

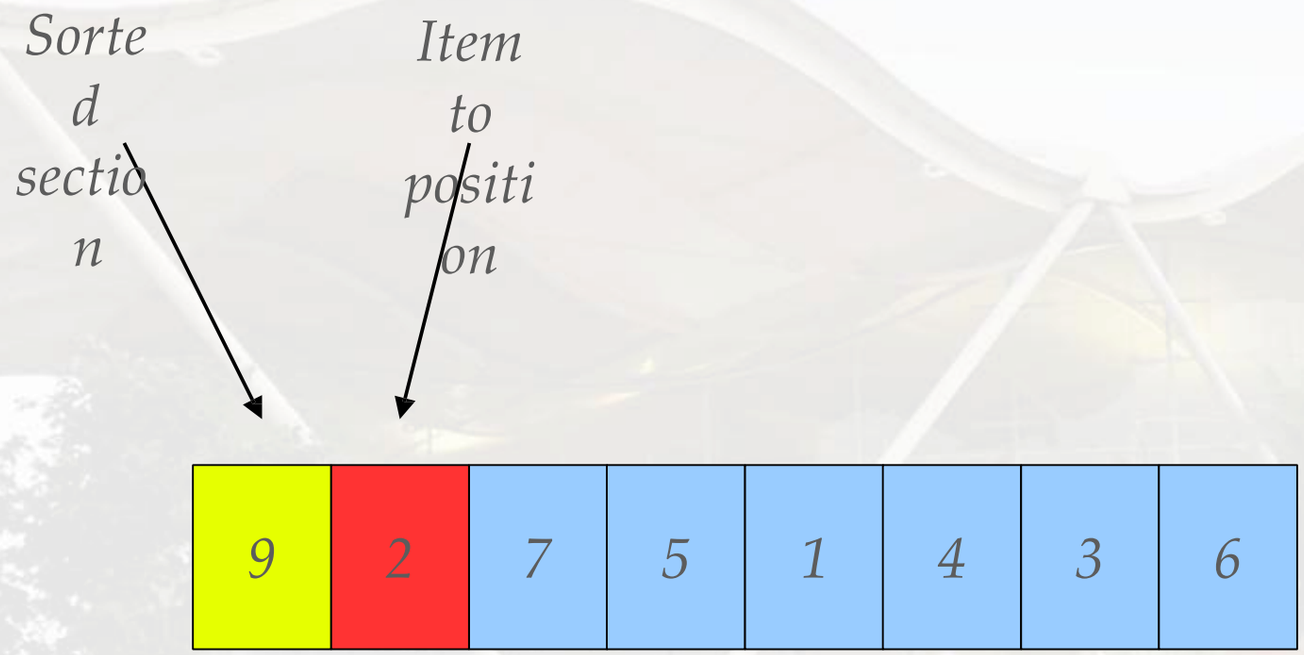
Example :

9	2	7	5	1	4	3	6

Insertion Sort

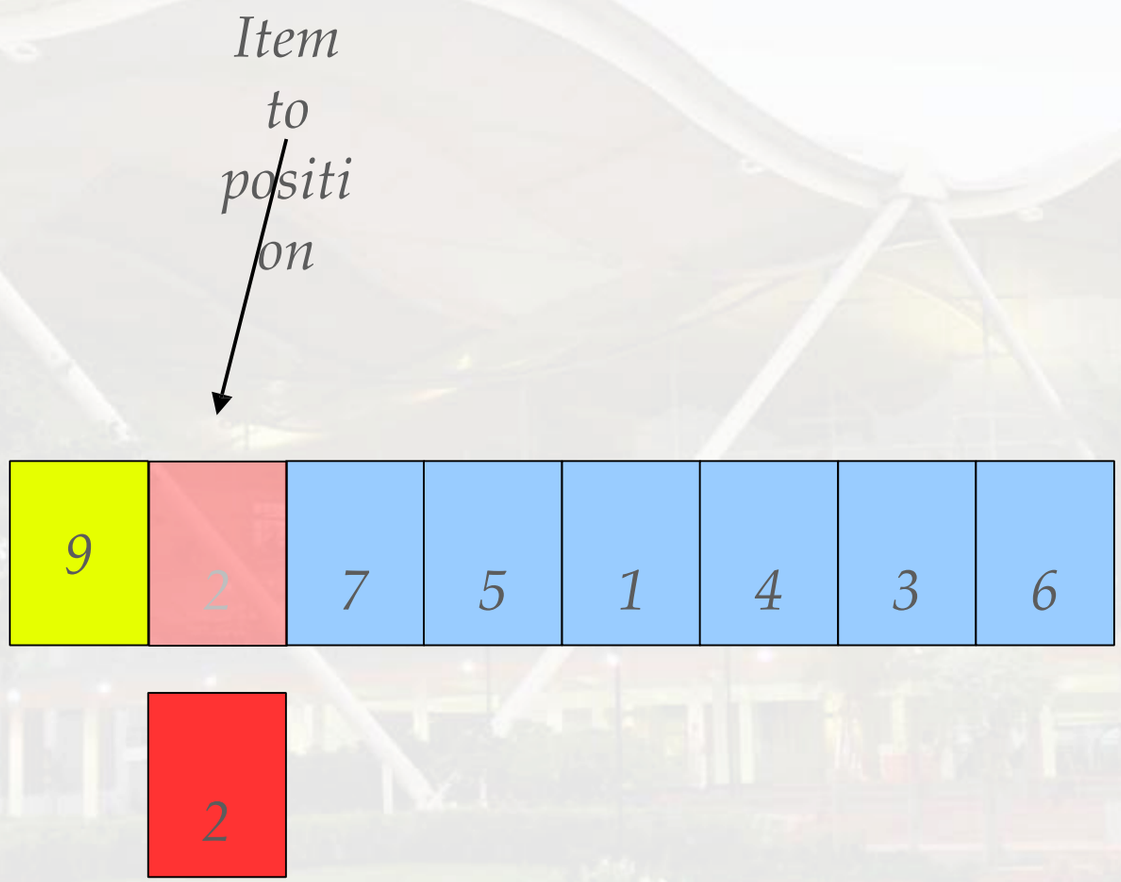
We start by dividing the array in a sorted section and an unsorted section. We put the first element as the only element in the sorted section, and the rest of the array is the unsorted section.

Insertion Sort



The first element in the unsorted section is the next element to be put into the correct position.

Insertion Sort

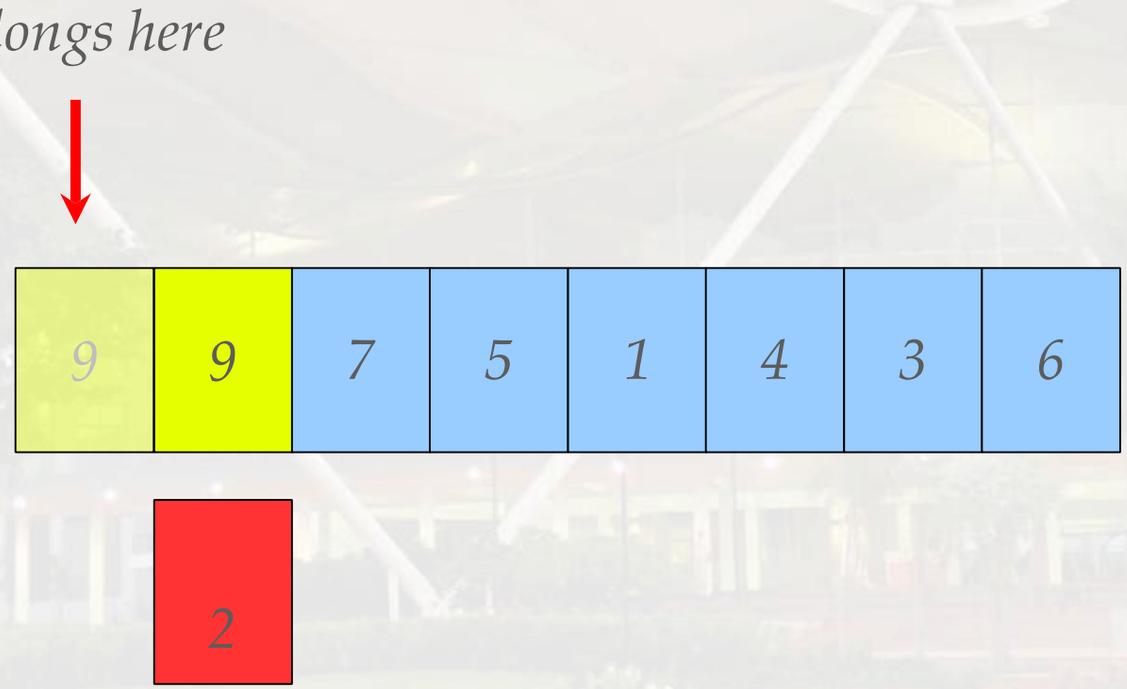


We copy the element to be placed into another variable so it doesn't get overwritten.

Insertion Sort

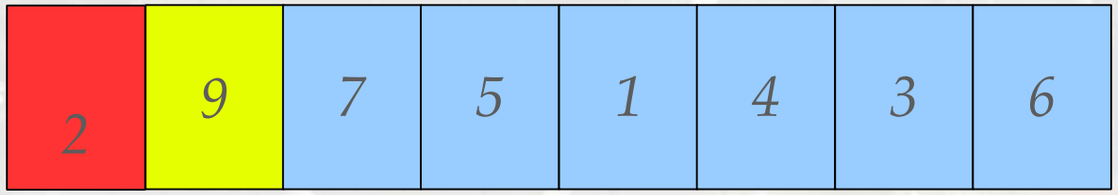
If the previous position is more than the item being placed, copy the value into the next position

Insertion Sort

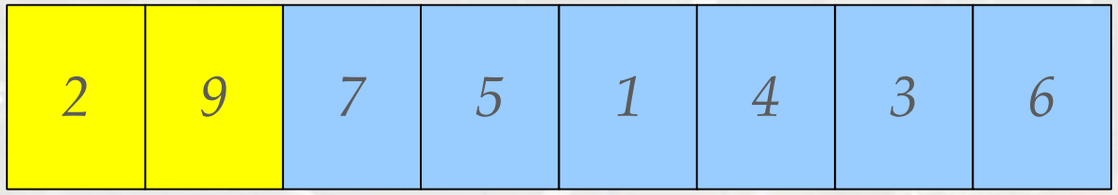


If there are no more items in the sorted section to compare with, the item to be placed must go at the front.

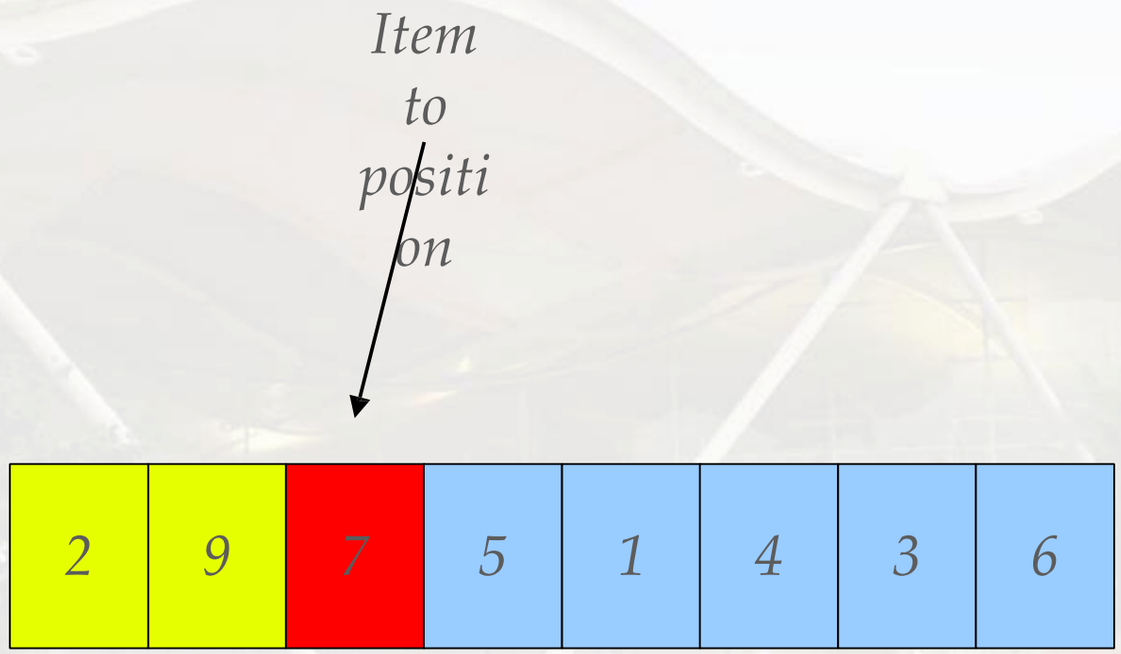
Insertion Sort



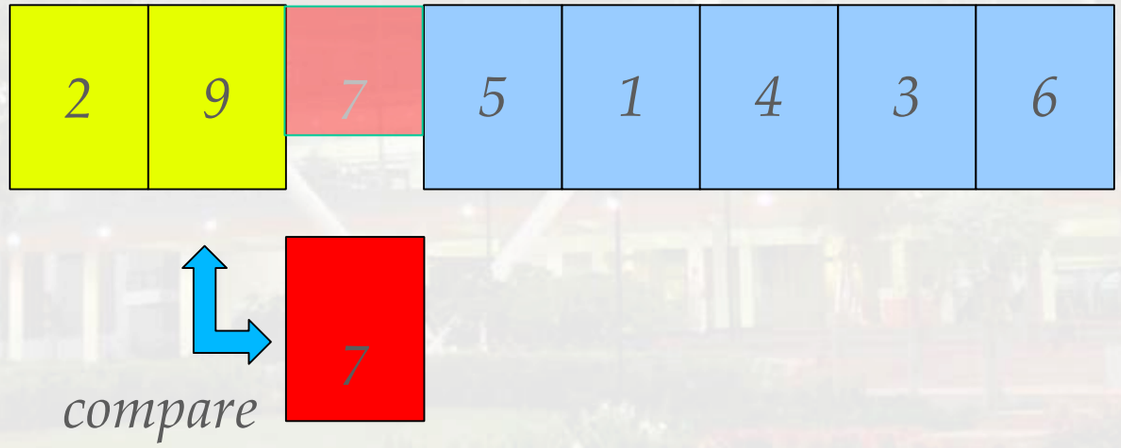
Insertion Sort



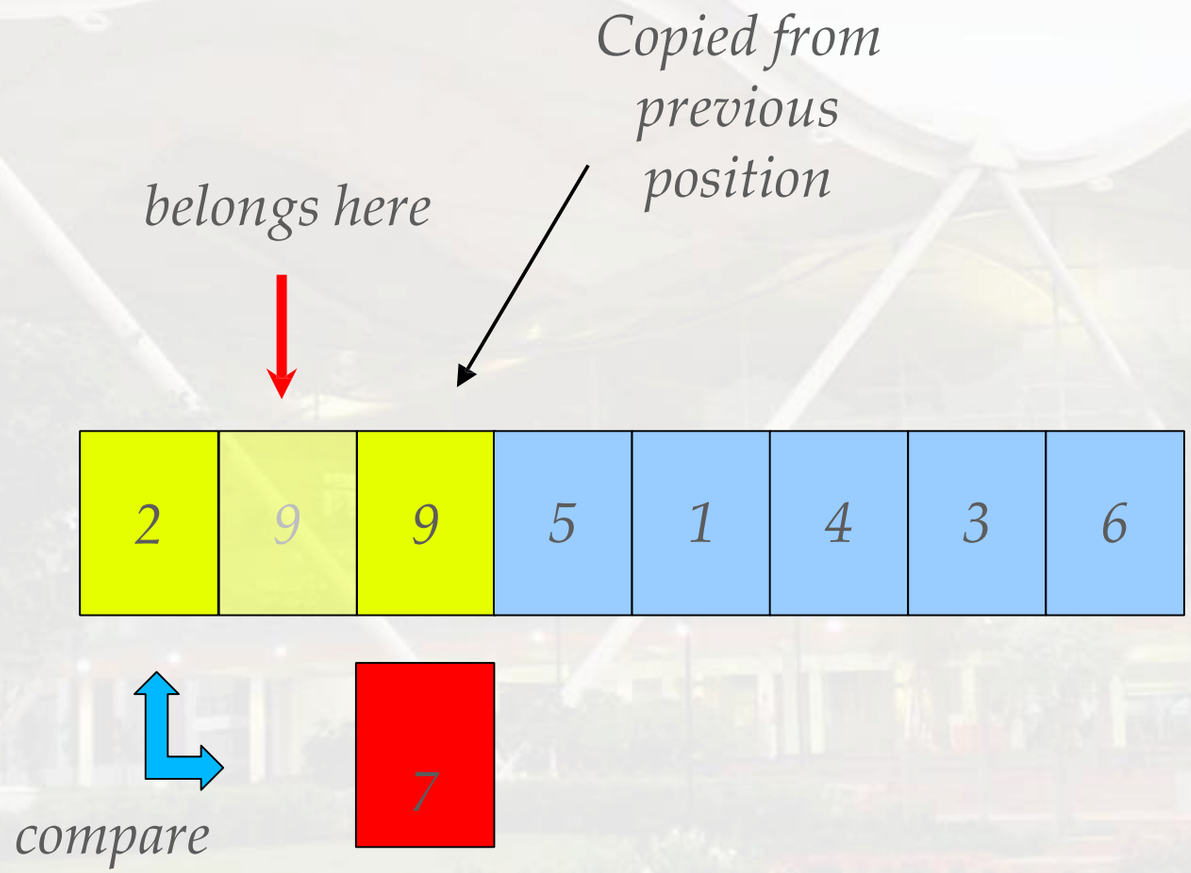
Insertion Sort



Insertion Sort

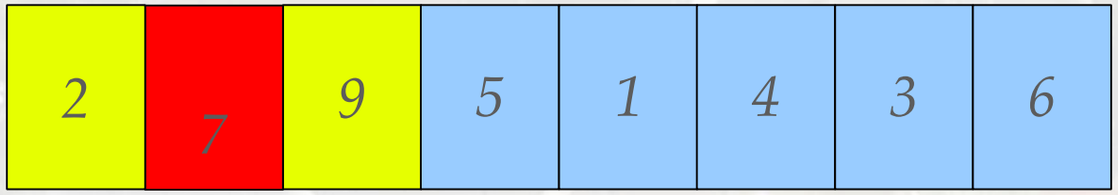


Insertion Sort

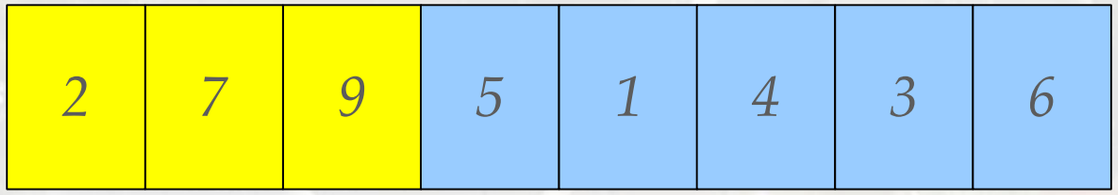


If the item in the sorted section is less than the item to place, the item to place goes after it in the array.

Insertion Sort

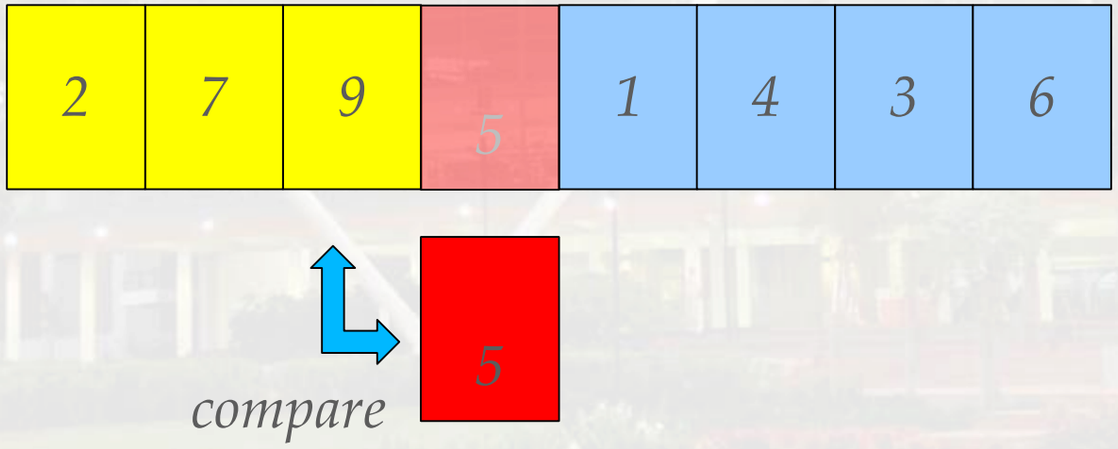


Insertion Sort

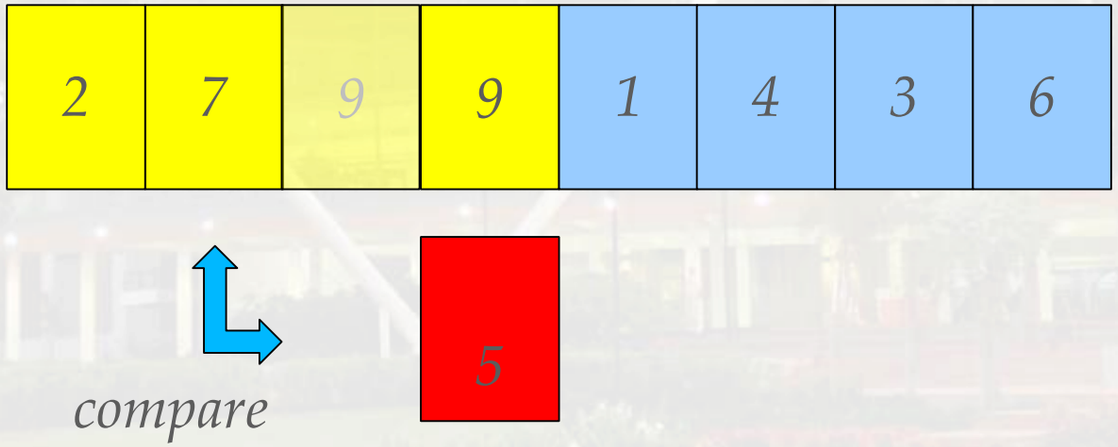


Insertion Sort

Insertion Sort

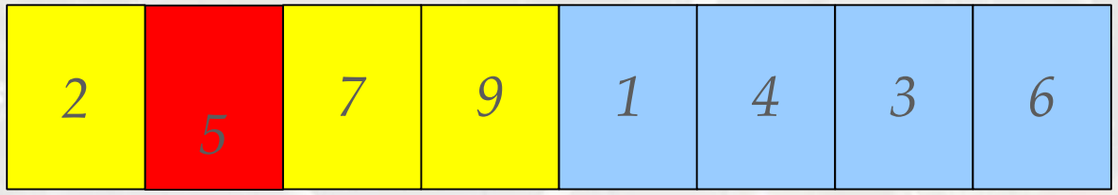


Insertion Sort

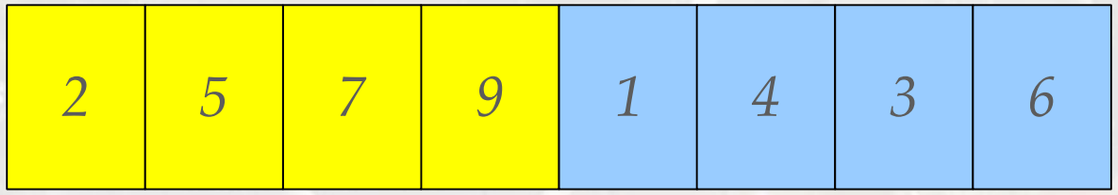


Insertion Sort

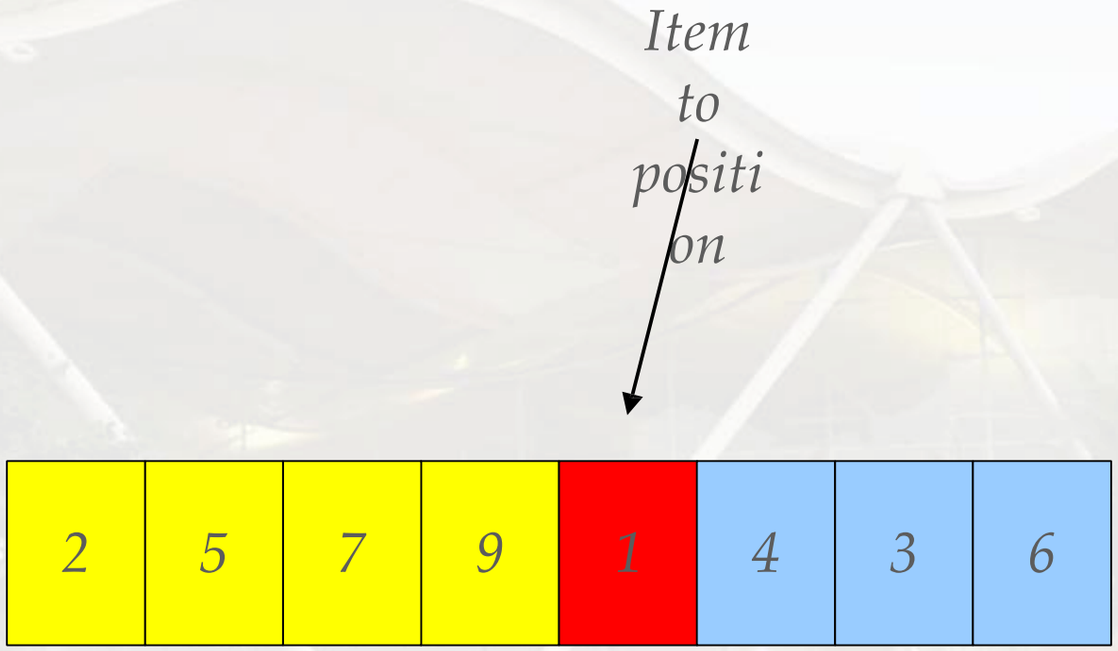
Insertion Sort



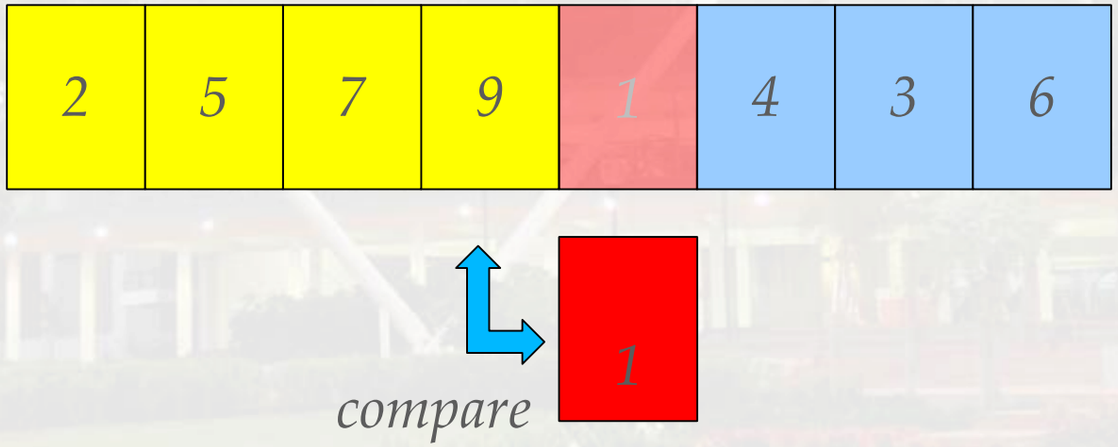
Insertion Sort



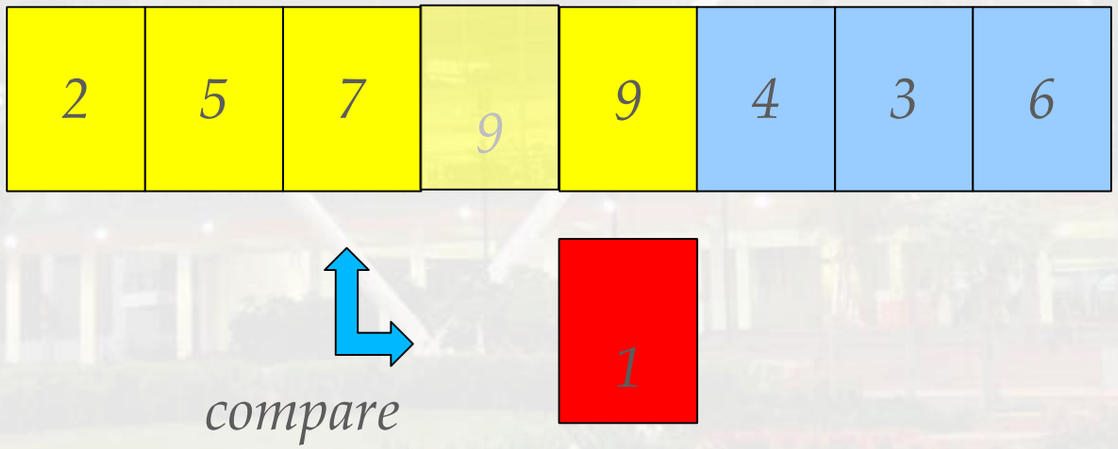
Insertion Sort



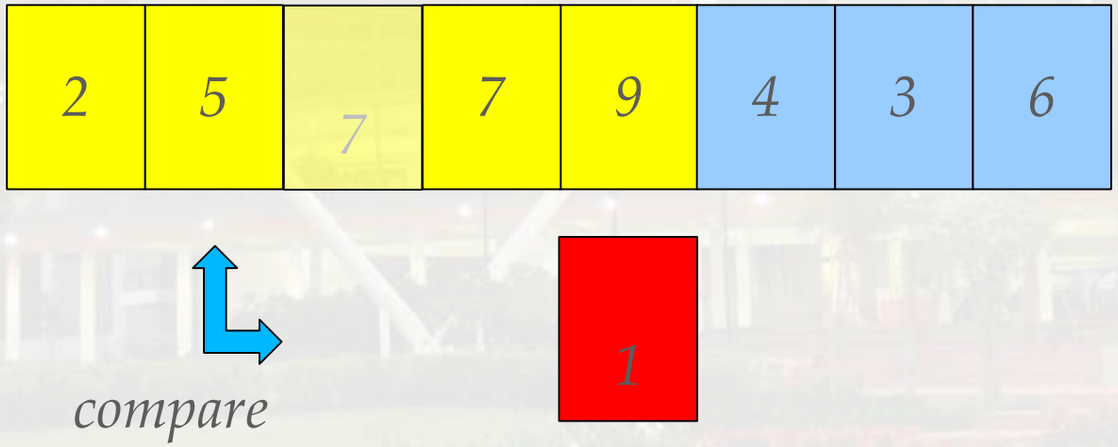
Insertion Sort



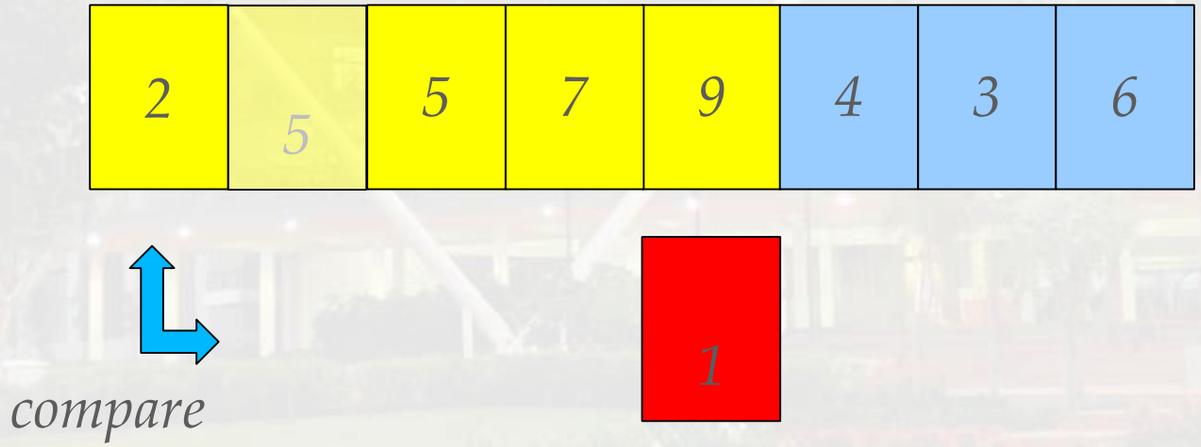
Insertion Sort



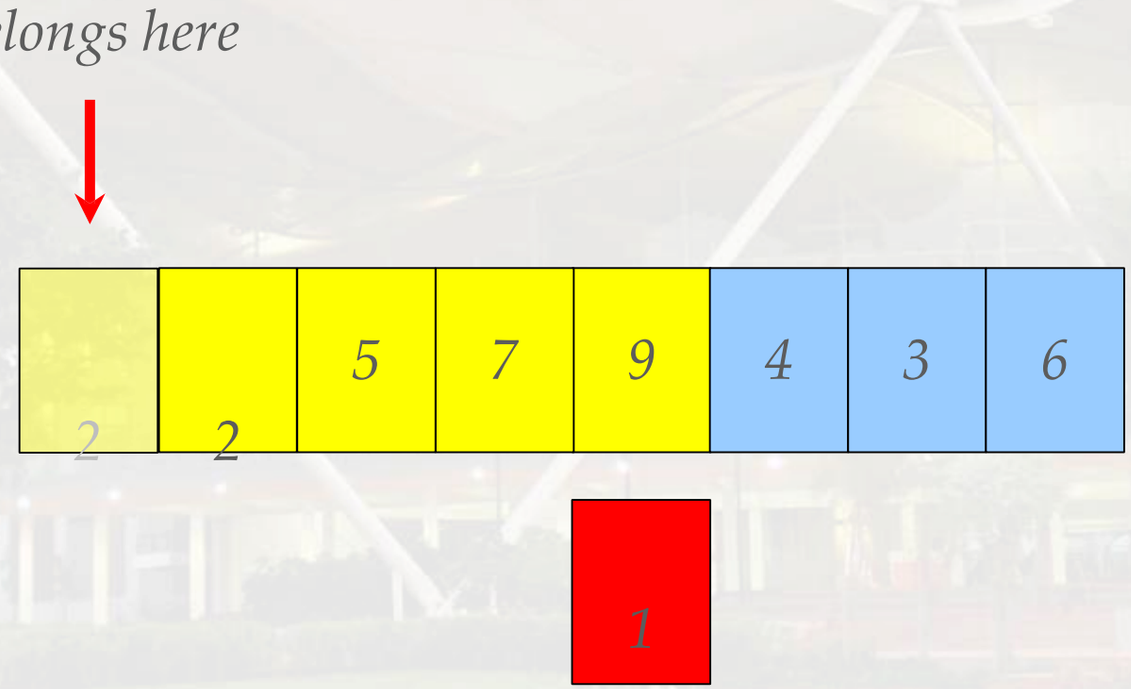
Insertion Sort



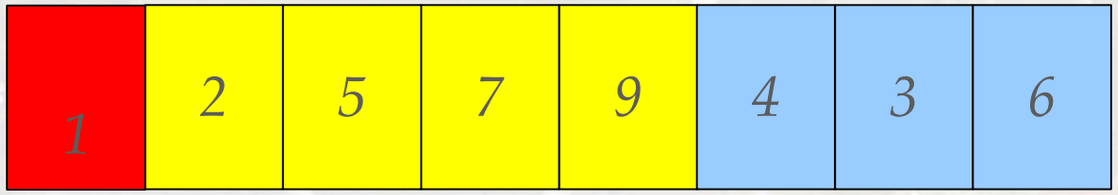
Insertion Sort



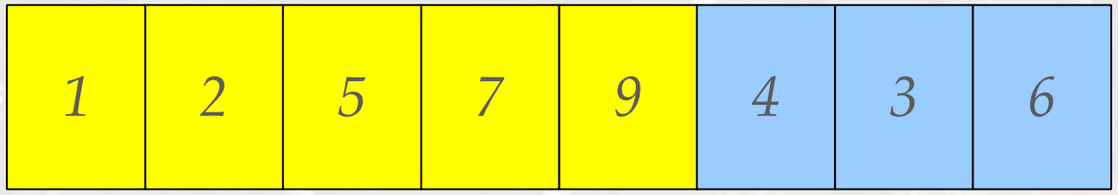
Insertion Sort



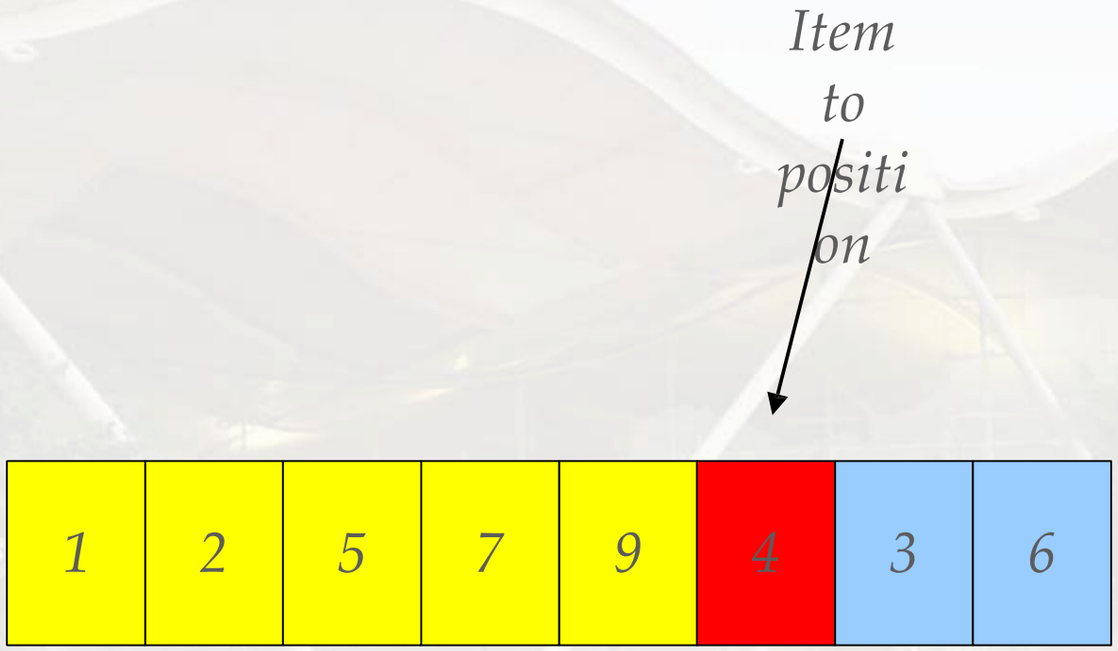
Insertion Sort



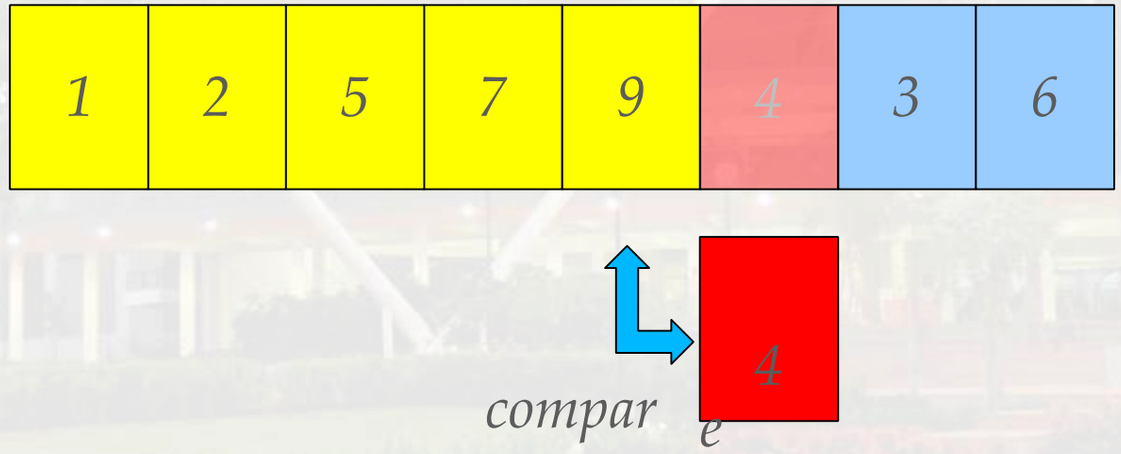
Insertion Sort



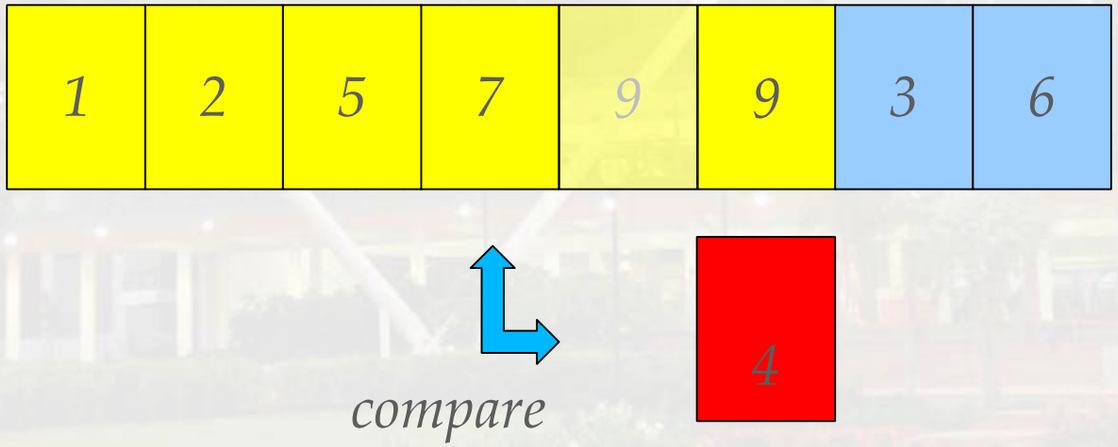
Insertion Sort



Insertion Sort



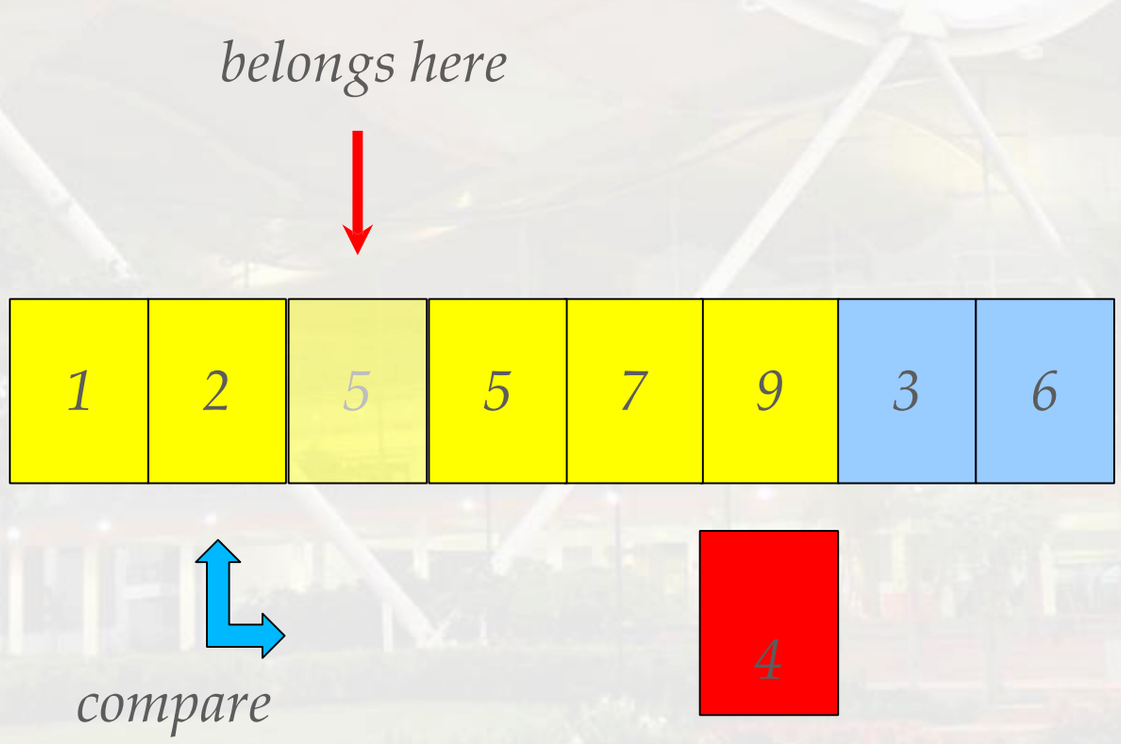
Insertion Sort



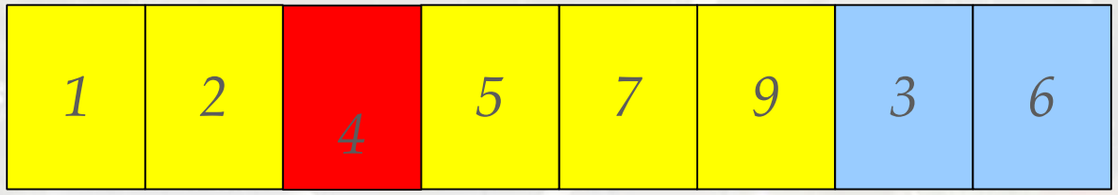
Insertion Sort

compare

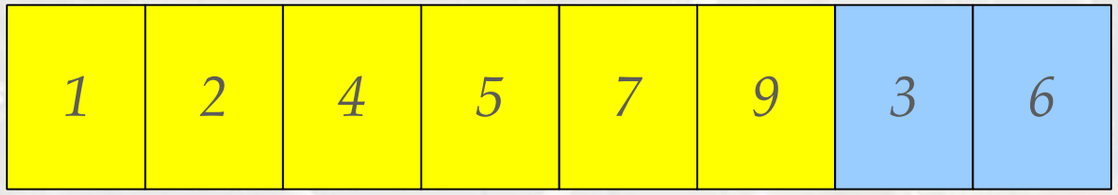
Insertion Sort



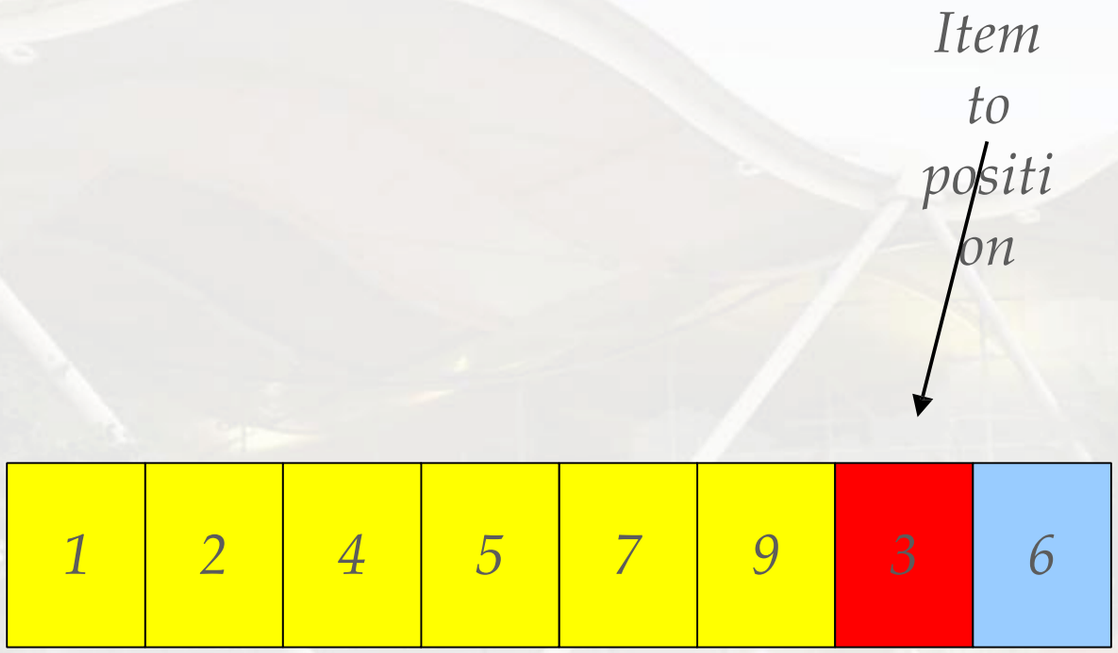
Insertion Sort



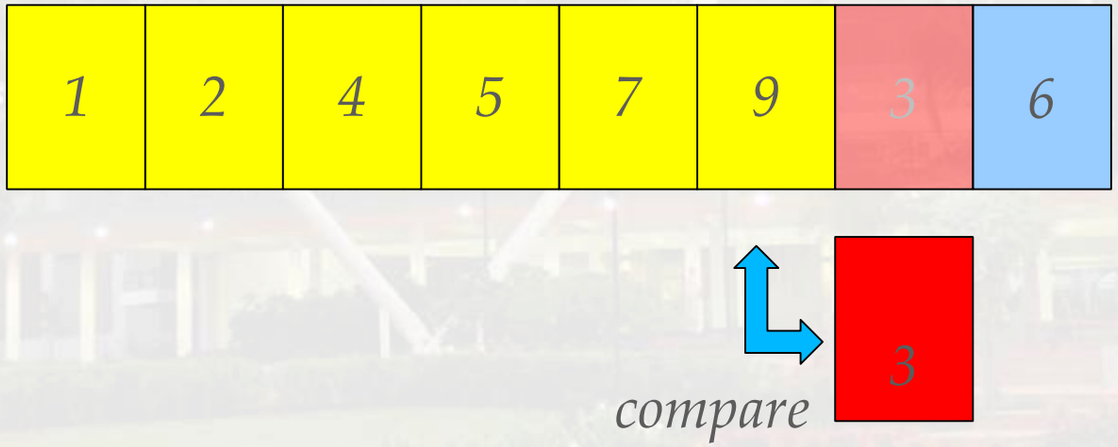
Insertion Sort



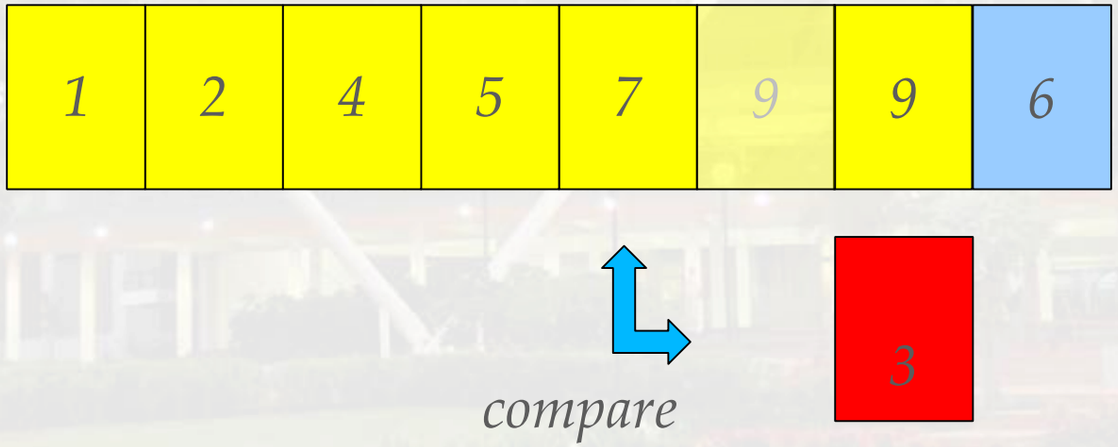
Insertion Sort



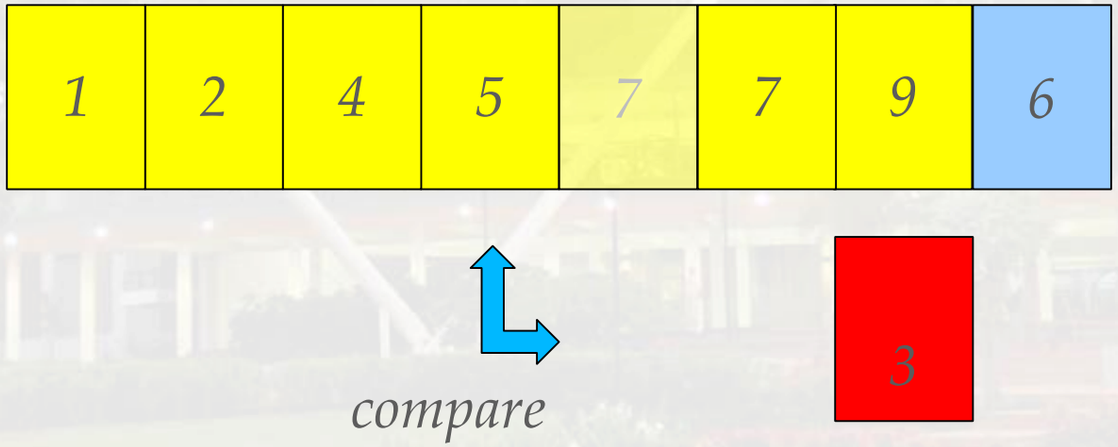
Insertion Sort



Insertion Sort



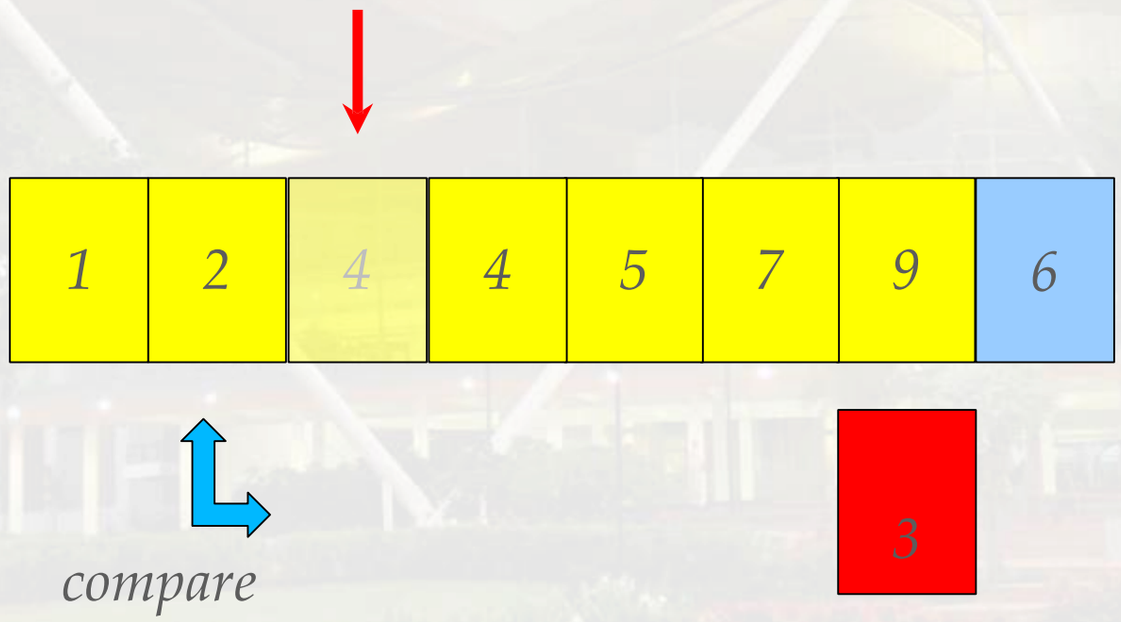
Insertion Sort



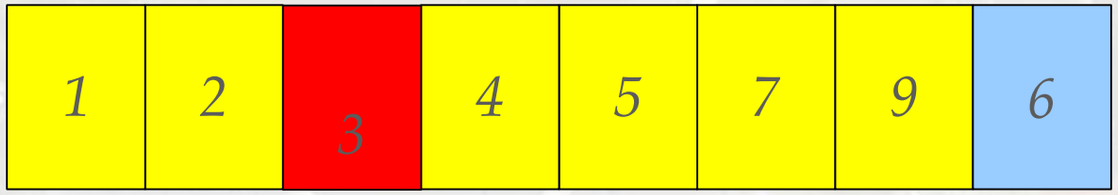
Insertion Sort

Insertion Sort

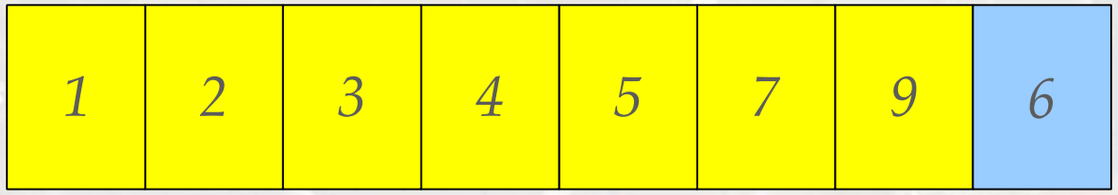
Insertion Sort



Insertion Sort



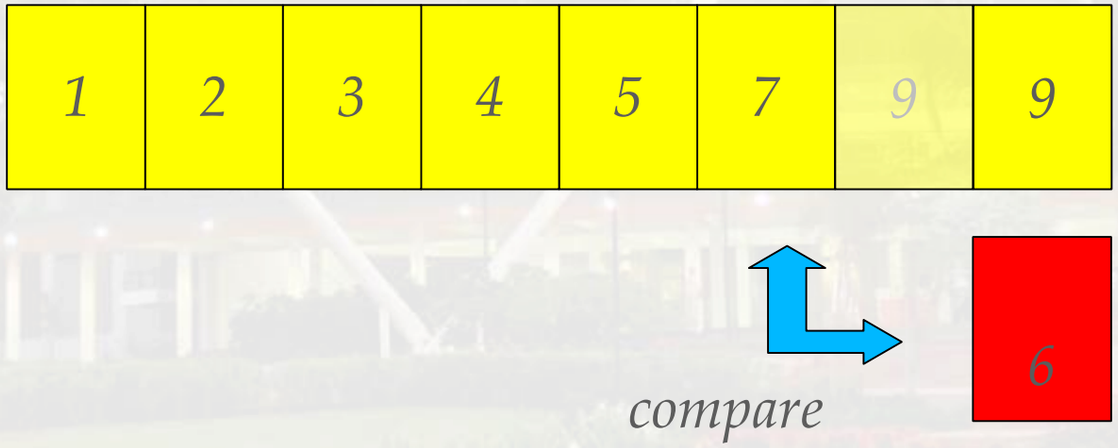
Insertion Sort



Insertion Sort

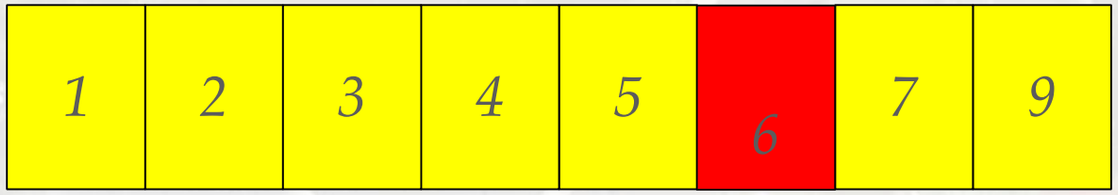
Insertion Sort

Insertion Sort



Insertion Sort

Insertion Sort

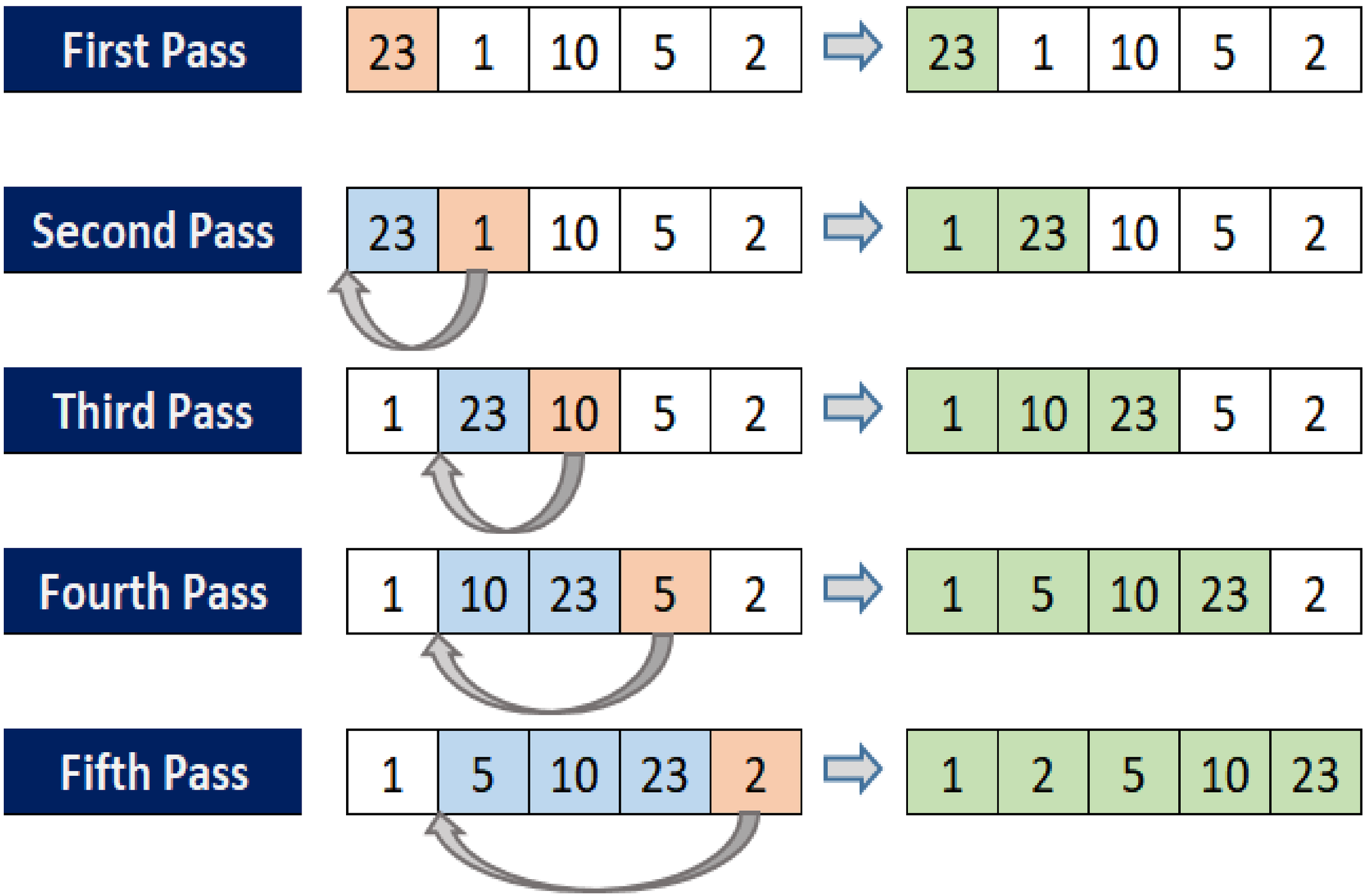


Insertion Sort

1	2	3	4	5	6	7	9

SORTED !

Examples of INSERTION SORT

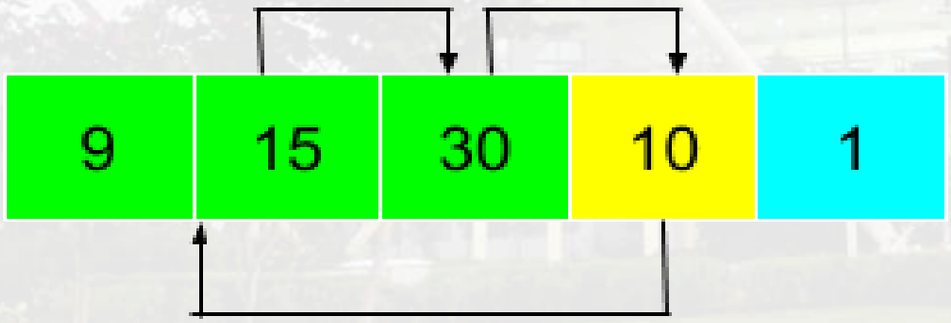


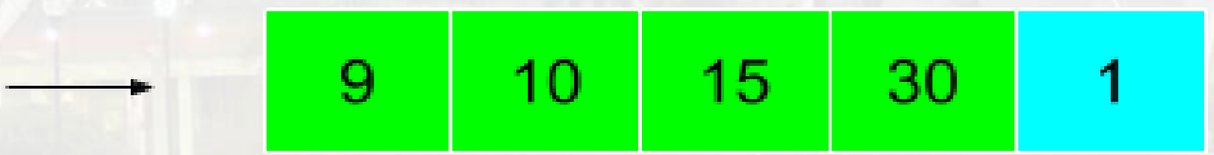
https://www.youtube.com/watch?v=mTNC0ERo-ZI

Examples of INSERTION SORT

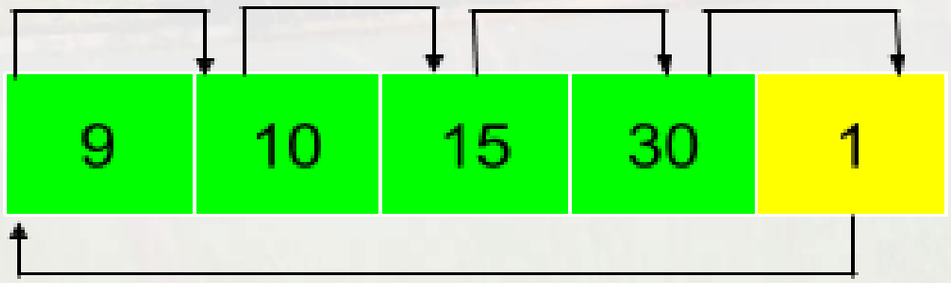
Since 15>9, move 15 to right and place 9 to its correct position.

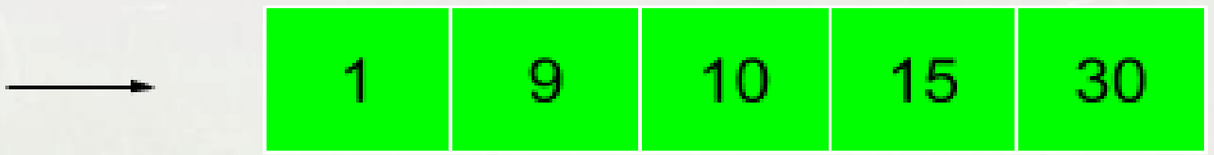
Since 15<30, 30 is at its correct position.





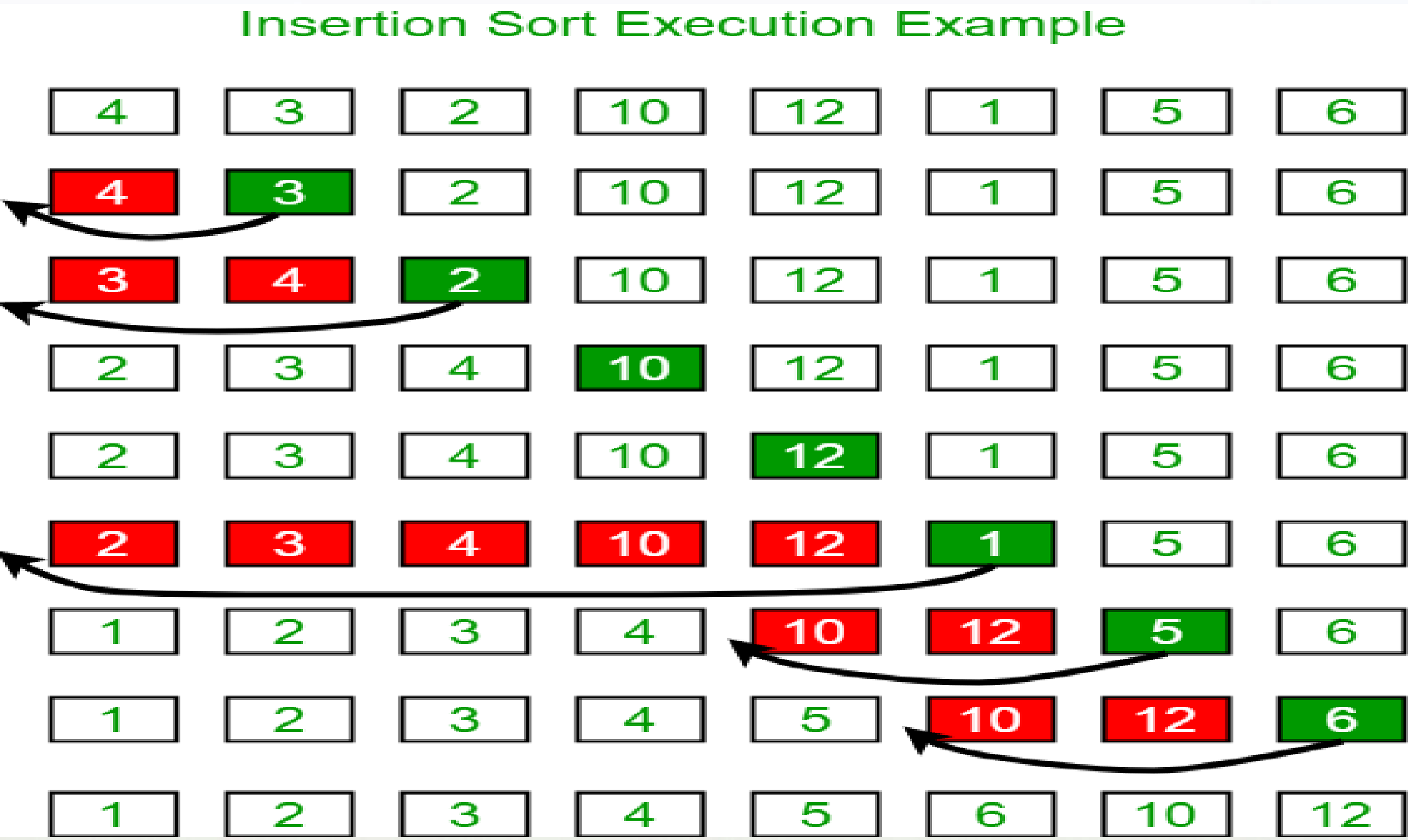
Since 30>10, move 30 to right and compare 10 with 15. Again, 15>10, move 15 to right and compare 9 with 10. As 9<10, place key(10) to A[1], which is correct position of 10.



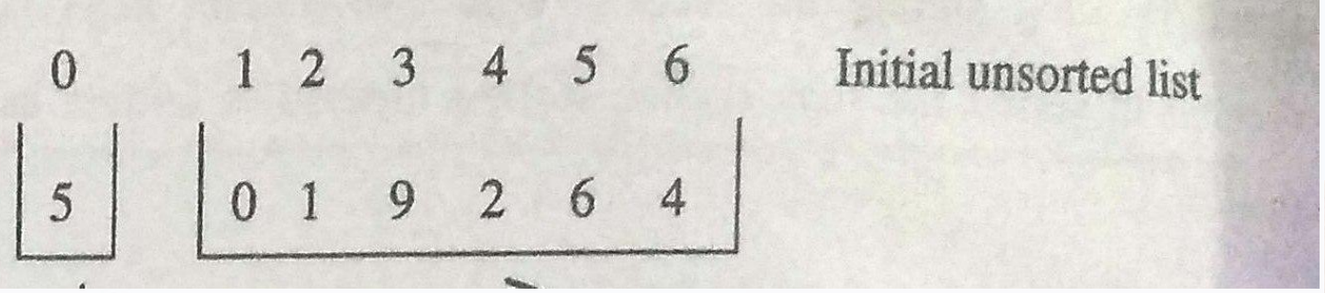


Since 30>1, move 30 to right, likewise move 15, 10 and 9 to right and place key(1) to A[0], which is its correct position.

Examples of INSERTION SORT



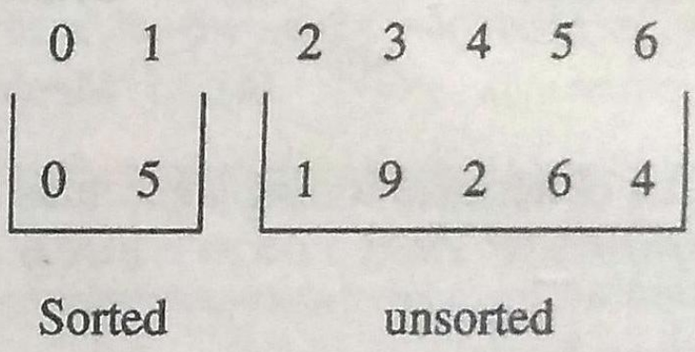
https://www.youtube.com/watch?v=mTNC0ERo-ZI



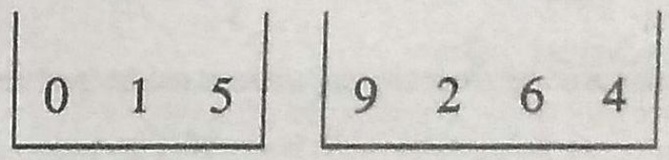
A list of sorted element (a list of single element is always sorted)

a list of unsorted element

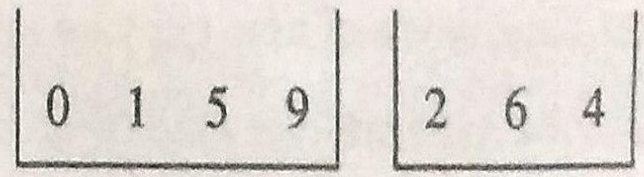
1st iteration (place element at location '1' i.e. a[1], at its orrect place)



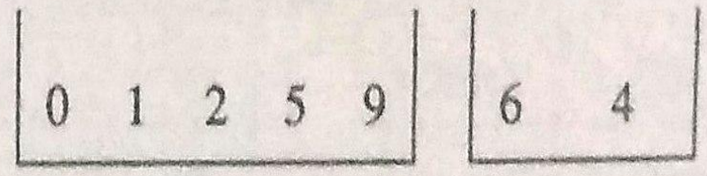
2nd iteration (place a[2] at its correct place)



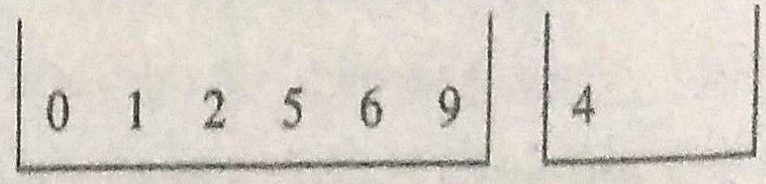
3rd iteration (place a[3] at its correct place)



4th iteration (Place a[4] at its correct place)



5th iteration (Place a[5] at its correct place)



6th iteration (Place a[6] at its correct place)

0 1 2 4 5 6 9

Fig. 8.2.1 : Sorting of elements using insertion sort

Elements	20	10	8	6	4	2	1	–	initially

							pass	Positions moved
10	20	8	6	4	2	1	1	1
8	10	20	6	4	2	1	2	2
6	8	10	20	4	2	1	3	3
4	6	8	10	20	2	1	4	4
2	4	6	8	10	20	1	5	5
1	2	4	6	8	10	20	6	6

Show all passes to sort the values in descending order using insertion sort.

56,12,84,56,28,0,–13,47,94,31,12,– 2

Solution :

Initial	56	12	84	56	28	0	–13	47	94	31	12	–2
After pass 1	56	12	84	56	28	0	–13	47	94	31	12	–2
After pass 2	84	56	12	56	28	0	–13	47	94	31	12	–2
After pass 3	84	56	56	12	28	0	–13	47	94	31	12	–2
After pass 4	84	56	56	28	12	0	–13	47	94	31	12	–2
After pass 5	84	56	56	28	12	0	–13	47	94	31	12	–2
After pass 6	84	56	56	28	12	0	–13	47	94	31	12	–2
After pass 7	84	56	56	47	28	12	0	–13	94	31	12	–2
After pass 8	94	84	56	56	47	28	12	0	–13	31	12	–2
After pass 9	94	84	56	56	47	31	28	12	0	– 13	12	–2
After pass 10	94	84	56	56	47	31	28	12	12	0	13	–2
After pass 11	94	84	56	56	47	31	28	12	12	0	– 13	–2

Sort the following nos. using insertion sort. Show all passes :
50, 10, 78, 40, 30, 02, 04, 15.

Solution :

Initial list

50	10	78	40	30	02	04	15

sorted To be sorted

After pass-1

10	50	78	40	30	02	04	15

sorted

After pass-2

10	50	78	40	30	02	04	15

sorted

After pass-3

10	40	50	78	30	02	04	15

Sorted

After pass-4

10	30	40	50	78	02	04	15

sorted

After pass-5

02	10	30	40	50	78	04	15

sorted

After pass-6

02	04	10	30	40	50	78	15

sorted

After pass-7

02	04	10	15	30	40	50	78

sorted

Selection Sort

Selection sort is a simple sorting algorithm. This sorting algorithm is an in-place comparison-based algorithm in which the list is divided into two parts, the sorted part at the left end and the unsorted part at the right end. Initially, the sorted part is empty and the unsorted part is the entire list.

The smallest element is selected from the unsorted array and swapped with the leftmost element, and that element becomes a part of the sorted array. This process continues moving unsorted array boundary by one element to the right.

This algorithm is not suitable for large data sets as its average and worst case complexities are of $O(n^2)$, where **n** is the number of items.

How Selection Sort Works?

Consider the following depicted array as an example.

For the first position in the sorted list, the whole list is scanned sequentially. The first position where 14 is stored presently, we search the whole list and find that 10 is the lowest value.

PROF. A. N. GHARU

https://www.youtube.com/watch?v=EdUWyka7kpI

So we replace 14 with 10. After one iteration 10, which happens to be the minimum value in the list, appears in the first position of the sorted list.

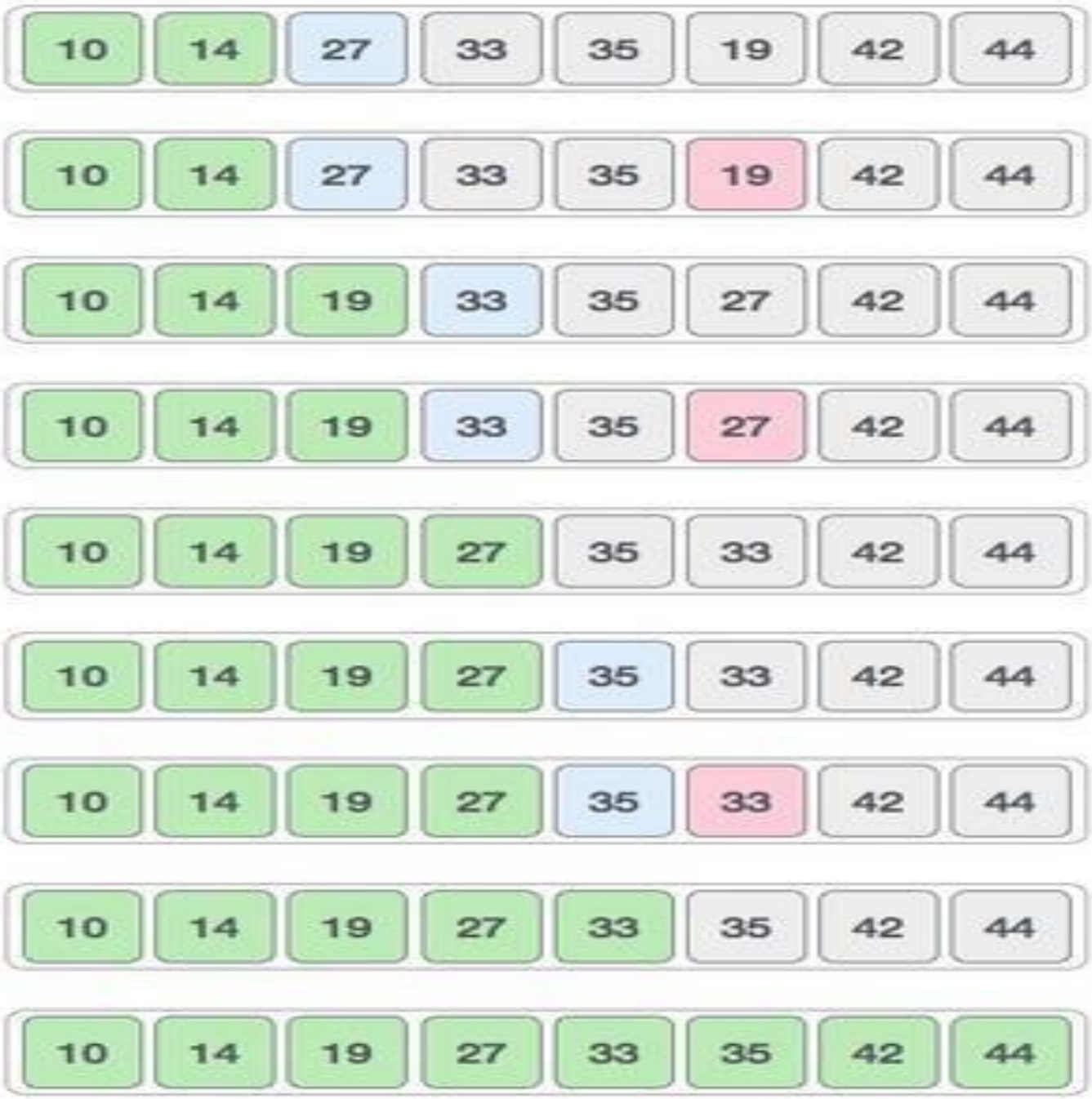
For the second position, where 33 is residing, we start scanning the rest of the list in a linear manner.

We find that 14 is the second lowest value in the list and it should appear at the second place. We swap these values.

After two iterations, two least values are positioned at the beginning in a sorted manner.

The same process is applied to the rest of the items in the array.

Following is a pictorial depiction of the entire sorting process −



Algorithm

```
Step 1 – Set MIN to location 0

Step 2 – Search the minimum element in the list

Step 3 – Swap with value at location MIN

Step 4 – Increment MIN to point to next element

Step 5 – Repeat until list is sorted
```

Pseudocode

```
procedure selection sort
   list  : array of items
   n     : size of list

   for i = 1 to n - 1
   /* set current element as minimum*/
      min = i

      /* check the element to be minimum */

      for j = i+1 to n
         if list[j] < list[min] then
            min = j;
         end if
      end for

      /* swap the minimum element with the current element*/
      if indexMin != i  then
         swap list[min] and list[i]
      end if

   end for

end procedure
```

6.12.1 Algorithm of Selection Sort

1. In first iteration first element is compared with rest of elements. If first element is greater than that then they are swapped.
2. After completion of first iteration smallest element is stored at 0^{th} location.
3. In second iteration second element is compared with rest of (3^{rd} to n^{th} location) elements and process of swapping is repeated.
4. If the list contains n elements, then (n - 1) iterations are required.

Selection Sort

Advantages of Selection Sort:

- Simple and easy to understand and implement.
- Efficient for small data sets or nearly sorted data.
- In-place sorting algorithm, meaning it doesn’t require extra memory.

Disadvantages of Selection Sort:

- Inefficient for large data sets, with a worst-case time complexity of O(n^2).
- Selection sort has a lot of comparisons, which can make it slow on modern computers.
- Unstable sorting algorithm, meaning it may not maintain the relative order of equal elements in the input array.

Insertion vs Selection Sort

Sr. No	Insertion Sort	Selection Sort
1.	Inserts the value in the presorted array to sort the set of values in the array.	Finds the minimum / maximum number from the list and sort it in ascending / descending order.
2.	It is a stable sorting algorithm.	It is an unstable sorting algorithm.
3.	The best-case time complexity is $\Omega(N)$ when the array is already in ascending order. It have $\Theta(N^2)$ in worst case and average case.	For best case, worst case and average selection sort have complexity $\Theta(N^2)$.
4.	The number of comparison operations performed in this sorting algorithm is less than the swapping performed.	The number of comparison operations performed in this sorting algorithm is more than the swapping performed.
5.	It is more efficient than the Selection sort.	It is less efficient than the Insertion sort.

Selection Sort

Elements are : 5 , 9, 1, 11, 2, 4

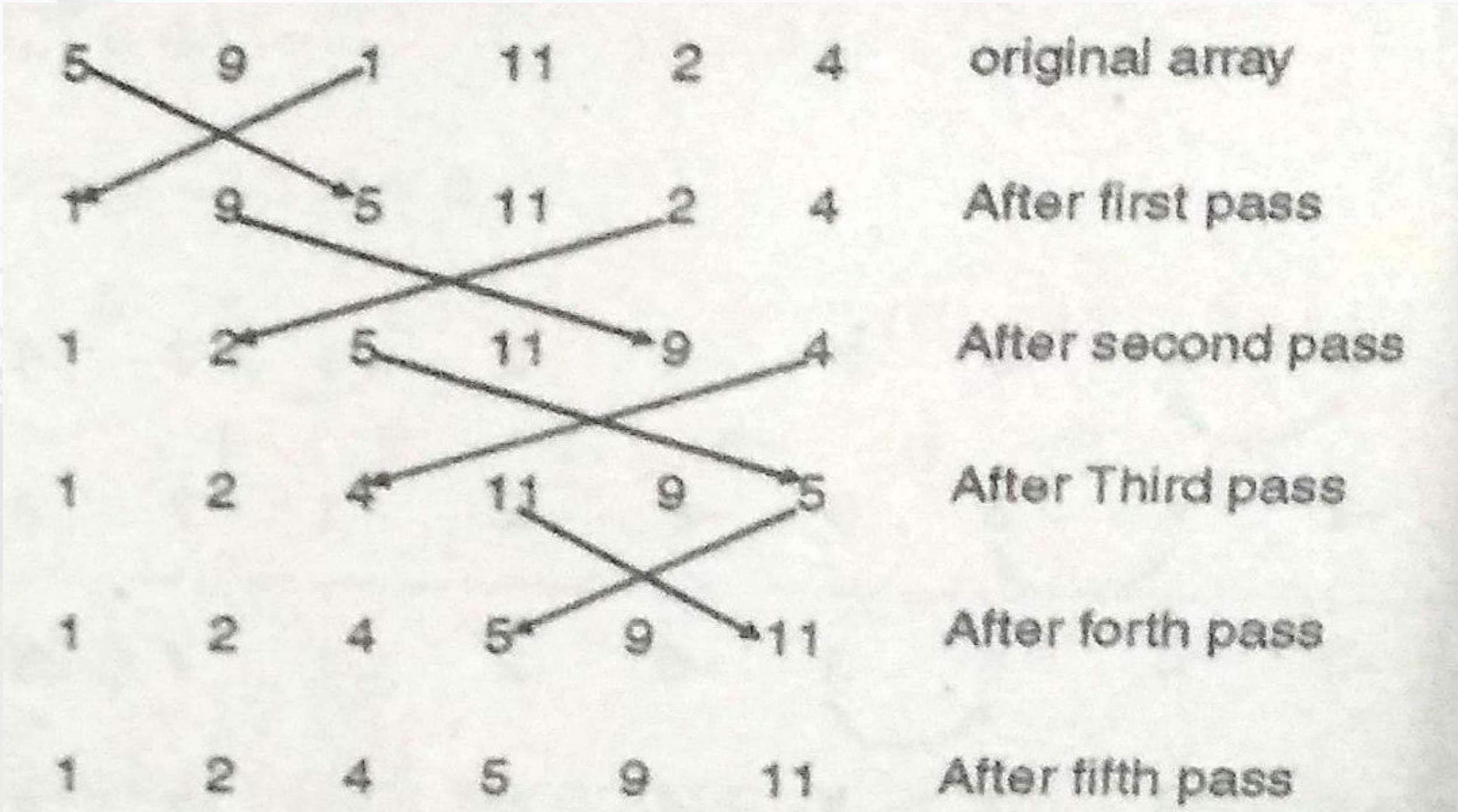
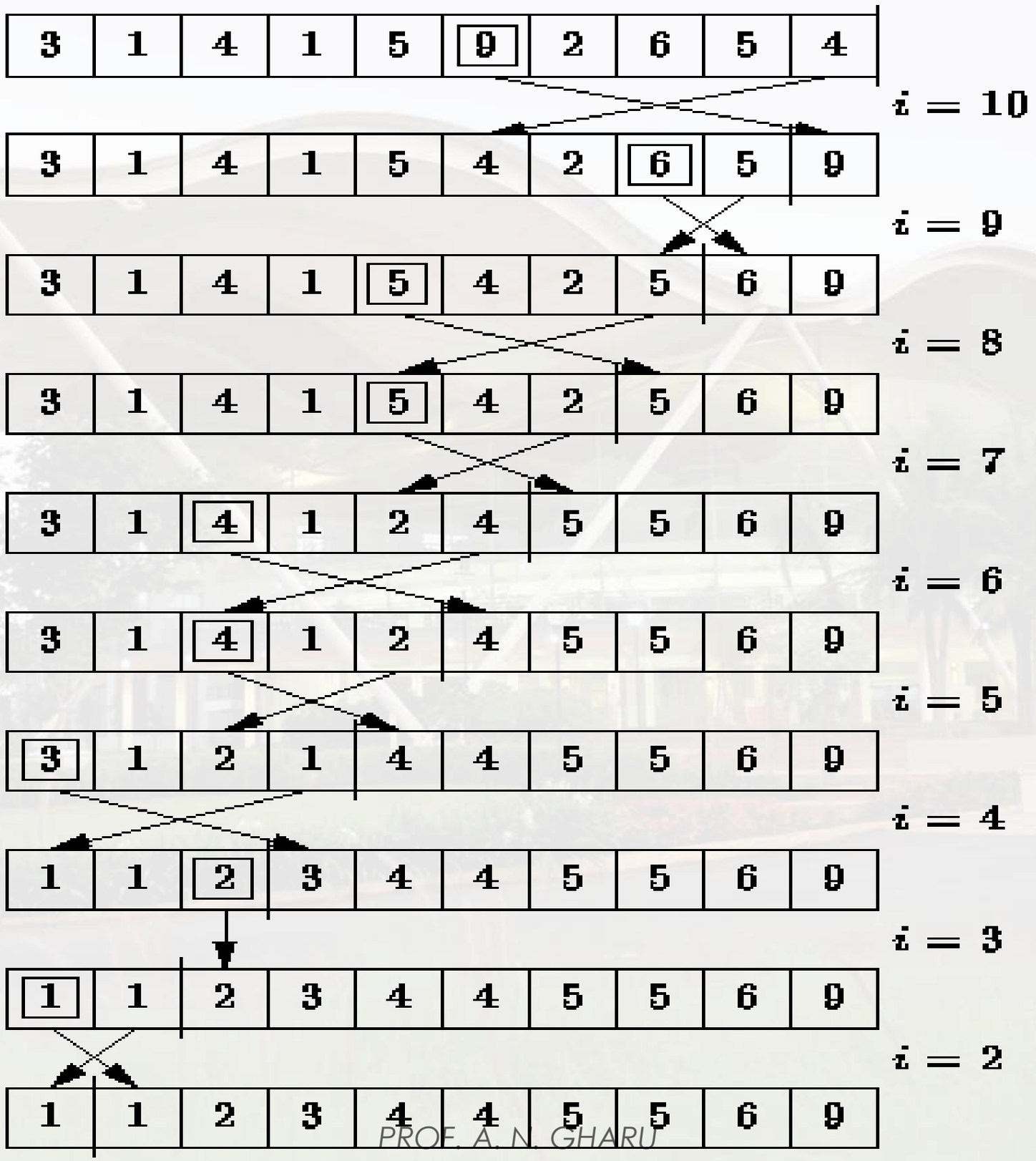


Illustration of selection sort

Selection Sort



Selection Sort

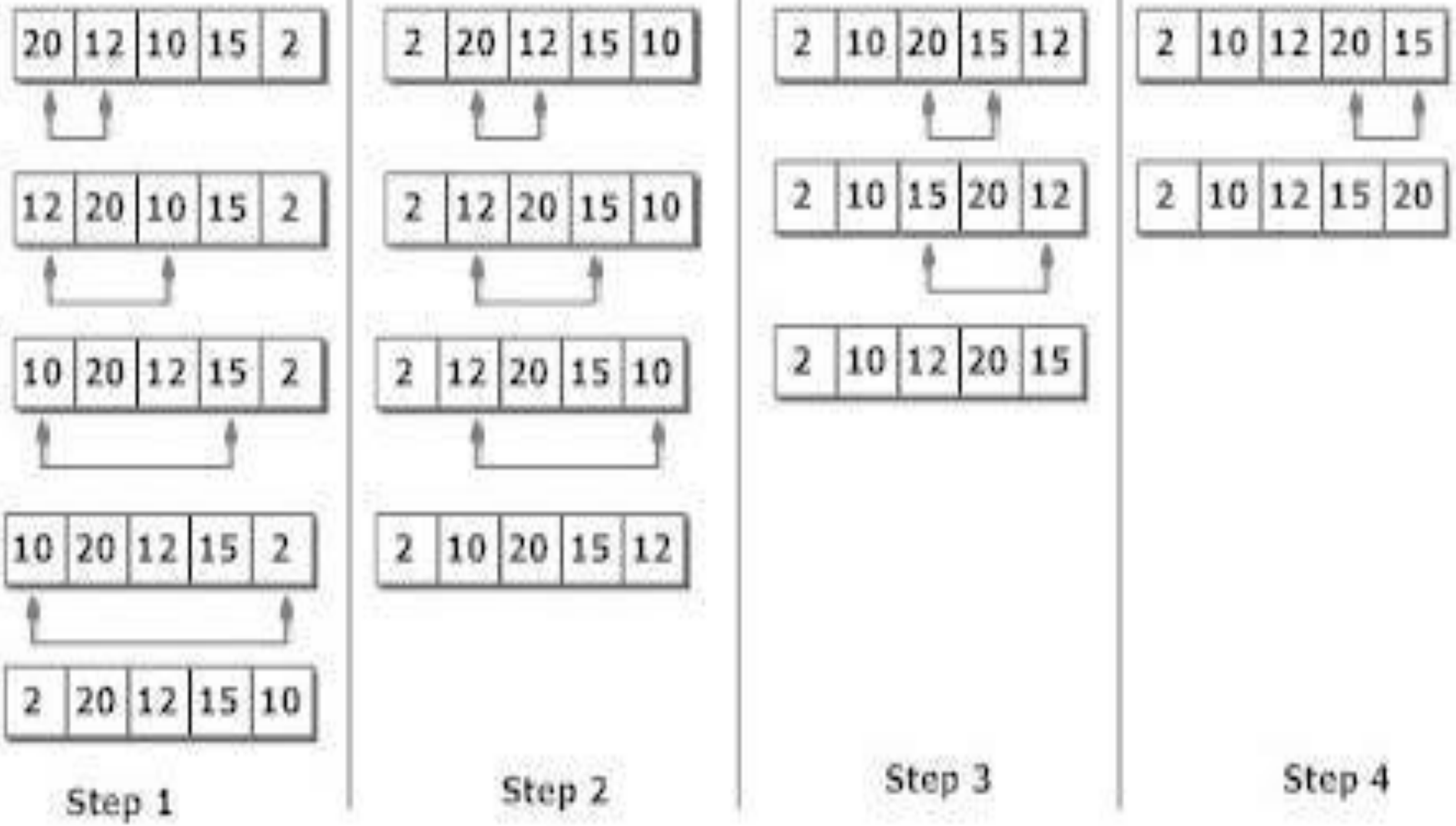


Figure: Selection Sort

Show all the passes to sort the values in descending order :
84, 56, 28, 0, –13, 47, 94, 31.

Solution :

Original array	84 56 28 0 -13 47 94 31
After pass 1	94 56 28 0 -13 47 84 31
After pass 2	94 84 28 0 -13 47 56 31
After pass 3	94 84 56 0 -13 47 28 31
After pass 4	94 84 56 47 -13 0 28 31
After pass 5	94 84 56 47 31 0 28 -13
After pass 6	94 84 56 47 31 28 0 -13
After pass 7	94 84 56 47 31 28 0 -13

Example 8.4.4

Consider the following numbers sort them using "Selection sort". Show the output after each pass. 50, 20, 70, 40, 30

Solution :

Original data	50	20	70	40	30
After Pass I	20	50	70	40	30
After Pass II	20	30	70	40	50
After Pass III	20	30	40	70	50
After Pass IV	20	30	40	50	70

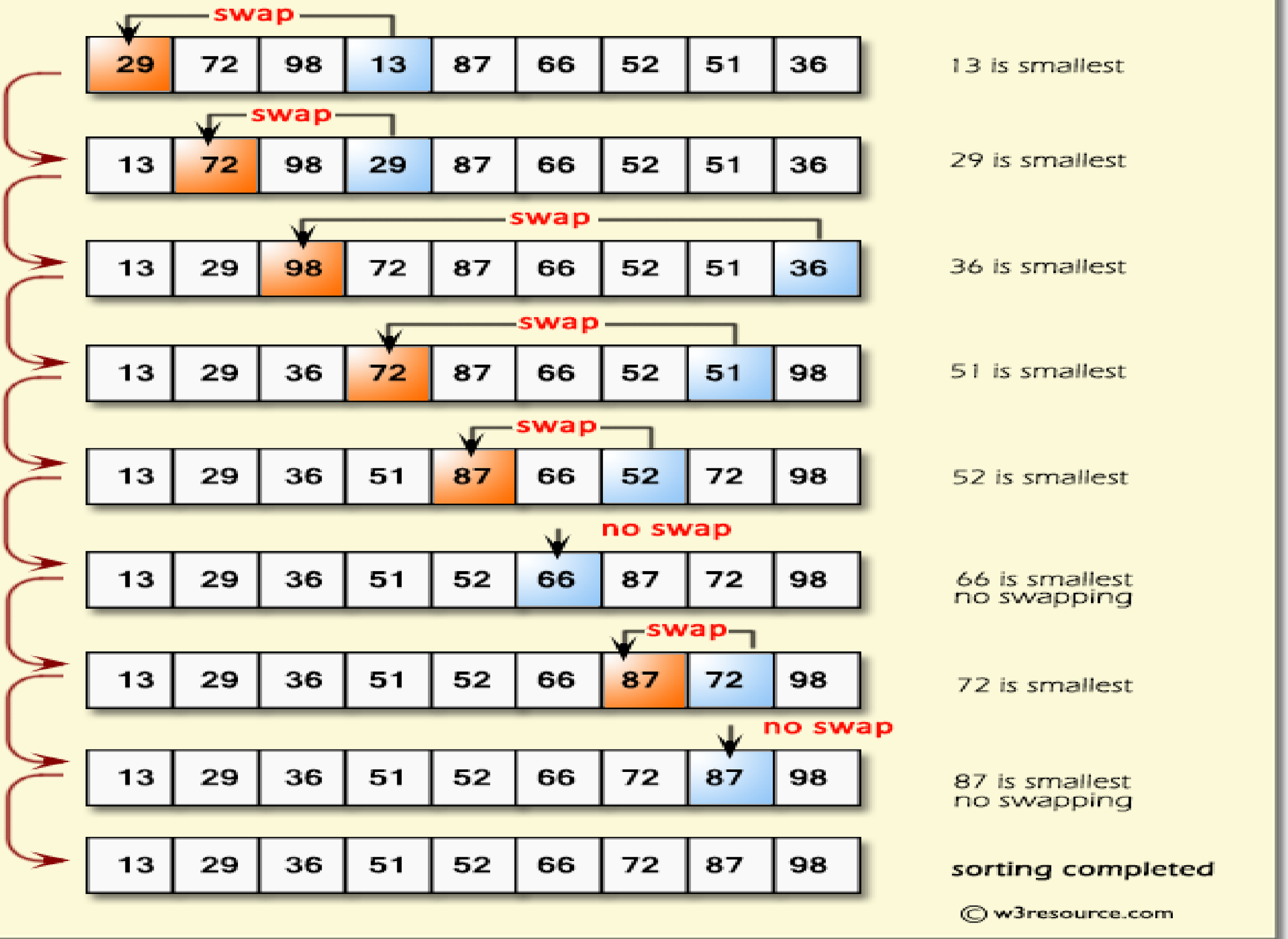
Sort the following list using selection sort. Show output each pass and write time complexity.

Data : 10,6,13,7,5,51,27,2,3,15,-3,4.

Solution :

Pass No.	Data
Initially	10 6 13 7 5 51 27 2 3 15 –3 4
1	–3 6 13 7 5 51 27 2 3 15 10 4
2	–3 2 13 7 5 51 27 6 3 15 10 4
3	–3 2 3 7 5 51 27 6 13 15 10 4
4	–3 2 3 4 5 51 27 6 13 15 10 7
5	–3 2 3 4 5 51 27 6 13 15 10 7
6	–3 2 3 4 5 6 27 51 13 15 10 7
7	–3 2 3 4 5 6 7 51 13 15 10 27
8	–3 2 3 4 5 6 7 10 13 15 51 27
9	–3 2 3 4 5 6 7 10 13 15 51 27
10	–3 2 3 4 5 6 7 10 13 15 51 27
11	–3 2 3 4 5 6 7 10 13 15 27 51

Selection Sort



Quick Sort Algorithm

- Quick sort is based on divide-and-conquer strategy
  - Quick sort is thus in-place, divide-and-conquer based massively recursive sort technique
  - This techniques reduces unnecessary swaps and moves the element at great distance in one move.

https://youtu.be/8hHWpuAPBHo

https://youtu.be/V6pF0Lr5Ykc

https://youtu.be/gKpl-RvVwV4

Quick Sort Algorithm

- **The recursive algorithm consists of four steps:**
  - If there is one or less element in the array to be sorted, return immediately
  - Pick an element in the array to serve as a 'pivot' usually the left-most element in the list)
  - Partition the array into two parts—one with elements smaller than the pivot and the other with elements larger than the pivot by traversing from both the ends and performing swaps if needed
  - Recursively repeat the algorithm for both partitions

PROF. A. V. GOHARU

Quick Sort Algorithm

1. Lowest index element set as pivot element.
2. Take two index variable, i and j. i points to 1st location element and j points to (n-1)th location element.
3. Index variable i is in search of element which is greater than pivot element. Here i will incremented by 1 till greater element is not found.
4. Index variable j is in search of element which is less than pivot element. Here j will be decremented by 1 till small element is not found.
5. If these two elements are found, they are swapped.
6. The process ends when these two variables are crossed or meet (In above example they are crossed). Then value at index j is swapped with pivot and list is divided into 2 sublists.
7. Above steps are repeated on these two sub arrays (sublists) until all sub arrays contain only 1 element.

Quick Sort

Advantages of Quick Sort:

1. It is a divide-and-conquer algorithm that makes it easier to solve problems.
2. It is efficient on large data sets.
3. It has a low overhead, as it only requires a small amount of memory to function.

Disadvantages of Quick Sort:

1. It has a worst-case time complexity of O(N2), which occurs when the pivot is chosen poorly.
2. It is not a good choice for small data sets.
3. It is not a stable sort, meaning that if two elements have the same key, their relative order will not be preserved in the sorted output in case of quick sort, because here we are swapping elements according to the pivot's position (without considering their original positions).

Quick Sort Algorithm

Let the array of number be

0	1	2	3	4	5	6	7	8	9	10	11	12
30	35	10	15	20	34	5	18	6	11	13	26	38

Initially $l = 0$, j = 12, V = a[l] {i.e. V = 30}

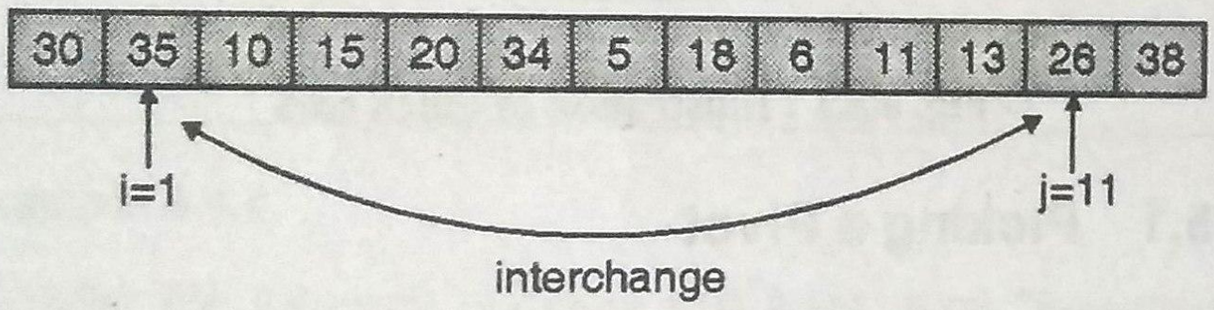
i is set to $l + 1$ and j is set to n – 1 as shown below

30	35	10	15	20	34	5	18	6	11	13	26	38

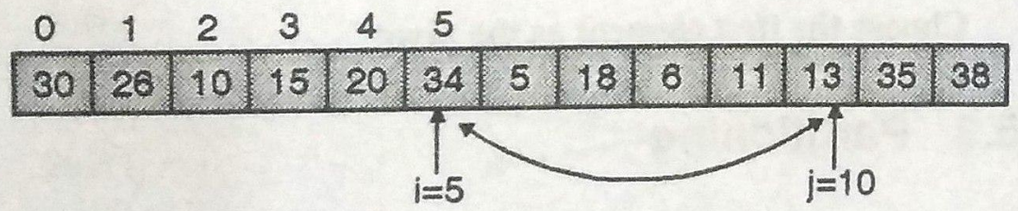
↑ i = 1 ↑ j = 12

Now i moves right, while a[i] < 30. Hence i does not move further to right as a[1] > 30.

Then, j moves left, while a[j] > 30.



At this point a[i] and a[j] are interchanged and the movement of i and j resumes.



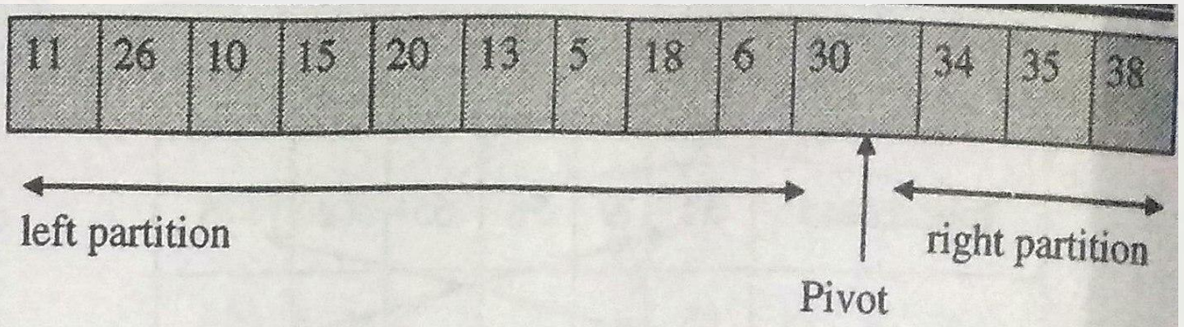
i moves right to a[5] as 34 > 30. j moves left to a[10] as 13 < 30. Once again a[i] and a[j] are swapped.

0	1	2	3	4	5	6	7	8	9	10	11	12
30	26	10	15	20	13	5	18	6	11	34	35	38

j = 9 i = 10

Now, i moves to a[10] as 34 > 30 and j moves to a[9] as 11 < 30.

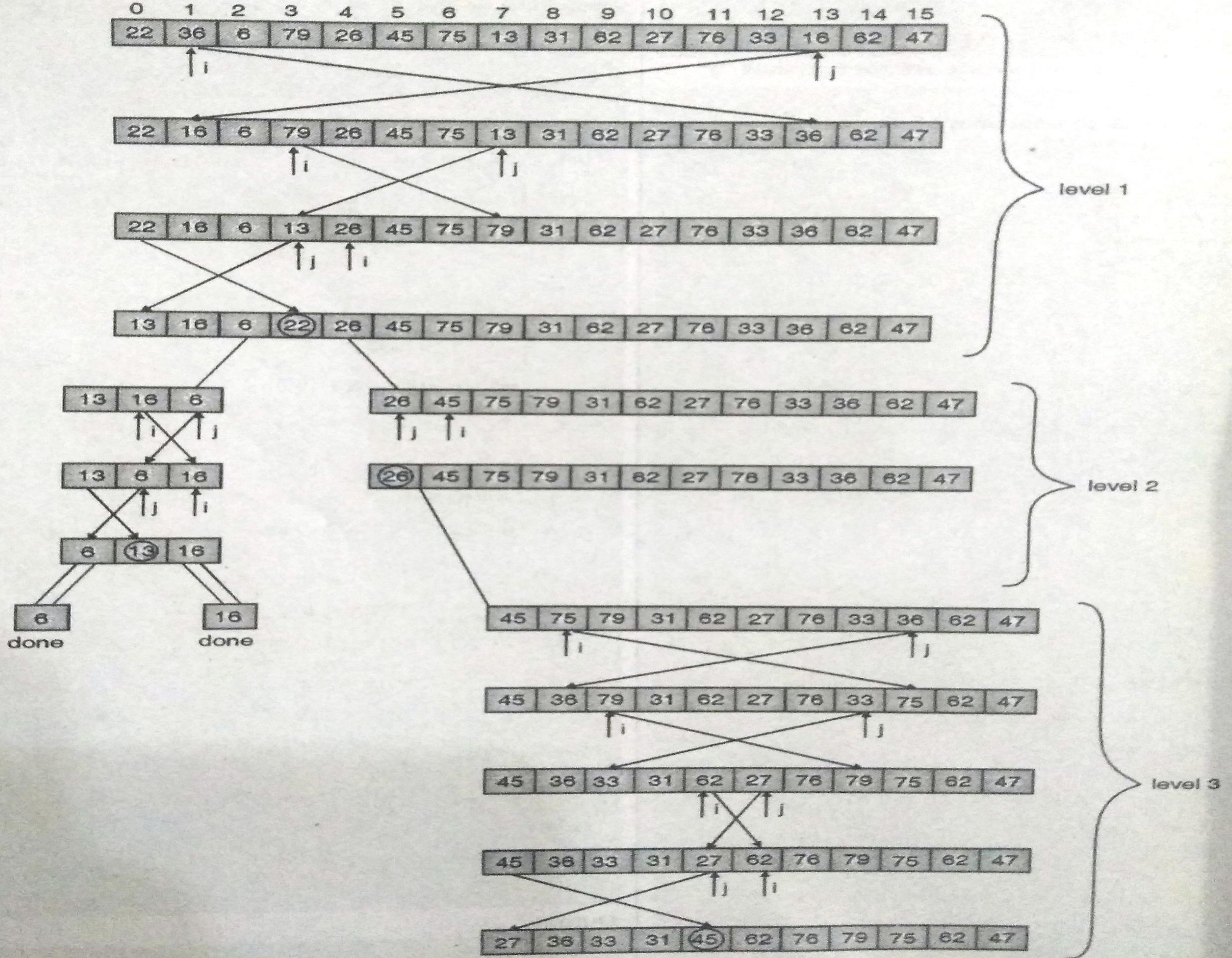
Since, i > j, the process ends. j gives the location of the pivot element. Pivot element a[l with $l = 0$] is interchanged with a[i] to partition the array.

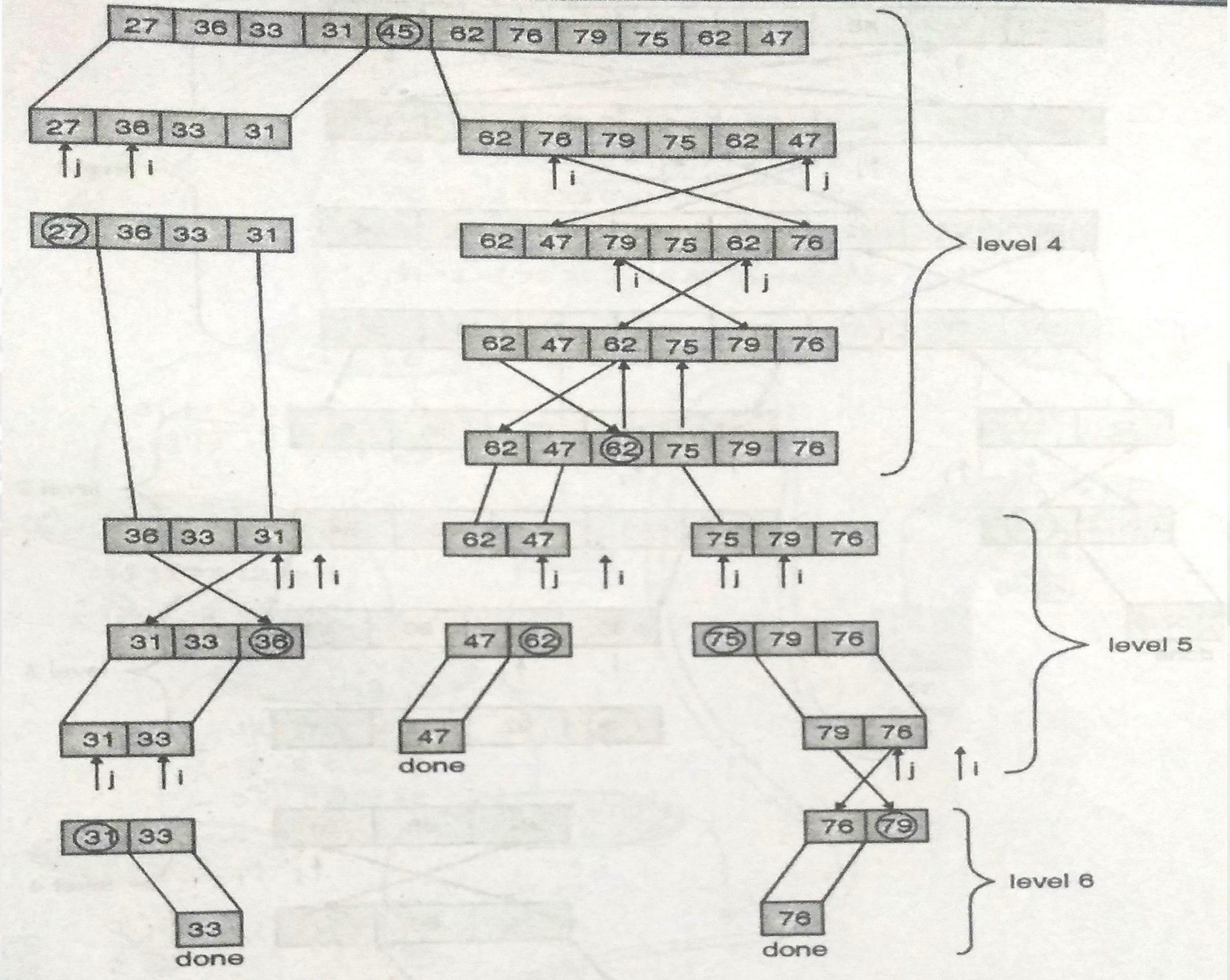


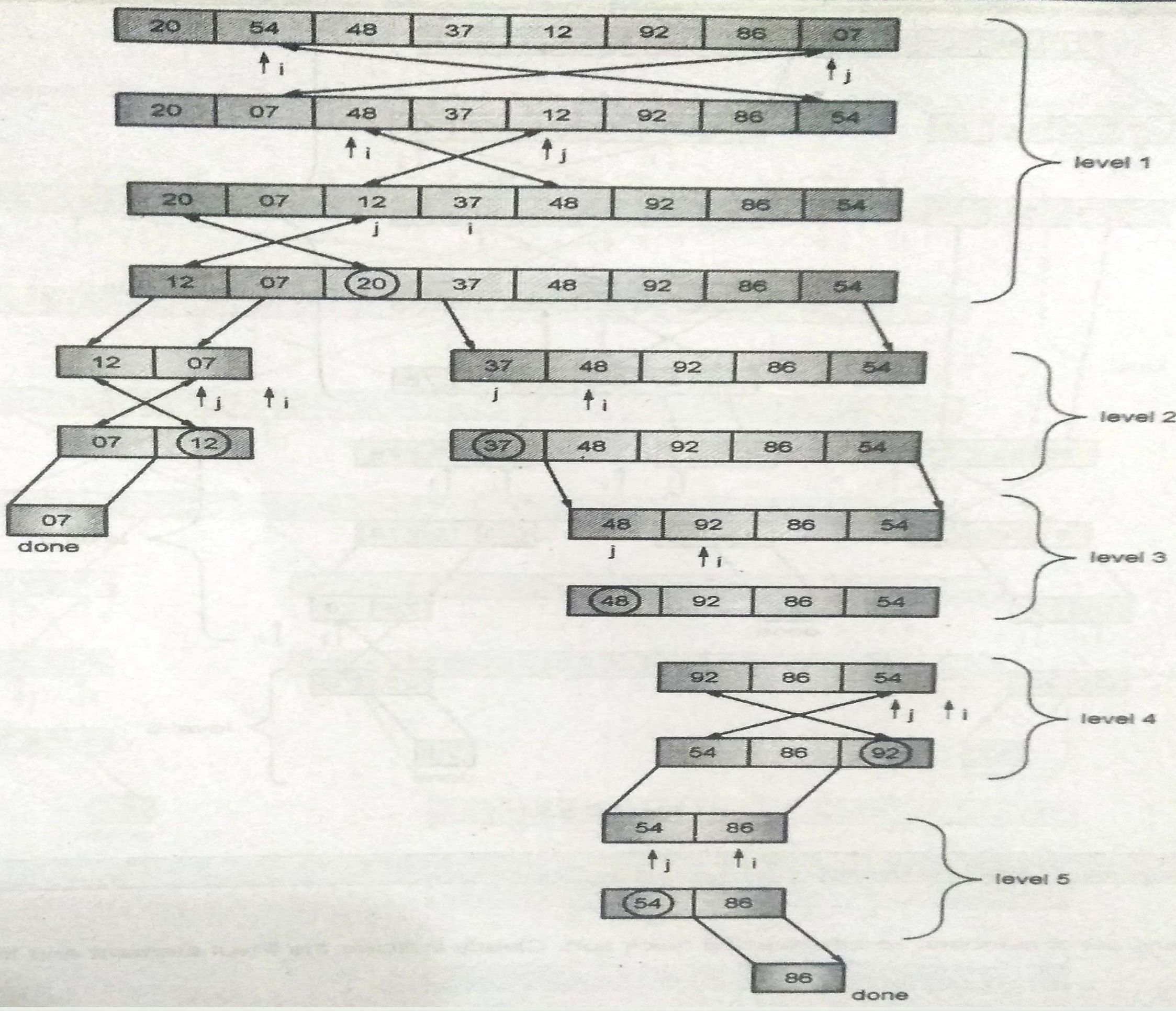
Please note that all element a[i] with $l \le i < j$ are less than 30 and all elements a[j] with $u \ge i > j$ are greater than 30.

Here are sixteen integers : 22, 36, 6, 7, 9, 26, 45, 75, 13, 31,62, 27, 76, 33, 16, 62, 49. Sort them using quick sort.

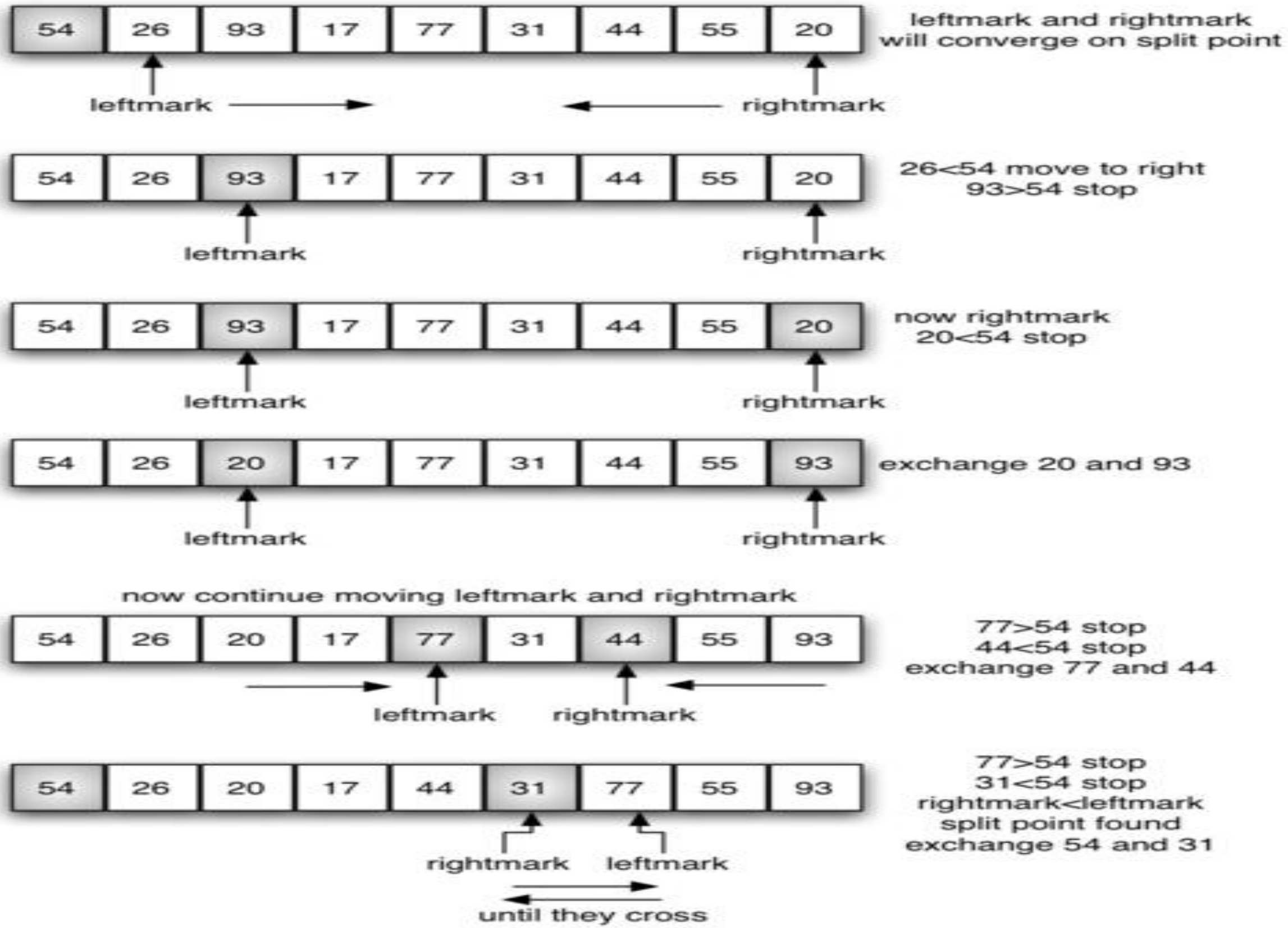
Solution :



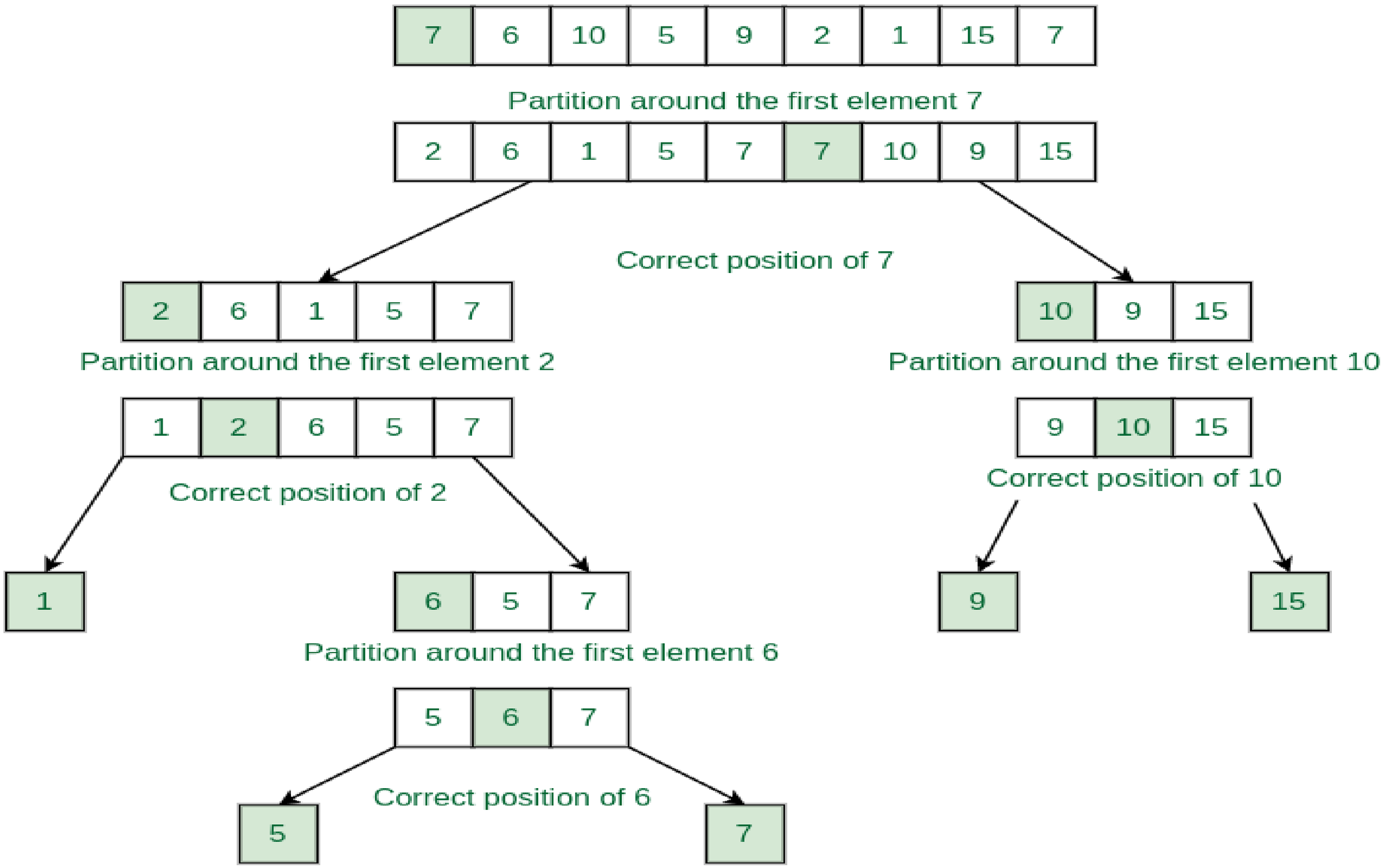

27 36 33 31 45 62 76 79 75 62 47
27 36 33 31
j i
62 76 79 75 62 47
i j
62 47 79 75 62 76
i j
62 47 62 75 79 76
62 47 62 75 79 76
level 4
36 33 31
j i
62 47
j i
75 79 76
j i
31 33 36
47 62
75 79 76
level 5
31 33
j i
47
done
79 76
j i
76 79
level 6
31 33
33
done
76
done


20 54 48 37 12 92 86 07
i
j
20 07 48 37 12 92 86 54
i
j
20 07 12 37 48 92 86 54
j
i
12 07 20 37 48 92 86 54
12 07
j
i
07 12
07
done
37 48 92 86 54
j
i
37 48 92 86 54
48 92 86 54
j
i
48 92 86 54
level 1
level 2
level 3
92 86 54
j
i
54 86 92
54 86
j
i
54 86
86
done
level 4
level 5

Quick Sort Algorithm



Quick Sort Algorithm



Merge Sort Algorithm

- *Merge sort is a sorting technique based on divide and conquer technique. With Average case and worst-case time complexity being O(n log n), it is one of the most respected algorithms.*
- *Merge sort first divides the array into equal halves and then combines them in a sorted manner.*

Merge Sort

- The most common algorithm used in external sorting is the merge sort
- Merging is the process of combining two or more sorted files into the third sorted file
- We can use a technique of merging two sorted lists
- Divide and conquer is a general algorithm design paradigm that is used for merge sort

Merge Sort

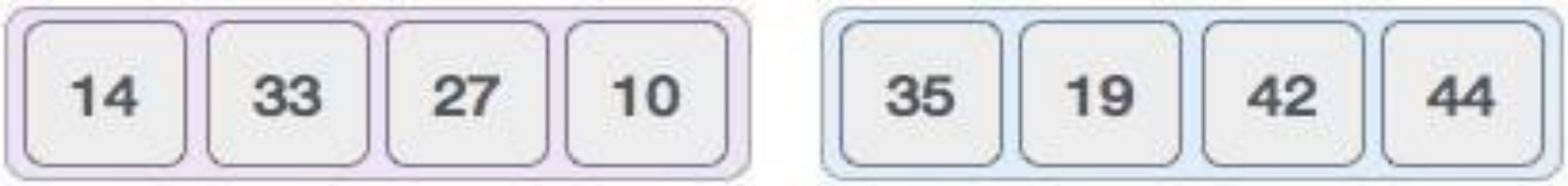
- *Time Complexity $T(n) = O(n \log n)$*

How merge sort works

- *To understand merge sort, we take a unsorted array as depicted below –*

- *We know that merge sort first divides the whole array iteratively into equal halves unless the atomic value`s are achieved. We see here that an array of 8 items is divided into two arrays of size 4.*

https://www.youtube.com/watch?v=HsZ-YRQM8sE

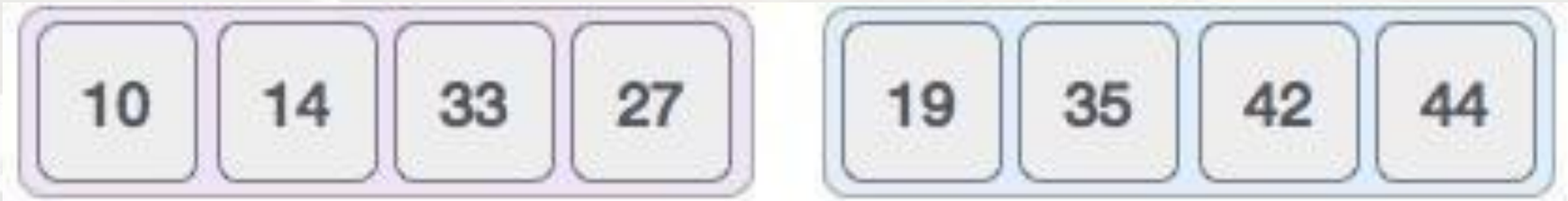


- *This does not change the sequence of appearance of items in the original. Now we divide these two arrays into halves.*

- *We further divide these arrays and we achieve atomic value which can no more be divided.*

- *Now, we combine them in exactly same manner they were broken down.*
- *We first compare the element for each list and then combine them into another list in sorted manner. We see that 14 and 33 are in sorted positions. We compare 27 and 10 and in the target list of 2 values we put 10 first, followed by 27. We change the order 19 and 35. 42 and 44 are placed sequentially.*

- *In next iteration of combining phase, we compare lists of two data values, and merge them into a list of four data values placing all in sorted order.*



- *After final merging, the list should look like this –*

Algorithm of merge sort

- *Merge sort keeps on dividing the list into equal halves until it can no more be divided. By definition, if it is only one element in the list, it is sorted. Then merge sort combines smaller sorted lists keeping the new list sorted too.*
  - ***Step 1 –*** *divide the list recursively into two halves until it can no more be divided.*
  - ***Step 2 –*** *if it is only one element in the list it is already sorted, return.*
  - ***Step 3 –*** *merge the smaller lists into new list in sorted order.*

Merge Sort

Advantages of Merge Sort:

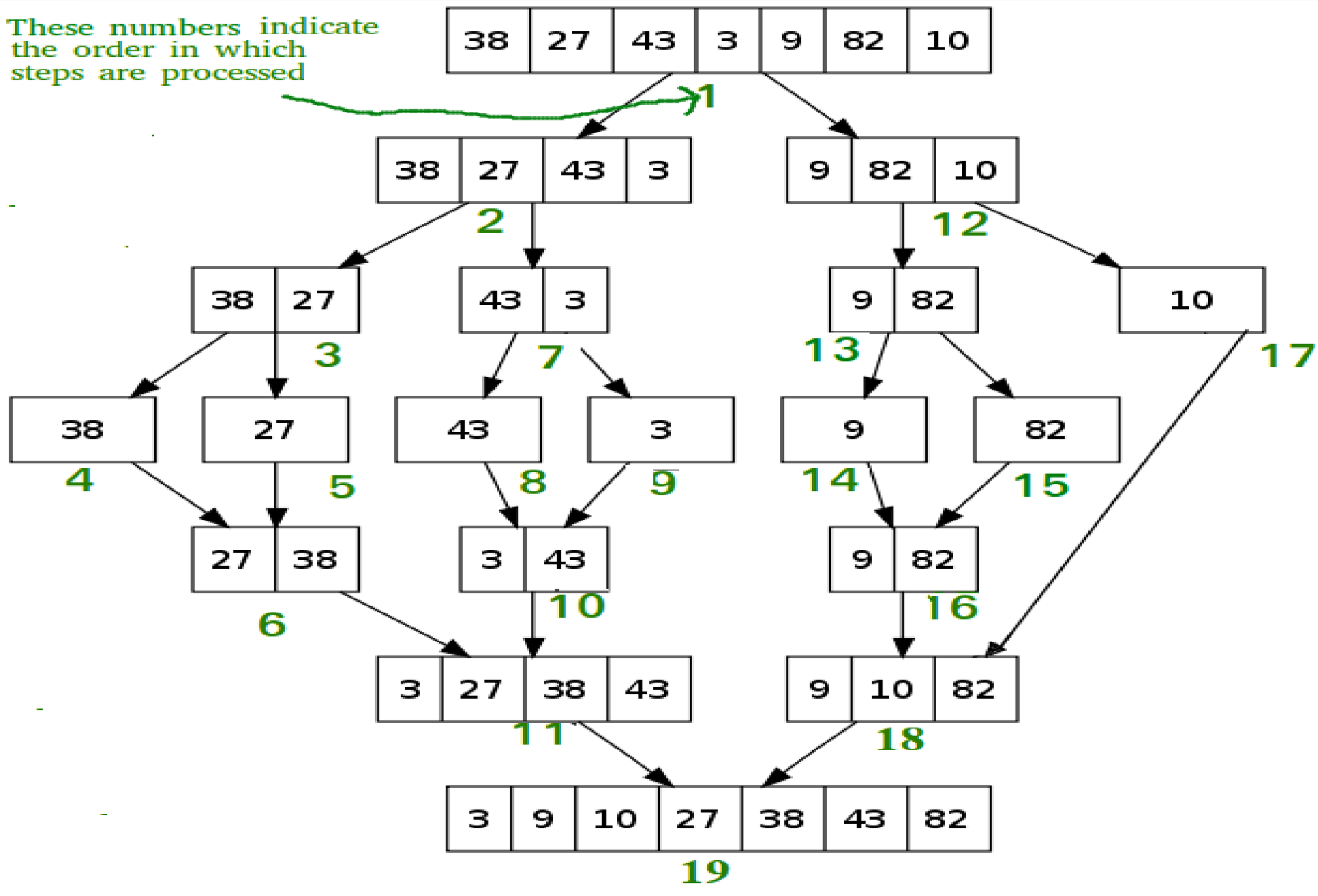
- **Stability:** Merge sort is a stable sorting algorithm, which means it maintains the relative order of equal elements in the input array**.**
- **Guaranteed worst-case performance:** Merge sort has a worst-case time complexity of O(N logN), which means it performs well even on large datasets.
- **Parallelizable:** Merge sort is a naturally parallelizable algorithm, which means it can be easily parallelized to take advantage of multiple processors or threads.

Drawbacks of Merge Sort:

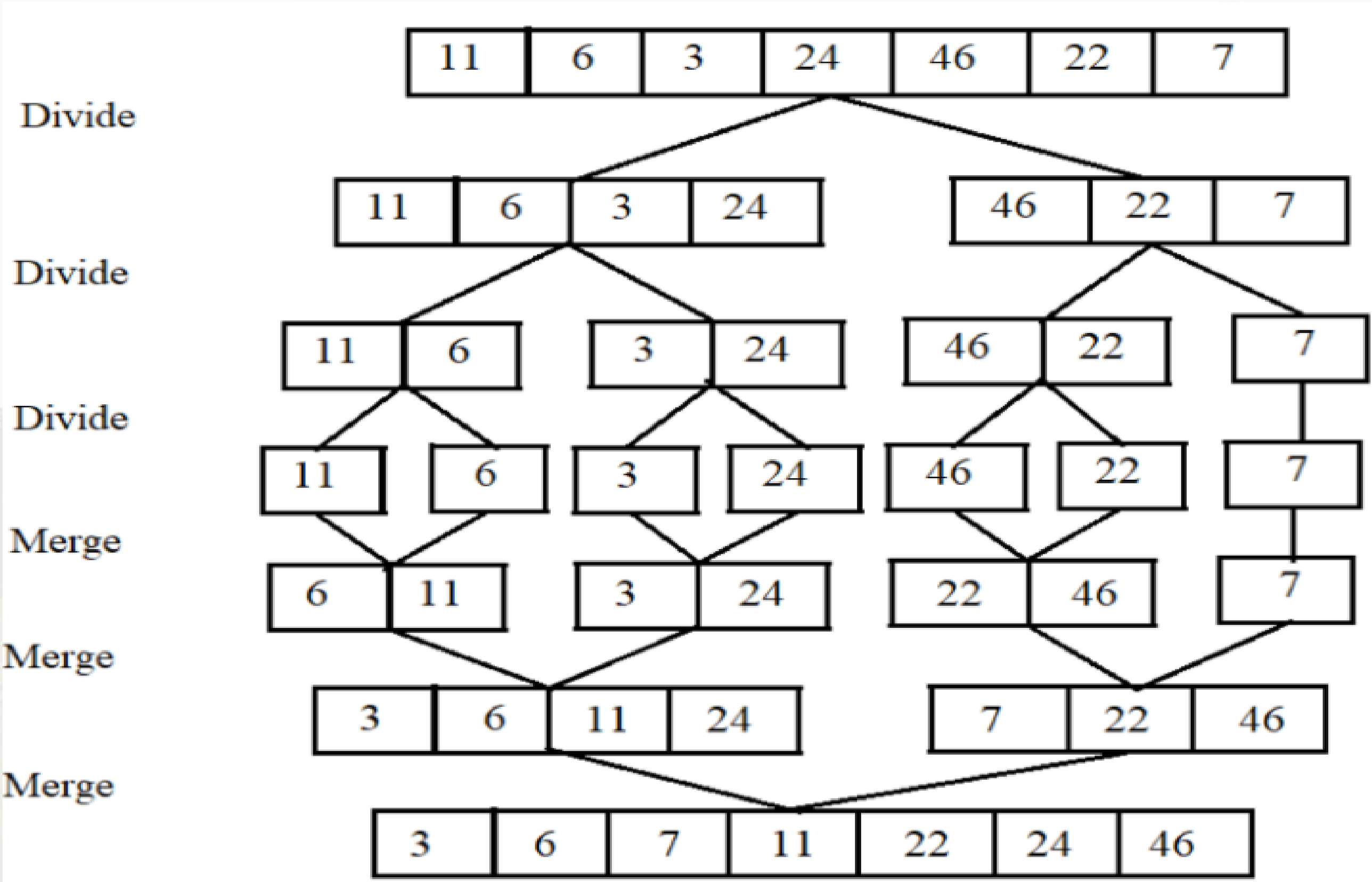
1. Space complexity:
2. Not in-place:
3. Not always optimal for small datasets:

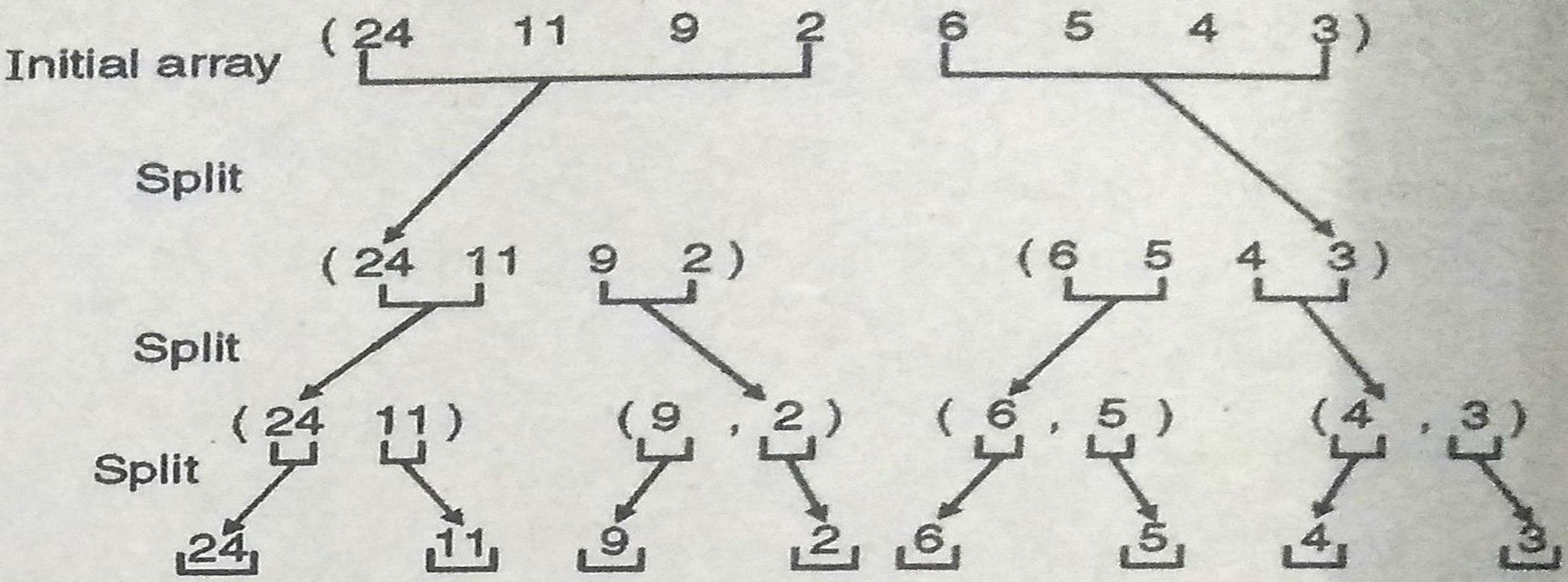
Merge Sort Examples

Merge Sort Examples



Merge Sort Examples




Initial array
(24 11 9 2 6 5 4 3)
Split
(24 11 9 2) (6 5 4 3)
Split
(24 11) (9 , 2) (6 , 5) (4 , 3)
Split
24 11 9 2 6 5 4 3
Merge
11 , 24 2 , 9 5 , 6 4 , 3
Merge
2, 9, 11, 24 3, 4, 5, 6
Merge
2 , 3 , 4 ,5 6, 9, 11, 24

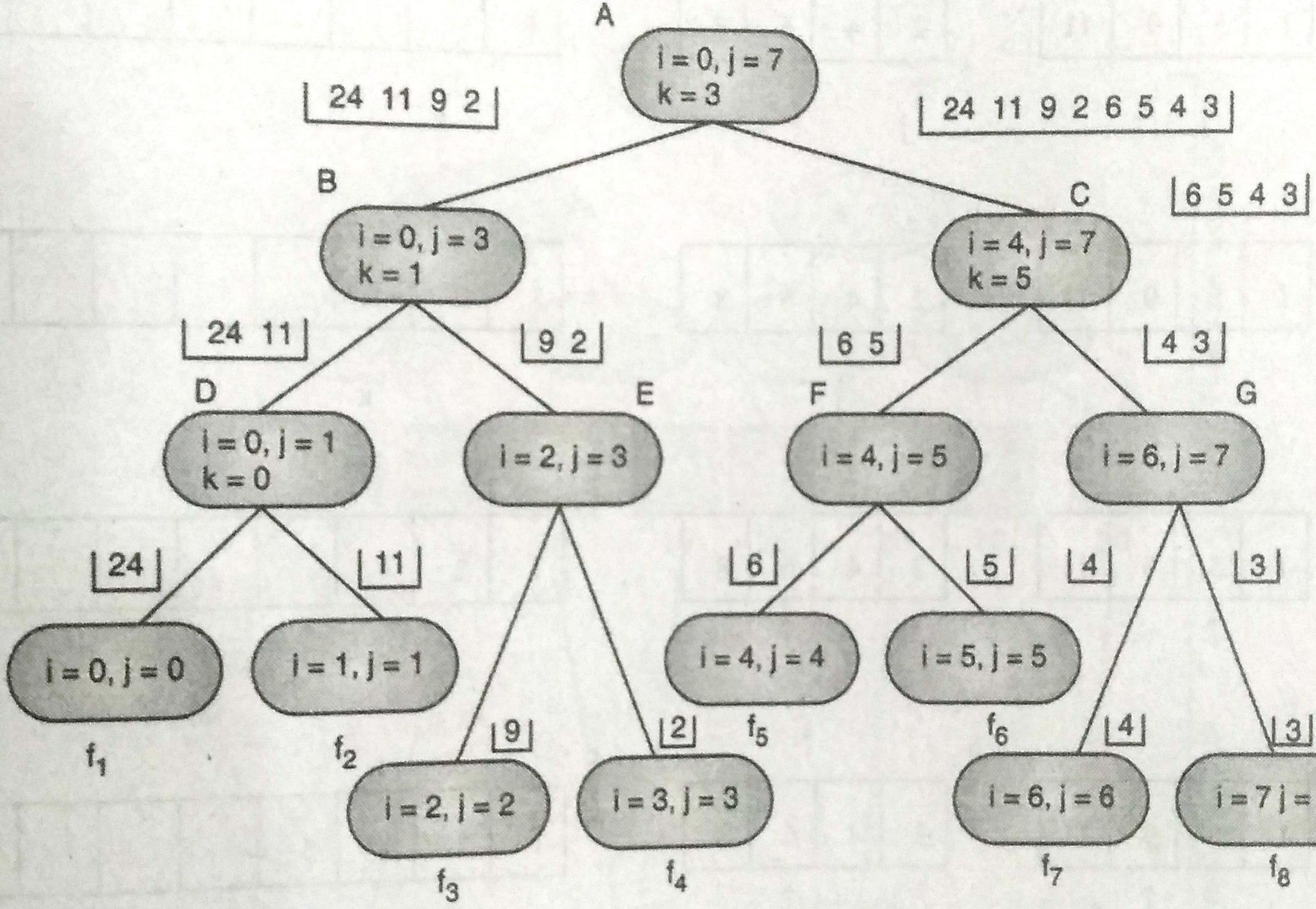
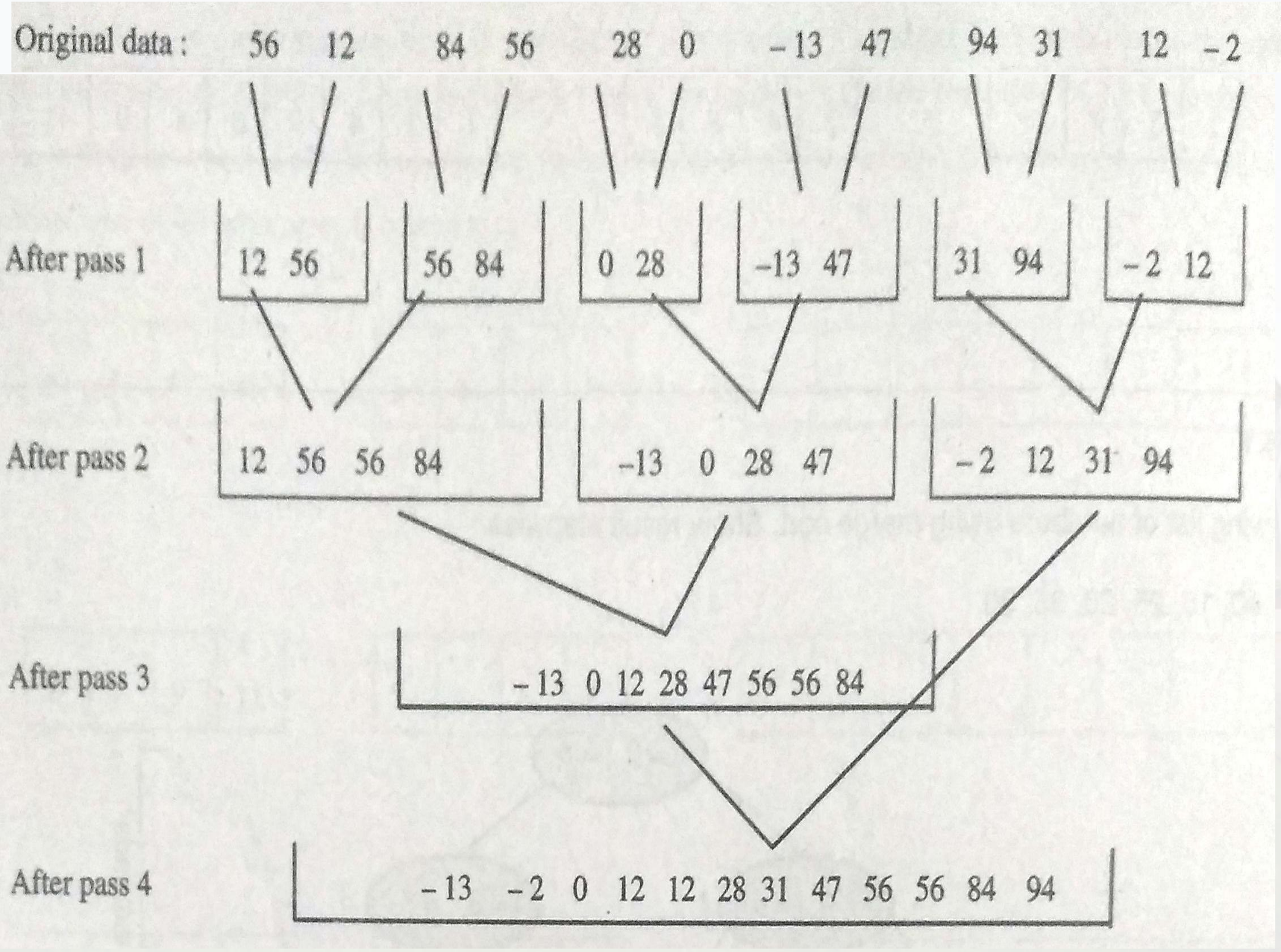
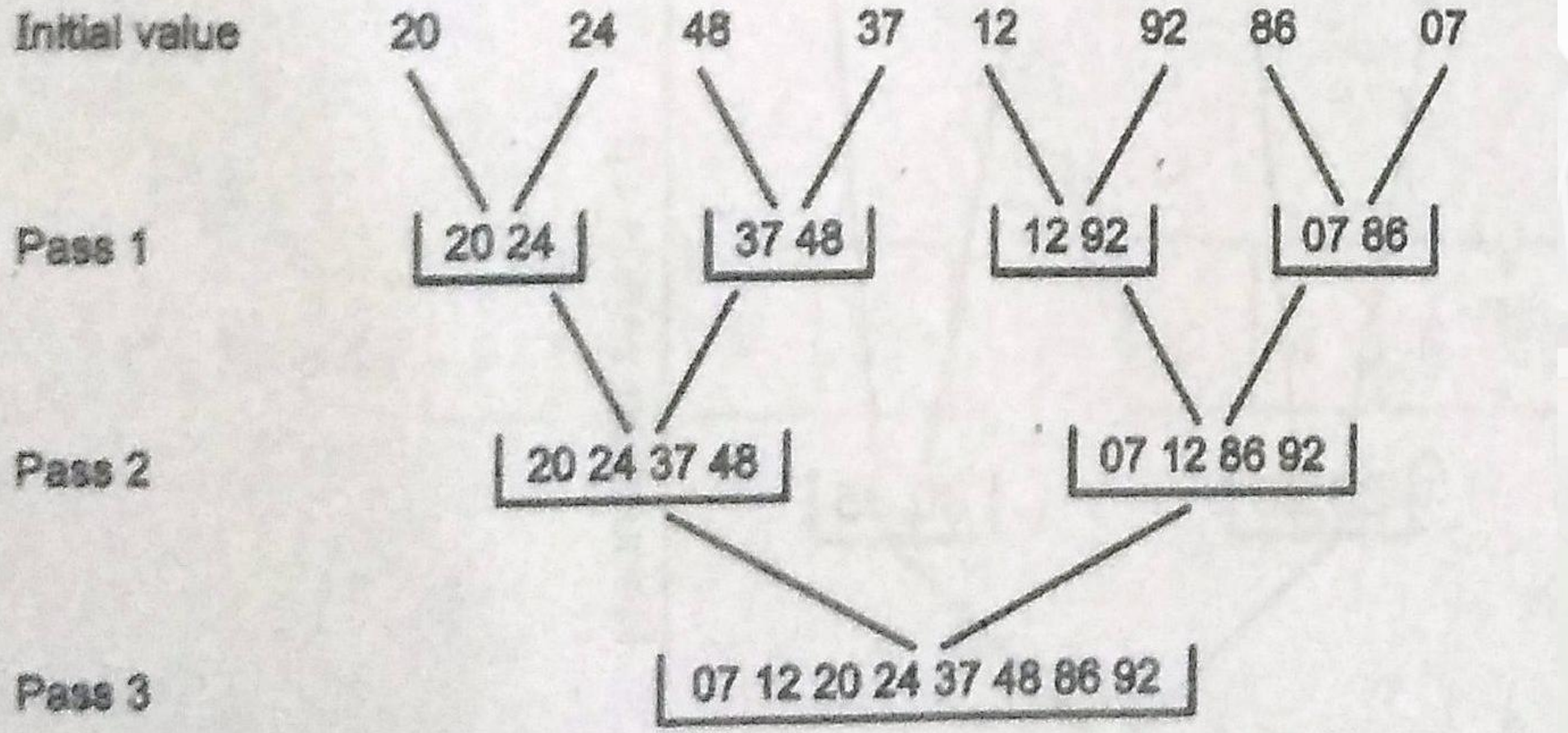

A
i = 0, j = 7
k = 3
24 11 9 2
24 11 9 2 6 5 4 3
B
i = 0, j = 3
k = 1
C
6 5 4 3
i = 4, j = 7
k = 5
24 11
9 2
6 5
4 3
D
E
F
G
i = 0, j = 1
k = 0
i = 2, j = 3
i = 4, j = 5
i = 6, j = 7
24
11
6
5
4
3
i = 0, j = 0
i = 1, j = 1
i = 4, j = 4
i = 5, j = 5
f1
f2
f5
f6
9
2
4
3
i = 2, j = 2
i = 3, j = 3
i = 6, j = 6
i = 7 j = 7
f3
f4
f7
f8

Fig. 8.6.2


Original data : 56 12 84 56 28 0 -13 47 94 31 12 -2
After pass 1
12 56
56 84
0 28
-13 47
31 94
-2 12
After pass 2
12 56 56 84
-13 0 28 47
-2 12 31 94
After pass 3
-13 0 12 28 47 56 56 84
After pass 4
-13 -2 0 12 12 28 31 47 56 56 84 94

Consider the following set of numbers, sort them using iterative merge sort. Show all passes

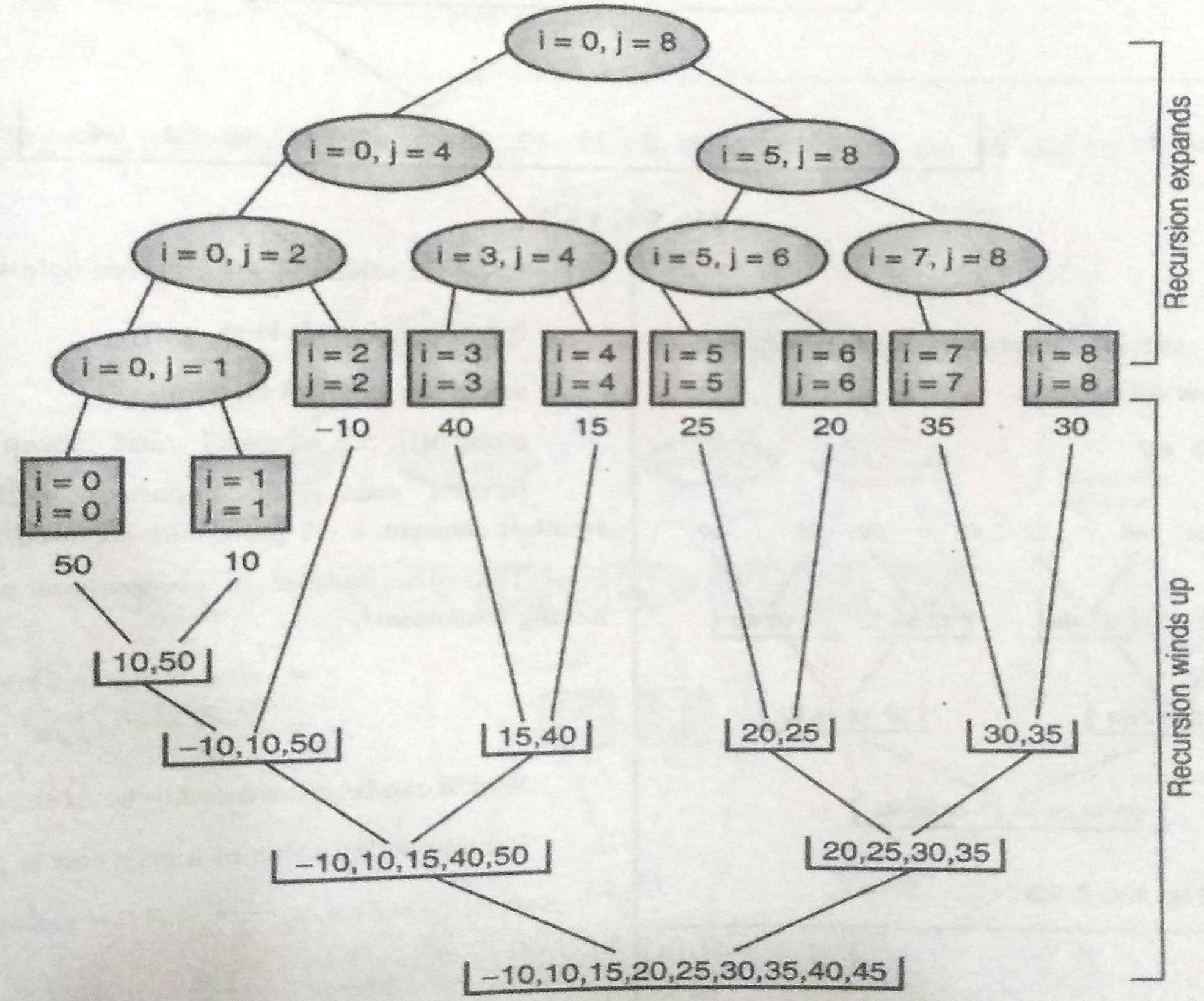
20 24 48 37 12 92 86 07



Sort the following list of numbers using merge sort. Show result stepwise :

50, 10, – 10, 40, 15, 25, 20, 35, 30

Solution :



Comparisoin of sorting

	Best-case	Worst-case	Average-case
Bubble sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$
Quick sort	$O(n \log n)$	$O(n^2)$	$O(n \log n)$
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Radix sort	$O(n)$	$O(n)$	$O(n)$

PROF. A. N. GHARU

Comparisoin of sorting

III. COMPARISON OF SORTING ALGORITHM IN TABULAR FORM

Sort	*Time Complexity*	*Advantages & disadvantages*
Insertion Sort	O(n)	The advantage of insertion sort is its simplicity. It is also good performance for smallest array. The disadvantage of insertion sort is that it is not useful for large elements array.
Selection Sort	O(n^2)	The advantage of selection sort is that it performs well on small array. The disadvantage of selection is that it is poor efficiency for large elements array.
Bubble Sort	O(n^2)	The advantage of bubble sort is that it is easily implemented. In bubble sort, the elements are swapped without additional temporary storage, so space requirement is minimum. The disadvantage of bubble sort is same as a selection sort.
Quick Sort	O(n log n)	The advantage of Quick sort is that it is used for small elements of array as well as large elements of array. Disadvantage of Quick sort is that the worst case of quick sort is same as a bubble sort or selection sort.

Comparisoin of sorting

BASIS FOR COMPARISON	BUBBLE SORT	SELECTION SORT
Basic	Adjacent element is compared and swapped	Largest element is selected and swapped with the last element (in case of ascending order).
Best case time complexity	O(n)	$O(n^2)$
Efficiency	Inefficient	Improved efficiency as compared to bubble sort
Stable	Yes	No
Method	Exchanging	Selection
Speed	Slow	Fast as compared to bubble sort

REFERENCES

1. https://dotnettutorials.net/lesson/array-abstract-data-type-in-c/
2. https://www.tutorialspoint.com/adt-array-representation-in-data-structure
3. https://www.codingninjas.com/studio/library/c-program-to-merge-two-sorted-array-in-ascending-order
4. https://quescol.com/interview-preparation/merge-two-arrays-python
5. https://www.faceprep.in/data-structures/introduction-to-arrays/
6. https://www.geeksforgeeks.org/calculation-of-address-of-element-of-1-d-2-d-and-3-d-using-row-major-and-column-major-order/
7. https://www.w3schools.in/data-structures/polynomials-using-linked-list-and-arrays#:~:text=Representation%20of%20Polynomials%20Using%20Arrays&text=The%20simple%20way%20is%20to,Exponent
8. https://www.bartleby.com/subject/engineering/computer-science/concepts/representation-of-polynomial
9. https://www.geeksforgeeks.org/sparse-matrix-representation/
10. https://byjus.com/maths/transpose-of-a-matrix/
11. https://www.youtube.com/watch?v=zXioPdQXf9A

References

1. https://www.geeksforgeeks.org/searching-algorithms/
2. https://www.javatpoint.com/linear-search
3. https://www.geeksforgeeks.org/c-program-for-sentinel-linear-search/
4. https://www.geeksforgeeks.org/binary-search/
5. https://www.tutorialspoint.com/design_and_analysis_of_algorithms/design_and_analysis_of_algorithms_fibonacci_search.htm
6. https://www.geeksforgeeks.org/sorting-algorithms/
7. https://www.computersciencebytes.com/sorting-algorithms/bubble-sort/
8. https://medium.com/austins-software-engineering-journey/insertion-sort-ea0645cc5a23
9. https://iq.opengenus.org/insertion-sort-analysis/#google_vignette
10. https://book.huihoo.com/data-structures-and-algorithms-with-object-oriented-design-patterns-in-c++/html/page502.html
11. https://www.geeksforgeeks.org/implement-quicksort-with-first-element-as-pivot/

References

12. https://www.programiz.com/dsa/merge-sort
13. https://www.geeksforgeeks.org/time-and-space-complexity-analysis-of-merge-sort/
14. https://www.shiksha.com/online-courses/articles/shell-sort-advantages-and-disadvantages/

THANK YOU!!!

My Blog : https://anandgharu.wordpress.com/

Email : gharu.anand@gmail.com